



MODUL PRAKTIKUM MATA KULIAH

Analitika Perkotaan

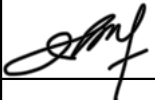
Semester Ganjil Tahun Akademik 2026/2027

Prof. Ibnu Syabri, B.Sc., M.Sc., Ph.D
Muhammad Abdul Mubdi Bindar, S.T., M.T.
Ahmad Luky Ramdani, S.Komp., M.Kom.
Valendya Rilansari, S.P.W.K., M.P.W.K.
Chania Rahmah, S.P.W.K., M.Sc.

Program Studi Perencanaan Wilayah dan Kota
Fakultas Teknologi Infrastruktur dan Kewilayahan
Institut Teknologi Sumatera
2026

	INSTITUT TEKNOLOGI SUMATERA	Nomor :
		Tanggal:
	PENGESAHAN MODUL PRAKTIKUM ANALITIKA PERKOTAAN	Revisi :
		Halaman:

MODUL PRAKTIKUM MATA KULIAH ANALITIKA PERKOTAAN
 PROGRAM STUDI PERENCANAAN WILAYAH DAN KOTA
 FAKULTAS TEKNOLOGI INFRASTRUKTUR DAN KEWILAYAHAN
 INSTITUT TEKNOLOGI SUMATERA

PROSES	PENANGGUNG JAWAB			TANGGAL
	Nama	Jabatan	Tanda Tangan	
Disusun oleh	Muhammad Abdul Mubdi Bindar, S.T., M.T.	Dosen Penanggung Jawab		13 Juli 2026
Diperiksa oleh	Chania Rahmah, S.P.W.K., M.Sc.	Gugus Kendali Mutu Prodi		20 Juli 2026
Pengesahan	Dr. nat. techn. Shahnaz Nabila Fuady, S.T., M.T.	Koordinator Program Studi Perencanaan Wilayah dan Kota		27 Juli 2026

Daftar Isi

Kata Pengantar	1
Cara Memakai Modul Ini	1
Dataset	1
Perangkat Lunak	1
1 Fondasi Lingkungan Kerja: Python <i>Offline</i> , Terminal, dan Direktori Proyek	2
1.1 Pendahuluan	2
1.2 Gambaran Umum Kasus	2
1.3 Instruksi	2
1.3.a Menata Meja Kerja: Menyiapkan Alat-alat Sesuai Kebutuhan	2
1.3.b Mendata Python yang Sudah Ada	5
1.3.c Memasang dan Mengaktifkan Conda	7
1.3.d Berbicara dengan Terminal: Menjalankan Perintah Python Interaktif	13
1.3.e Menulis Catatan Kerja: Membuat Skrip <code>sapa.py</code> dan Menjalankannya	15
1.3.f Menulis Perintah di Python	17
1.3.g Membaca Pesan Kegagalan: Tiga <i>Error</i> Paling Umum dan <i>Debugging</i> Dasar	18
1.3.h Membandingkan Dua Berkas: Mengapa Satu File Harus Lengkap-Mandiri	20
1.3.i Menyiapkan Buku Lapangan: Membuat Jupyter Notebook dan Menyambungkan <i>Kernel</i>	21
1.4 Latihan Mandiri	22
2 Pengolahan Data Terstruktur dengan Pandas	24
2.1 Pendahuluan	24
2.2 Gambaran Umum Kasus	24
2.3 Instruksi	24
2.3.a Jembatan Jupyter: Membuktikan Perilaku Sel dan <i>State</i> Bersama	24
2.3.b Eksperimen Papan Tulis: Mengapa Urutan Eksekusi Sel Sangat Menentukan Hasil . . .	25
2.3.c Membuka Rekap Data: Membaca File CSV dengan Pandas	26
2.3.d Mengenal Struktur Tabel: Anatomi Baris, Kolom, dan Tipe Data <i>DataFrame</i>	28
2.3.e Kelurahan vs Kecamatan: Membedakan Unit Amatan dan Unit Analisis	30
2.3.f Merapikan Tabel: Membuat, Menyusun Ulang, dan Menghapus Kolom	31
2.3.g Menyaring Baris: Filtrasi Data Kelurahan Berdasarkan Kriteria Tertentu	33
2.4 Latihan Mandiri	36
3 Pengolahan Data Terstruktur dengan SQL (SQLite)	37
3.1 Pendahuluan	37
3.2 Gambaran Umum Kasus	37
3.3 Instruksi	37
3.3.a Lemari Arsip Digital: Pengenalan SQLite dan Koneksi Database	37
3.3.b Menyusun Laci Data: Membuat Tabel dan Mengisi Data dari Pandas	42
3.3.c Kueri Dasar: Mengambil Data dengan <code>SELECT</code> dan Menyaring dengan <code>WHERE</code>	44
3.3.d Agregasi Kota: Mengelompokkan Data dengan <code>GROUP BY</code>	47
3.3.e Jembatan SQL-Pandas: Membaca Hasil Kueri ke <i>DataFrame</i> Pandas	49
3.4 Latihan Mandiri	50
4 <i>Exploratory Data Analysis</i> (EDA)	51
4.1 Pendahuluan	51
4.2 Gambaran Umum Kasus	51
4.3 Instruksi	51

4.3.a	Data Kategorikal: Melihat Frekuensi dan Modus dengan <code>value_counts()</code>	54
4.3.b	Menemukan Keanehan: Deteksi <i>Outlier</i> (Nilai Ekstrem) Menggunakan Metode IQR . .	55
4.3.c	Mengisi Lembar Kosong: Identifikasi dan Penanganan <i>Missing Values</i>	58
4.3.d	Hubungan Awal: Analisis Korelasi Kasar Antarvariabel Numerik	61
4.3.e	Sintesis EDA: Menarik Kesimpulan Karakteristik Sosio-Demografi Kota	62
4.4	Latihan Mandiri	63
5	Visualisasi Data dengan Matplotlib dan Seaborn	65
5.1	Pendahuluan	65
5.2	Gambaran Umum Kasus	65
5.3	Instruksi	66
5.3.a	Prinsip Visualisasi: Memilih Grafik Sesuai Pertanyaan Perencanaan	66
5.3.b	Persiapan Data dan Impor Pustaka	67
5.3.c	Diagram Batang: Membandingkan Fasilitas Antar-Kecamatan	68
5.3.d	Scatter Plot: Menelusuri Hubungan Kepadatan dan Fasilitas	69
5.3.e	Histogram, Density Plot, dan Box Plot: Membaca Distribusi Kepadatan Penduduk Kota	71
5.3.f	Estetika Visual: Menambahkan Judul, Label, Legenda, dan Warna Harmonis	72
5.3.g	Storytelling: Menuliskan Interpretasi Naratif di Bawah Grafik	75
5.4	Latihan Mandiri	76
6	Pengenalan Data Spasial dengan GeoPandas	77
6.1	Pendahuluan	77
6.2	Gambaran Umum Kasus	77
6.3	Instruksi	78
6.3.a	Mengenal Format Vektor GeoJSON	78
6.3.b	Membaca Berkas GeoJSON Batas Kelurahan dengan GeoPandas	79
6.3.c	Memplot Titik Koordinat: Mengonversi Data Tabular Menjadi Geometri Spasial	82
6.3.d	Penyelarasan CRS: Memastikan Data Spasial Menempel pada Proyeksi yang Sama . . .	90
6.3.e	Penggabungan Data: Menggabungkan Atribut Kelurahan ke Poligon Batas	95
6.3.f	Peta Tematik Pertama: Choropleth Kepadatan Penduduk	97
6.4	Latihan Mandiri	103
7	Analisis Spasial Lanjutan: Tumpang Susun (<i>Overlay</i>) dan Agregasi Spasial	104
7.1	Pendahuluan	104
7.2	Gambaran Umum Kasus	104
7.3	Instruksi	105
7.3.a	Tumpang Susun (<i>Overlay</i>): Ketika Data Tidak Punya Alamat	105
7.3.b	Menyiapkan Kedua Lapis: Batas Kelurahan dan Penyelarasan CRS	107
7.3.c	Operasi Tumpang Susun: <code>gpd.sjoin</code> dan Pemilihan <i>Predicate</i>	112
7.3.d	Titik Wakil: Jalan Keluar dari Poligon yang Terpotong Batas	115
7.3.e	Agregasi Spasial: Menghitung Jumlah Bangunan per Kelurahan	116
7.3.f	Kepadatan dan Tutupan: Menormalkan Jumlah terhadap Luas	118
7.3.g	Visualisasi: Peta Kepadatan Bangunan per Kelurahan	123
7.4	Latihan Mandiri	128
8	Pemanenan Data Mandiri Berbasis Agen (<i>Agentic Data Harvesting</i>) dan OpenStreetMap	130
8.1	Pendahuluan	130
8.1.a	Memahami Agen AI	130
8.1.b	Perangkat Lunak Agen AI	130
8.2	Gambaran Umum Kasus	131
8.3	Instruksi	131

8.3.a	Menyiapkan Mesin Agen dan Memeriksa Lingkungan	131
8.3.b	Merumuskan Prompt Satu Langkah (<i>One-Shot Prompt</i>) untuk Pemanenan Data Pasar	132
8.3.c	Bagian 3: Memuat dan Memeriksa Integritas Dataset Hasil Tangkapan Agen	133
8.3.d	Bagian 4: Validasi Bounding Box dan Presisi Koordinat Spasial	135
8.3.e	Bagian 5: Memanen Konteks Wilayah dari OpenStreetMap (osmnx)	137
8.3.f	Bagian 6: Sintesis Spasial: Memadukan Sebaran Ruko dengan Jaringan Kota	139
8.4	Latihan Mandiri	141

Kata Pengantar

<masukkan kata pengantar dari tim dosen di sini>

Modul praktikum ini menemani perkuliahan **Analitika Perkotaan (PL25-41003)** di Program Studi Perencanaan Wilayah dan Kota, Institut Teknologi Sumatera. Setiap modul dirancang untuk satu pertemuan praktikum (150 menit) dan mengikuti satu kasus besar yang sama: **Kota Bandar Lampung**.

Cara Memakai Modul Ini

Setiap modul memiliki susunan yang sama sehingga ritme belajar Anda terjaga dari pekan ke pekan:

1. **Sub-CPMK** — kemampuan yang menjadi target pertemuan tersebut.
2. **Pendahuluan** — mengapa kemampuan ini penting bagi seorang perencana.
3. **Gambaran Umum Kasus** — cerita kasus perkotaan yang akan kita kerjakan.
4. **Instruksi** — langkah-langkah praktik yang Anda ikuti dan ketik sendiri di kelas.
5. **Latihan Mandiri** — tugas dengan alur yang sama persis seperti Instruksi, tetapi memakai kolom data yang berbeda. Inilah cara Anda menguji pemahaman tanpa bimbingan.

Pada sebagian sel kode, Anda akan melihat nomor kecil di ujung kanan beberapa baris. Nomor itu **bukan bagian dari kode** dan tidak perlu Anda ketik — ia menunjuk ke daftar penjelasan tepat di bawah sel tersebut, yang menerangkan apa yang dikerjakan baris itu. Di versi HTML, mengklik nomornya akan menyorot barisnya.

Sepanjang modul, Anda akan menemui kotak **callout** seperti ini:

Konsep Python: Contoh Callout

Kotak seperti ini muncul **tepat setelah** sebuah konsep Python dipakai untuk pertama kalinya di dalam instruksi. Isinya penjelasan konsep tersebut, untuk Anda baca ulang secara mandiri di luar kelas. Saat praktikum, Anda tidak perlu berhenti lama di sini.

Dataset

Seluruh praktikum memakai dataset yang dibagikan dosen dalam folder [dataset-ganjil-2026-2027](#). Sumber utamanya adalah data **PODES 2021** (BPS) untuk Kota Bandar Lampung dan Kota Metro, serta data satuan pendidikan dari Kemendikdasmen. Kami berterima kasih kepada **Prof. Ibnu Syabri, B.Sc., M.Sc., Ph.D.** atas penyediaan dataset PODES 2021 untuk pembelajaran praktikum ini. Cara menata folder dataset ini di komputer Anda dijelaskan pada Modul 1.

Perangkat Lunak

Praktikum memakai perangkat lunak yang seluruhnya gratis: **Miniconda** (pengelola lingkungan Python), **Visual Studio Code** (penyunting kode), dan sejumlah *library* Python yang akan dipasang bertahap sesuai kebutuhan tiap modul. Semua langkah penyiapannya dituntun pada Modul 1.

1 Fondasi Lingkungan Kerja: Python Offline, Terminal, dan Direktori Proyek

! Sub-CPMK Pertemuan Ini

Mahasiswa mampu menyiapkan lingkungan kerja Python *offline* (Conda), mengoperasikan perintah terminal dasar, mengelola berkas dengan struktur direktori proyek standar studio, serta membuat dan menyambungkan *kernel* Jupyter Notebook.

1.1 Pendahuluan

<masukkan narasi yang menceritakan ‘big why’ menyiapkan fondasi lingkungan kerja analitika perkotaan di sini>

1.2 Gambaran Umum Kasus

Setiap kali sebuah studio perencanaan memulai proyek baru, hal pertama yang dikerjakan bukanlah menggambar. Meja gambar dirapikan, map-map berkas disiapkan dan diberi label, lalu penggaris, pensil, dan kalkulator ditata di rak masing-masing. Tanpa persiapan itu, di tengah proyek kita akan kehilangan berkas, meminjam alat orang lain yang ternyata berbeda ukuran, dan hasil kerja menjadi kacau.

Pekerjaan analitika perkotaan persis sama. Dalam sebuah *command center* perkotaan, lingkungan kerja untuk mengolah semua data juga butuh persiapan. Untuk itu, sebelum menyentuh data Kota Bandar Lampung pada pertemuan-pertemuan berikutnya, dalam modul ini kita merapikan “meja kerja digital” terlebih dahulu.

Kita akan menata direktori proyek dengan struktur yang baku, memasang peralatan Python secara *offline* menggunakan Conda agar tidak bergantung pada koneksi internet yang tidak stabil, belajar berbicara dengan komputer lewat terminal, menulis skrip Python pertama, berlatih membaca pesan kegagalan (*error*) yang paling sering muncul, membuktikan mengapa satu berkas skrip harus lengkap berdiri sendiri, dan menutup pertemuan dengan menyiapkan “buku catatan lapangan” digital kita: Jupyter Notebook yang tersambung ke mesin Python (*kernel*) milik kita sendiri.

1.3 Instruksi

1.3.a Menata Meja Kerja: Menyiapkan Alat-alat Sesuai Kebutuhan

Selama menjadi operator *command center* Anda akan bekerja dengan banyak alat berupa kode-kode dengan fungsi khusus dari berbagai sumber. Agar semuanya siap pakai, kita perlu atur lingkungan kerjanya. Ini persis seperti menata ruang studio. Bayangkan di hari pertama kita masuk ruang studio: kita diberi satu laci khusus, lalu di dalamnya kita menyiapkan map-map bertuliskan “data mentah”, “gambar kerja”, dan “laporan”. Di komputer, laci itu bernama **direktori** (sebutan lain untuk *folder*), dan seluruh pekerjaan satu semester ini akan tinggal di dalam satu direktori induk.

Ikuti langkah berikut:

1. Buka **File Explorer**, lalu masuk ke folder `C: → Users → <Nama User Anda> → Documents`.

⚠️ Hindari Folder Downloads

Jangan gunakan folder seperti `Downloads` sebagai direktori proyek utama. Berkas di folder tersebut rentan terhapus secara tidak sengaja ketika Anda melakukan pembersihan berkas sementara atau pemeliharaan sistem komputer.

i Pengetahuan Komputer

Root Directory dan Media Penyimpanan (HDD/SSD)

Pangkal teratas dari struktur folder di komputer disebut **direktori akar** (*root directory*). Di sistem operasi Windows, direktori akar ini direpresentasikan oleh huruf kandar (*drive letter*) seperti `C:`, `D:`, atau `E:`.

Kandar `C:` umumnya merupakan partisi utama pada media penyimpanan permanen tempat sistem operasi Windows terpasang. Komputer modern saat ini umumnya menggunakan **SSD (Solid-State Drive)** yang memiliki kecepatan baca-tulis jauh lebih cepat dibandingkan **HDD (Hard Disk Drive)** mekanis lama. Memahami struktur ini sangat penting agar kita tahu persis di mana data fisik kita disimpan di dalam perangkat keras komputer.

2. Buat folder baru bernama `analitika-perkotaan`. Inilah **direktori proyek** kita, padanan laci studio tadi.
3. Di dalamnya, buat empat subfolder sehingga susunannya seperti ini:

```
analitika-perkotaan/      <- direktori proyek (laci induk)
├── dataset-ganjil-2026-2027/ <- map data mentah dari dosen
├── modul/                 <- map buku catatan (notebook) praktikum
├── skrip/                 <- map skrip Python (.py)
└── keluaran/              <- map hasil kerja (tabel, grafik, peta)
```

4. Salin folder `dataset-ganjil-2026-2027` yang dibagikan dosen ke dalam direktori proyek, sehingga namanya persis seperti bagan di atas.

i Pengetahuan Komputer

Padanan Path dan Struktur Folder Tree

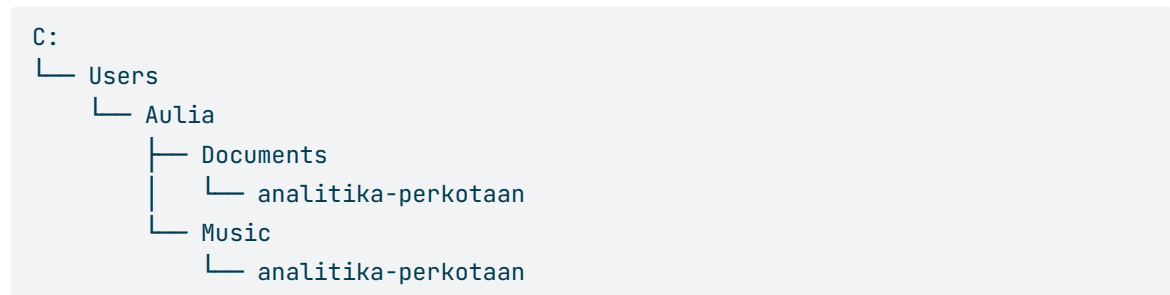
Struktur folder tree di atas dapat direpresentasikan ke dalam bentuk **path**. Sebagai contoh, struktur folder tree berikut:

```
C:
├── Users
│   ├── Aulia
│   │   ├── Documents
│   │   │   └── analitika-perkotaan
```

akan dituliskan sebagai path absolut: `C:/Users/Aulia/Documents/analitika-perkotaan`.

Mengapa pemahaman path ini penting?

Perhatikan contoh penamaan direktori dengan path yang salah di bawah ini:



Meskipun kedua folder sama-sama bernama `analitika-perkotaan`, path absolutnya sangat berbeda: yang satu berada di `C:/Users/Aulia/Documents/analitika-perkotaan` dan yang satu lagi di `C:/Users/Aulia/Music/analitika-perkotaan`. Jika program kita salah mencari lokasi, program itu tidak akan menemukan berkas yang tepat.

i Pengetahuan Komputer

Penggunaan Simbol Slash di Windows vs Unix untuk penulisan Path

Sistem operasi Windows secara tradisional menggunakan simbol *backslash* (`\`) untuk memisahkan nama folder di dalam path (misalnya `C:\Users\Aulia`). Namun, sistem berbasis Unix (seperti Linux dan macOS), serta browser web, menggunakan *forward slash* (`/`) (misalnya `/Users/Aulia`). Dalam pemrograman Python, disarankan untuk menggunakan *forward slash* (`/`) karena Python dapat menerjemahkannya secara otomatis di Windows dan mencegah galat akibat karakter khusus (*escape character*).

Susunan bercabang seperti bagan di atas disebut **folder tree** (pohon direktori). Untuk menunjuk lokasi sebuah berkas di dalam pohon itu, komputer memakai **path**: rangkaian nama folder yang dilalui untuk sampai ke berkas tersebut, dipisahkan garis miring. Ada dua cara menuliskannya:

- **Path lengkap (absolut):** dari pangkal pohon sampai ujung, misalnya `C:/Users/Aulia/Documents/analitika-perkotaan/skrip/sapa.py`. Seperti alamat surat lengkap: kota, kecamatan, jalan, nomor rumah.
- **Path relatif:** dihitung dari posisi kita berdiri sekarang. Jika kita sedang berada di dalam `analitika-perkotaan`, berkas yang sama cukup ditunjuk dengan `skrip/sapa.py`. Seperti berkata “dua rumah di sebelah kanan” kepada orang yang sudah berdiri di jalan yang sama.

```
analitika-perkotaan/
├── dataset-ganjil-2026-2027/
├── modul/
├── skrip/
│   └── sapa.py          ← berkas yang ditunjuk
└── keluaran/
```

Sepanjang semester kita akan lebih sering memakai *path* relatif, dan karena itu **posisi kita berdiri** (folder mana yang sedang terbuka) menjadi penting. Ini akan segera terasa saat kita bekerja dengan terminal.

1.3.b Mendata Python yang Sudah Ada

Sebelum memasang apa pun, periksa dulu apa yang sudah terpasang. Prinsipnya sama seperti peserta studio yang memeriksa dulu meja siapa saja yang sudah menumpuk di ruangan sebelum menata meja kerjanya sendiri, supaya tidak keliru memakai meja orang lain dan tidak menggeser barang yang ternyata masih dipakai. Di komputer, langkah itu berarti mendata Python apa saja yang **sudah ada** sebelum kita menambah apa pun.

Langkah nol: buka Command Prompt atau PowerShell, lalu ketik dua perintah berikut satu per satu. Tuliskan persis seperti tertulis, lengkap dengan akhiran `.exe`-nya:

```
1 where.exe python
2 where.exe conda
```

Perintah `where.exe` menampilkan **semua** lokasi program bernama itu yang ditemukan komputer, berurutan dari yang paling didahulukan. Salin hasilnya ke buku catatan Anda; kita akan menagihnya lagi di langkah kelima pada bagian berikutnya. Jangan kaget bila keluarannya lebih dari satu baris, karena itu yang biasa terjadi.

Penting!

Mengapa Ditulis `where.exe`, Bukan `where`

Windows menyediakan dua terminal yang tampilannya nyaris serupa: **Command Prompt** dan **PowerShell**. Bedakan dari tulisan di awal barisnya. PowerShell selalu mengawali barisnya dengan `PS`, seperti `PS C:\Users\Nama>`, sedangkan Command Prompt langsung `C:\Users\Nama>`. Perhatikan ini baik-baik, karena pada Windows 11 PowerShell-lah yang paling sering terbuka lebih dulu, termasuk saat kita membuka terminal bawaan VS Code nanti.

Perbedaannya berakibat langsung di langkah ini. Di Command Prompt, `where` adalah nama program pencari lokasi yang kita maksud. Di PowerShell, nama `where` sudah dipakai lebih dulu sebagai singkatan perintah bawaannya sendiri (`Where-Object`) yang tugasnya menyaring daftar, sama sekali tidak berhubungan dengan mencari lokasi program.

Bentuk kegagalannya justru yang paling menyesatkan: PowerShell **tidak menampilkan pesan galat apa pun**. PowerShell hanya diam, kursor kembali ke baris kosong, seolah-olah perintahnya sudah dikerjakan. Layar kosong itu sangat mudah dibaca sebagai baris terakhir tabel di bawah, “belum ada Python”, padahal komputer Anda bisa jadi punya tiga sekaligus. Berbeda dengan pesan `'...' is not recognized` yang jelas-jelas menyuruh kita memperbaiki sesuatu, kegagalan yang diam justru membuat kita melangkah ke tahap berikutnya dengan kesimpulan yang keliru.

Akhiran `.exe` menutup celah itu dengan memaksa terminal menjalankan program pencariannya, bukan singkatan tadi. Bentuk `where.exe` **aman dipakai di ketiganya**: Command Prompt, PowerShell, maupun Anaconda Prompt. Biasakan menulis akhiran itu sepanjang mata kuliah ini.

Sekarang cocokkan keluaran Anda dengan daftar berikut, supaya langkah nol ini menghasilkan sebuah **kesimpulan**, bukan sekadar tumpukan tulisan. Perhatikan **artinya** dan **apa yang perlu Anda lakukan sesudahnya**:

- **C:\Python27\ArcGIS10.8\... atau ... \ArcGISx6410.8\...**
 - *Artinya*: Komputer Anda memiliki instalasi ArcGIS Desktop 10.x yang membawa Python 2.7.
 - *Tindak lanjut*: Baca **Kasus A** pada kotak “Komputer Anda Mungkin Sudah Punya Python Tanpa Anda Sadari” di bawah sebelum melanjutkan ke bagian berikutnya.
- **... \ArcGIS\Pro\bin\Python\... (pada where.exe conda)**
 - *Artinya*: Komputer Anda memiliki instalasi ArcGIS Pro yang membawa Conda tersendiri.
 - *Tindak lanjut*: Baca **Kasus B** pada kotak “Komputer Anda Mungkin Sudah Punya Python Tanpa Anda Sadari” di bawah sebelum melanjutkan ke bagian berikutnya.
- **... \WindowsApps\python.exe**
 - *Artinya*: Python terpasang melalui Microsoft Store.
 - *Tindak lanjut*: Kondisi ini tidak menghalangi pekerjaan kita. Lanjutkan langsung ke bagian “Memasang dan Mengaktifkan Conda”; baris ini akan dipakai lagi sebagai pembanding di langkah kelima.
- **C:\Python3xx\... atau ... \AppData\Local\Programs\Python\...**
 - *Artinya*: Python terpasang dari situs resmi python.org.
 - *Tindak lanjut*: Tidak menghalangi, tetapi baca terlebih dahulu peringatan “kegagalan diam” pada poin keempat kotak “Komputer Anda Mungkin Sudah Punya Python Tanpa Anda Sadari” di bawah, kemudian lanjutkan ke bagian “Memasang dan Mengaktifkan Conda”.
- **Tidak ada keluaran sama sekali**
 - *Artinya*: Belum ada Python yang terpasang di komputer Anda (kondisi paling bersih).
 - *Tindak lanjut*: Pastikan kembali bahwa perintah yang Anda ketik adalah `where.exe`, bukan `where`. Jika memang tidak ada keluaran, lanjutkan langsung ke bagian “Memasang dan Mengaktifkan Conda” tanpa kekhawatiran tambahan.

Komputer Anda Mungkin Sudah Punya Python Tanpa Anda Sadari

Bila di komputer Anda terpasang **ArcGIS**, Python **sudah ada di sana sejak lama**, dan ada dua kemungkinan yang penanganannya berbeda. Tabel di atas menunjukkan Anda termasuk yang mana. Baca **hanya kasus Anda**; bila keduanya muncul, kerjakan keduanya.

- **Kasus A: ArcGIS Desktop 10.x (ArcMap). Keputusannya: biarkan Python bawaan ArcGIS ini apa adanya (jangan dihapus, jangan dipakai untuk mata kuliah ini), lalu lanjutkan ke bagian “Memasang dan Mengaktifkan Conda” di bawah.** Yang dibawanya adalah Python **versi 2.7**, umumnya di `C:/Python27/ArcGIS10.8`, dan ArcGIS Desktop sama sekali tidak memakai Conda. Apakah Python 2.7 ini terdaftar lebih dulu di daftar alamat komputer tidak bisa ditebak dari luar: pada sebagian mesin Python 2.7 menang, pada sebagian lain Python 2.7 tidak terdaftar sama sekali meski ArcGIS-nya terpasang. Bila Python lama itu menang, mengetik `python` di terminal baru menjalankan **Python 2.7**, bukan Python yang kita instal sendiri. Bahayanya justru bukan karena Python 2.7 menolak bekerja, melainkan sebaliknya: Python lama itu mau berjalan, tapi gagal menjalankan sintaks yang baru ada di versi setelahnya. Kode seperti `print(f"HaLo, {nama}")` yang baru diperkenalkan di versi 3.6 akan ditolaknya dengan `SyntaxError`, padahal kode itu benar. Anda akan mengira diri sendiri salah ketik, padahal yang keliru adalah versi Python yang menjalankannya. Inilah alasan langkah nol dikerjakan sebelum apa pun.
- **Kasus B: ArcGIS Pro. Keputusannya: jangan pakai terminal “Python Command Prompt” bawaan ArcGIS Pro untuk mata kuliah ini; pakai Anaconda Prompt milik Miniconda pada bagian**

berikutnya. ArcGIS Pro membawa **Conda-nya sendiri**, lengkap dengan sebuah *environment* bernama `arcgispro-py3` di `C:/Program Files/ArcGIS/Pro/bin/Python/envs/arcgispro-py3`. Bahayanya lebih halus dan baru terasa di bagian berikutnya: perintah `conda` yang dikenali komputer Anda bisa jadi **milik ArcGIS Pro**, sehingga ruang kerja mata kuliah ini tanpa sadar dibangun di dalam instalasi ArcGIS. Periksa keluaran `where.exe conda` Anda: bila ada baris yang memuat potongan `\ArcGIS\Pro\`, jangan pakai terminal “Python Command Prompt” bawaan ArcGIS. Gunakan **Anaconda Prompt** milik Miniconda pada bagian berikutnya; bila Anaconda Prompt belum ada di komputer Anda, ikuti dulu instruksi pemasangan Miniconda di sana.

- **Berlaku untuk kedua kasus: jangan dihapus, dan jangan dipakai untuk mata kuliah ini.** Jangan dihapus, karena ArcGIS bergantung sepenuhnya pada Python bawaannya; merusaknya berarti merusak ArcGIS Anda. Jangan dipakai, dan ini bukan soal selera: Python 2.7 pada Kasus A sudah berhenti dikembangkan sejak 1 Januari 2020 sehingga tidak satu pun *library* yang kita pakai semester ini berjalan di sana, sedangkan *environment* bawaan ArcGIS Pro pada Kasus B memang dirancang untuk tidak diubah-ubah. *Environment* yang kita buat pada bagian berikutnya berdiri **berdampingan** dengan Python-Python itu: menambah satu ruang kerja baru, tanpa menggusur ruang kerja yang sudah ada.
- Di luar dua kasus itu, Python juga bisa masuk lewat **Microsoft Store** atau pemasangan dari situs **python.org**, sering kali tanpa kita sadari. Jangan buru-buru menganggapnya sepele: seringkali justru Python dari python.org inilah yang **memenangkan baris pertama** daftar alamat, mengalahkan Python bawaan ArcGIS sekalipun. Bahayanya berkebalikan dengan Kasus A, dan justru karena itu lebih licin: Python dari python.org **tidak menolak apa pun**. Kode Anda jalan, tidak ada tulisan merah, tetapi *library* yang dipakainya berbeda versi sehingga angka yang keluar bisa berbeda dari yang muncul di layar teman sebelah, tanpa satu pun pesan yang memberi tahu. Kegagalan yang diam jauh lebih sulit dilacak daripada yang berisik. Karena itu keberadaan mereka tidak cukup diingat, melainkan harus dibuktikan siapa yang sesungguhnya berjalan, dan itulah yang kita kerjakan di langkah kelima pada bagian berikutnya.

Catatan hasil langkah nol ini akan dipakai lagi pada bagian berikutnya, “Memasang dan Mengaktifkan Conda”. Jangan tutup buku catatan Anda dulu.

1.3.c Memasang dan Mengaktifkan Conda

Di studio, setiap perencana punya ruang kerja studio pribadi berisi meja, kursi, dan rak peralatan miliknya sendiri, supaya pengaturan alat untuk proyek yang satu tidak tercampur dengan proyek lain. Dalam dunia Python, ruang kerja pribadi itu bernama ***environment***, dan petugas yang menyiapkannya bernama **Conda**. Satu komputer bisa memiliki banyak *environment* sekaligus, masing-masing berisi versi Python dan *library* yang berbeda, tanpa saling mengganggu.

Ada dua kemungkinan tempat Anda mengerjakan bagian ini, dan keduanya menentukan seberapa banyak langkah di bawah yang perlu Anda tempuh:

- **(a) Anda di PC laboratorium kampus.** Tim dosen sudah memasang Conda beserta *environment* mata kuliah ini di komputer lab. Anda cukup memverifikasinya, bukan memasangnya dari nol; lompat ke Langkah kedua di bawah.
- **(b) Anda di laptop pribadi, kemungkinan besar sebelum masuk kelas.** Modul ini dibagikan di awal perkuliahan justru supaya Anda bisa mencobanya lebih dulu secara mandiri. Ikuti seluruh langkah dari Langkah pertama, karena bagian ini ditulis lengkap dari nol dan tidak mengasumsikan ada asisten yang mendampingi saat itu juga.

Langkah pertama: periksa dulu, jangan langsung memasang. Cari **Anaconda Prompt** lewat kotak pencarian Start Menu (ketik “anaconda”). Jika Anaconda Prompt belum ada di komputer Anda, pasanglah **Miniconda** terlebih dahulu mengikuti panduan pada kotak “Panduan Pemasangan Miniconda” di bawah.

 **Penting!**

Anaconda Prompt Tidak Ditemukan Sama Sekali?

Ada beda antara “Anaconda Prompt ditemukan tapi belum mengenali `conda`” dengan “Anaconda Prompt tidak ada sama sekali di daftar aplikasi”. Yang kedua bukan kesalahan pencarian, melainkan tanda pasti bahwa **Conda memang belum pernah terpasang** di komputer Anda. Jika itu yang terjadi, lewati dulu langkah memeriksa versi di bawah, pasang Miniconda mengikuti panduan pada kotak “Panduan Pemasangan Miniconda” di bawah, lalu kembali ke sini dan buka **Anaconda Prompt** yang baru muncul.

 **Pengetahuan Komputer**

Panduan Pemasangan Miniconda

Miniconda adalah versi minimal dari Conda yang hanya memuat Python dan sistem manajernya, sehingga berukuran jauh lebih ringkas. Rujukan langkah awal instalasi perangkat lunak perkuliahan dapat dilihat pada [[panduan-instalasi-awal-vscode-conda]].

Untuk memasang Miniconda di komputer Anda:

1. Memperoleh berkas penginstal (*installer*):

- **Melalui berkas *offline*:** Dosen membagikan berkas penginstal `Miniconda3-latest-Windows-x86_64.exe` sehingga Anda dapat langsung memasangnya tanpa mengunduh.
- **Melalui internet (*online*):** Buka peramban ke situs <https://www.anaconda.com/download>, gulir laman hingga menemukan tombol **Download Miniconda Installer**, lalu unduh **Windows 64-Bit Graphical Installer** (atau installer macOS yang sesuai bagi pengguna Mac).

2. Menjalankan instalasi:

- Buka fail `.exe` penginstal, klik **Next**, lalu terima persetujuan lisensi dengan mengeklik **I Agree**.
- Pilih opsi **Just Me** untuk tipe instalasi (khusus akun pengguna aktif), lalu klik **Next**.
- Biarkan destinasi folder lokasi instalasi apa adanya (*default*), lalu klik **Next**.
- Centang seluruh opsi bawaan **kecuali** `Add installation to my PATH environment variable` (biarkan tidak tercentang; alasannya dibahas pada kotak “Satu Kotak Centang yang Harus Dibiarkan Kosong” di bawah).
- Klik **Install** dan tunggu hingga proses selesai.

3. Membuka terminal kerja:

Setelah selesai, buka Start Menu, ketik “anaconda prompt”, dan buka aplikasi **Anaconda Prompt** untuk melanjutkan verifikasi.

Jika Anaconda Prompt ditemukan, buka aplikasi itu, lalu ketik:

```
1 conda --version
```

Ada dua kemungkinan jawaban:

- Muncul tulisan seperti `conda 24.11.0` (angkanya boleh berbeda): Conda **sudah terpasang**. Lanjutkan ke Langkah kedua.
- Muncul pesan seperti `'conda' is not recognized ...`: Conda **belum ada**. Pasang **Miniconda** mengikuti panduan pada kotak “Panduan Pemasangan Miniconda” di atas, tutup terminalnya, buka **Anaconda Prompt** yang baru, lalu ulangi perintah di atas sampai versi Conda muncul.

 Penting!

Satu Kotak Centang yang Harus Dibiarkan Kosong

Penginstal Miniconda menampilkan pilihan “**Add Miniconda3 to my PATH environment variable**”. **Biarkan kotak ini tidak dicentang**; ini memang pilihan bawaannya, dan penyedia Miniconda sendiri menyarankan demikian. Ini relevan khusus untuk Anda yang komputernya sudah punya Python lain (ingat hasil langkah nol pada bagian sebelumnya): mencentangnya berisiko membuat urutan PATH bentrok dengan Python-Python yang sudah ada, dan gejalanya baru terasa berminggu-minggu kemudian dalam bentuk yang membingungkan. Pilihan “Register Miniconda3 as my default Python 3.x” boleh dibiarkan tercentang seperti bawaannya; itu hanya menentukan program apa yang membuka berkas `.py` saat diklik dua kali, tidak memengaruhi apa yang berjalan saat Anda mengetik `python` di terminal.

 Penting!

Konsep Path & Menjalankan Conda di Terminal Biasa

Anaconda Prompt adalah terminal khusus yang sudah dikonfigurasi secara otomatis agar dapat mengenali perintah `conda`. Untuk seluruh instruksi di modul ini, Anaconda Prompt sudah cukup; Anda **tidak wajib** membuat `conda` dikenali di Command Prompt biasa atau PowerShell.

Bila suatu saat Anda tetap ingin mengetik `conda` langsung di Command Prompt biasa atau PowerShell, di luar keperluan modul ini, jalankan `conda init` satu kali di Anaconda Prompt, lalu tutup dan buka kembali terminal biasa Anda. Satu jebakan yang sering muncul sesudahnya: PowerShell menolak menjalankan skrip aktivasi Conda dengan pesan “cannot be loaded because running scripts is disabled on this system”. Pesan itu bukan berarti Conda gagal terpasang, melainkan kebijakan keamanan bawaan Windows. Perbaikannya satu baris, dijalankan sekali di PowerShell tersebut: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, lalu jawab `Y` saat diminta konfirmasi.

Langkah kedua: periksa apakah *environment* mata kuliah sudah ada. Ketik:

```
1 conda env list
```

Perintah ini menampilkan daftar semua ruang kerja (*environment*) yang ada di komputer. Jika di daftarnya sudah ada baris bernama `analitika-perkotaan` (kondisi normal di komputer lab), ruang kerja kita sudah berdiri; lompat ke Langkah keempat.

Langkah ketiga (hanya jika belum ada): buat *environment* dari berkas resep. Dosen membagikan sebuah berkas bernama `environment.yml`. Salin berkas itu ke direktori proyek `analitika-perkotaan` yang kita buat tadi. Isinya seperti berikut:

```

1  name: analitika-perkotaan
2
3  channels:
4      - conda-forge
5      - nodefaults
6
7  dependencies:
8      # Dasar: Modul 2-4
9      - python=3.12
10     - pandas
11     - openpyxl
12
13     # Visual: Modul 5
14     - matplotlib-base
15     - seaborn
16
17     # Spasial: Modul 6-7
18     - geopandas
19     - leafmap
20     - mapclassify
21     - folium
22     - contextily
23
24     # Penarikan data: Modul 9
25     - requests
26     - beautifulsoup4
27     - osmnx
28     - nodejs>=20
29     - pip
30     - pip:
31         - browser-act-cli
32
33     # Pemodelan: Modul 10
34     - scikit-learn
35
36     # Dashboard: Tugas Besar Pekan 11-16 & UAS
37     - streamlit
38     - streamlit-folium
39
40     # Infrastruktur agar notebook berjalan
41     - ipykernel
42     - jupyterlab
43     - notebook
44     - nbformat
45     - nbclient
46     - pyyaml
47

```

```

48 # Bantu: pembacaan dataset dan tabel interaktif
49 - dbfread
50 - itables
51
52 variables:
53 PYTHONNOUSERSITE: "1"

```

Baris 1

Nama ruang kerja yang akan dibuat Conda, dipakai setiap kali kita mengetik `conda activate analitika-perkotaan`.

Baris 3

Toko tempat Conda membeli setiap paket di bawahnya. `conda-forge` adalah toko utamanya; `nodefaults` mencegah Conda melirik toko bawaan lain yang bisa memberi versi paket yang berbeda.

Baris 7

Daftar rak, kursi, dan meja yang dibeli untuk ruang kerja ini, dikelompokkan per modul lewat baris komentar `#` supaya mudah ditelusuri paket mana dipakai di pertemuan mana. Baris berawalan `#` adalah komentar: keterangan untuk manusia, diabaikan komputer.

Baris 52

Satu aturan rumah yang berlaku di dalam ruangan ini. `PYTHONNOUSERSITE` melarang ruang kerja kita mengambil perabot dari rak titipan bersama di luar ruangan; alasannya dibahas pada kotak “Jangan Pernah Memasang Paket di Luar Environment” di akhir bagian ini.

Dengan satu berkas ini kita tidak perlu mengetik nama *library* satu per satu; cukup satu perintah, dan Conda membelanjakan semuanya:

```

1 cd Documents\analitika-perkotaan
2 conda env create -f environment.yml

```

Perintah `cd` (*change directory*) memindahkan posisi berdiri kita ke direktori proyek. Ingat pembahasan *path* sebelumnya: `environment.yml` ditunjuk dengan *path* relatif, sehingga kita harus berdiri di folder tempat berkas itu berada. Proses ini butuh beberapa menit.

Langkah keempat: masuk ke ruang kerja. Aktifkan *environment* dengan:

```

1 conda activate analitika-perkotaan

```

Perhatikan awalan tulisan di terminal berubah dari `(base)` menjadi `(analitika-perkotaan)`. Awalan itu adalah papan nama ruangan yang memberi tahu ruang kerja mana yang sedang kita buka pintunya untuk mulai bekerja. Mulai sekarang, setiap kali membuka terminal untuk mata kuliah ini, biasakan memeriksa papan nama tersebut.

Langkah kelima: buktikan ruang kerjanya benar-benar terbuka. Papan nama `(analitika-perkotaan)` itu *klaim*, bukan *bukti*. Papan nama bisa menyala sementara yang benar-benar bekerja adalah Python lain, dan ini bukan kemungkinan teoretis, melainkan kejadian yang wajar di komputer yang sudah punya banyak Python (ingat hasil langkah nol pada bagian sebelumnya). Karena itu, sekali di awal semester, kita buktikan. Dengan papan nama `(analitika-perkotaan)` menyala, jalankan tiga perintah berikut. Perhatikan lagi akhiran `.exe` pada perintah pertama:

```
1 where.exe python
2 python -c "import sys; print(sys.prefix)"
3 python -c "import sys; print(sys.path)"
```

Ruang kerja Anda dinyatakan **lulus** bila ketiga syarat ini terpenuhi:

1. Baris **paling atas** keluaran `where.exe python` memuat potongan `envs\analitika-perkotaan`.
2. Keluaran perintah kedua berakhir dengan `envs\analitika-perkotaan`.
3. Keluaran perintah ketiga **tidak memuat** potongan `AppData\Roaming\Python` di mana pun.

Catat ketiga keluaran itu; Latihan Mandiri butir 5 akan menagihnya.

⚠ Jika Ruang Kerja Anda Tidak Lulus

Perintah pertama tidak mengeluarkan apa pun: hampir pasti yang terketik `where`, bukan `where.exe`, dan terminal Anda PowerShell. Ulangi dengan akhiran `.exe`-nya.

Syarat 1 atau 2 gagal (yang muncul Python lain): tutup terminal, buka **Anaconda Prompt** baru, jalankan `conda activate analitika-perkotaan`, lalu ulangi. Bila masih gagal, kemungkinan besar *environment*-nya belum benar-benar terbentuk; ulangi Langkah kedua dan ketiga. Bila yang muncul di baris teratas justru `C:\Python27\...`, itu Kasus A pada kotak “Komputer Anda Mungkin Sudah Punya Python Tanpa Anda Sadari” di bagian sebelumnya; penyebabnya hampir selalu `conda activate` yang belum dijalankan di terminal tersebut.

Syarat 3 gagal (muncul `AppData\Roaming\Python`): **jangan diperbaiki sendiri, laporkan ke dosen.** Kondisi ini berarti ada rak paket titipan yang membayangi ruang kerja Anda, dan membersihkannya sembarangan bisa merusak program lain di komputer Anda, termasuk ArcGIS Pro bila Anda memakainya.

i Pengetahuan Komputer

Satu Komputer, Banyak Python: Mengapa Harus Dibuktikan

Ketika kita mengetik `python` di terminal, komputer **tidak tahu** Python mana yang kita maksud. Komputer hanya menyusuri daftar alamat bernama **PATH** dari atas ke bawah, lalu menjalankan **program pertama** bernama `python` yang ditemukan. Itulah sebabnya `where.exe python` bisa menampilkan beberapa baris sekaligus: baris pertama adalah yang menang, sisanya menganggur.

`conda activate` bekerja dengan cara menyelipkan alamat *environment* kita ke **urutan paling atas** daftar itu. Bila ada sesuatu yang menyelipkan alamat lain lebih atas lagi, papan nama tetap menyala tetapi yang berjalan Python yang lain. Analogi studionya: papan nama di pintu ruangan bisa saja tertulis nama Anda, sementara yang duduk mengerjakan gambar di dalamnya orang lain. Satu-satunya cara memastikan adalah menengok ke dalam, dan `where.exe python` beserta `sys.prefix` adalah cara menengoknya.

Perintah `python -c "..."` sendiri berarti “jalankan kode di dalam tanda kutip ini, sekali saja, lalu keluar”. Isi kodenya akan Anda pahami sepenuhnya di Modul 2; untuk sekarang, kode itu cukup diperlakukan sebagai **alat periksa**, seperti meteran yang kita pakai tanpa perlu tahu isi mesinnya.

⚠ Jangan Pernah Memasang Paket di Luar *Environment*

Suatu hari Anda akan menemukan tutorial di internet yang menyuruh mengetik `pip install nama-paket`. **Jangan pernah menjalankannya sebelum papan nama (*analitika-perkotaan*) menyala.**

Alasannya begini. Selain rak di dalam tiap ruang kerja, Windows menyediakan **satu rak titipan bersama** di `C:/Users/<Nama User Anda>/AppData/Roaming/Python/`. Rak itu dipakai bersama-sama oleh **semua** Python bernomor versi sama di komputer itu, termasuk Python bawaan program lain seperti ArcGIS Pro, bila nomor versinya kebetulan sama. Lebih berbahaya lagi: isi rak titipan itu **didahulukan** atas isi ruang kerja kita.

Akibatnya, satu perintah `pip install` yang dijalankan tanpa mengaktifkan *environment* bisa: (a) diam-diam mengubah perilaku ruang kerja mata kuliah ini, (b) mengganggu program lain yang memakai Python bernomor versi sama, dan (c) melakukan keduanya **tanpa satu pun pesan galat**, sehingga kerusakannya baru terasa berminggu-minggu kemudian dan sangat sulit dilacak.

Aturan kelas ini sederhana dan tidak ada pengecualiannya: `conda activate analitika-perkotaan` **dulu, baru memasang apa pun.**

Sejauh ini, bahaya “kegagalan diam” di atas mungkin terasa seperti urusan nilai tugas semata. Bayangkan sekarang kursi Anda bukan lagi kursi mahasiswa, melainkan kursi operator di sebuah **command center** perkotaan: layar di depan Anda menampilkan kepadatan penduduk per kecamatan, titik rawan kemacetan, atau peringatan dini genangan, dan angka-angka itu dipakai orang lain untuk memutuskan ke mana petugas dikirim malam ini. *Environment* yang tidak terverifikasi bisa membuat salah satu angka di layar itu keliru tanpa satu pun peringatan merah, persis seperti Python python.org yang “menang diam-diam” pada kotak “Komputer Anda Mungkin Sudah Punya Python Tanpa Anda Sadari” sebelumnya. Bedanya, di ruang kelas kesalahan itu berakibat nilai tugas berkurang; di *command center*, kesalahan yang sama berakibat keputusan nyata dibuat berdasarkan angka yang salah. Kebiasaan membuktikan ruang kerja pada Langkah kelima bukan formalitas kuliah; itu kebiasaan kerja yang akan terus Anda pakai setelah lulus.

1.3.d Berbicara dengan Terminal: Menjalankan Perintah Python Interaktif

Terminal adalah loket tempat kita memberi instruksi kerja kepada komputer dengan tulisan, bukan klik. Seperti berbicara dengan petugas loket: kita menyebut satu permintaan, petugas mengerjakannya, lalu melapor balik.

Buka **Visual Studio Code**, lalu buka direktori proyek kita lewat menu **File** → **Open Folder** → *analitika-perkotaan*.

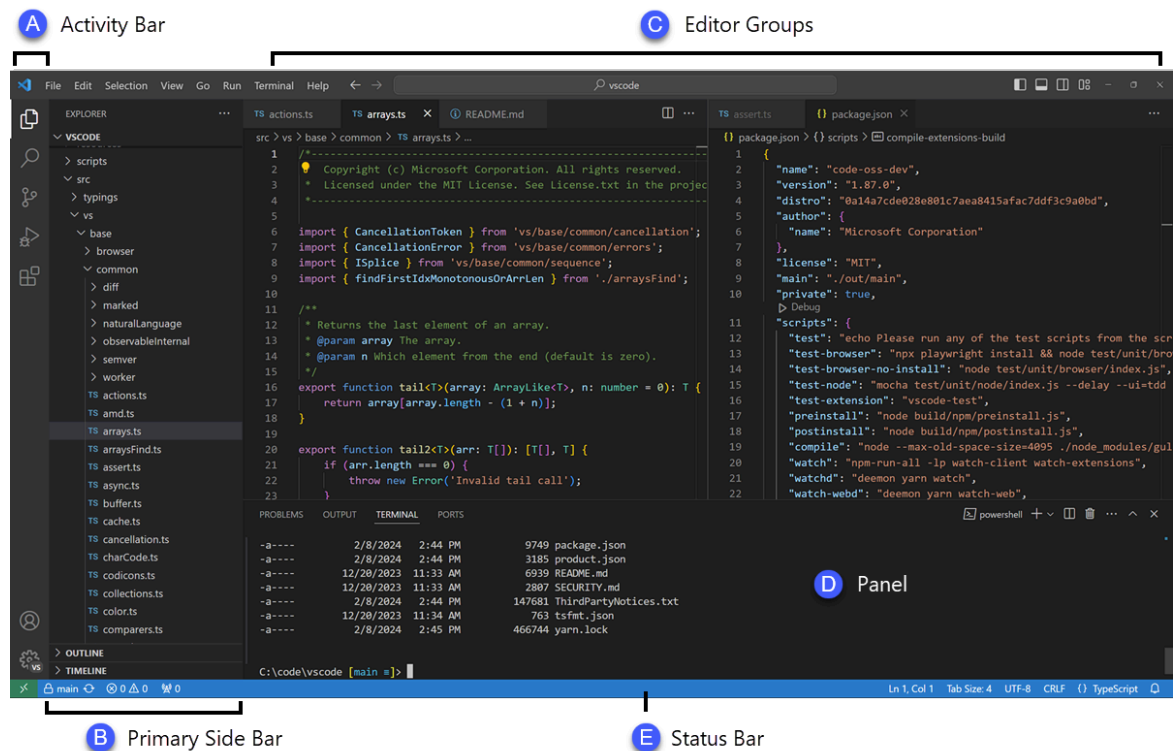
Pengetahuan VS Code

Mengenal Antarmuka Visual Studio Code

Saat pertama kali membuka Visual Studio Code, terdapat beberapa area utama yang esensial untuk menjalankan praktik kita:

1. **Activity Bar** (Sisi Paling Kiri): Berisi ikon-ikon pintasan untuk mengelola lembar kerja, seperti *Explorer* (melihat berkas proyek), *Search* (mencari teks), dan *Extensions* (memasang fitur tambahan).

2. **Side Bars (Sampling Activity Bar):** Menampilkan daftar berkas dan folder aktif dalam direktori proyek Anda (*Explorer*).
3. **Editor Area/Editor Groups** (Bagian Tengah): Tempat utama untuk menulis dan menyunting berkas kode (.py atau .ipynb). Tempat tersebut bisa berjumlah lebih dari 1 dan mengelompokkan beberapa berkas, sehingga disebut "Groups".
4. **Panel / Integrated Terminal** (Bagian Bawah): Area terminal terintegrasi tempat kita menjalankan perintah baris tanpa harus keluar dari VS Code.



Gambar 1: Tata Letak Antarmuka Visual Studio Code

Buka terminal bawaannya dengan menu **Terminal** → **New Terminal**. Terminal ini otomatis berdiri di direktori proyek, jadi kita tidak perlu `cd` lagi. Pastikan papan nama (*analitika-perkotaan*) menyala; jika belum, jalankan lagi `conda activate analitika-perkotaan`.

Sekarang panggil Python:

```
1 python
```

Tampilan berubah: muncul keterangan versi Python dan tanda `>>>`. Tanda itu artinya Python siap menerima perintah **satu per satu**. Mode ini disebut mode **interaktif** (atau *console*). Coba ketik dua perintah berikut, tekan Enter setiap selesai satu baris:

```
1 >>> print("Halo, Studio!")
2 Halo, Studio!
3 >>> 2 + 3
4 5
```

Python langsung menjawab setiap perintah, seperti petugas loket yang menjawab lisan pertanyaan lisan. Jika sudah puas, pamit dengan:

```
1 >>> exit()
```

Tanda `>>>` hilang dan kita kembali ke terminal biasa. Mode interaktif enak untuk coba-coba cepat, tetapi percakapannya **tidak tersimpan**. Untuk pekerjaan yang ingin kita simpan dan ulangi, kita butuh cara kedua: menuliskan perintah ke dalam berkas.

1.3.e Menulis Catatan Kerja: Membuat Skrip `sapa.py` dan Menjalankannya

Daripada mendiktekan perintah satu per satu, kita bisa menuliskan seluruh instruksi kerja di selembar kertas, lalu menyerahkannya ke loket untuk dikerjakan petugas dari atas sampai bawah. Lembar instruksi tertulis itu adalah **skrip**: berkas teks berisi perintah Python, disimpan dengan akhiran `.py`.

Di VS Code, buat berkas baru di dalam folder `skrip` dan beri nama `sapa.py`. Ketik isinya seperti berikut (silakan ganti nama sesuai nama Anda), lalu simpan:

```
1 nama_praktikan = "Aulia"
2 prodi = "Perencanaan Wilayah dan Kota"
3
4 print(f"Halo, {nama_praktikan} dari {prodi}!")
5 print("Selamat datang di praktikum Analitika Perkotaan.")
```

Untuk meminta petugas loket mengerjakannya, panggil `python` diikuti *path* menuju skripnya. Karena terminal kita berdiri di direktori proyek, *path* relatifnya adalah `skrip/sapa.py`:

```
1 python skrip/sapa.py
```

Sel berikut menjalankan isi `sapa.py` yang sama persis, supaya hasilnya terlihat langsung di modul ini. Empat baris kode ini sekaligus memperkenalkan tiga hal paling mendasar dalam Python, yang akan dipakai di **setiap** pertemuan setelah ini:

```
1 nama_praktikan = "Aulia"
2 prodi = "Perencanaan Wilayah dan Kota"
3
4 print(f"Halo, {nama_praktikan} dari {prodi}!")
5 print("Selamat datang di mata kuliah Analitika Perkotaan.")
```

Baris 1-2

`nama_praktikan` dan `prodi` diisi lewat tanda `=`. Keduanya adalah **variabel**: kotak berlabel yang menyimpan nilai, dan tanda `=` di sini berarti **penugasan** (*assignment*), bukan “sama dengan” ala matematika. Rinciannya ada pada kotak “Variabel dan Penugasan Nilai” di bawah.

Baris 4-5

Kedua baris `print()` menampilkan isi ke layar. Baris pertama memakai **f-string** (`f"..."`) untuk menyisipkan nilai `nama_praktikan` dan `prodi` langsung ke dalam teks lewat kurung kurawal `{...}`. Rinciannya ada pada kotak “`print()` dan f-string” di bawah.

Pelajari 3 kotak berikut untuk:

- memahami penggunaan dan penulisan variabel,

- tipe data skalar yang ada di Python beserta perbedaan cara penulisannya, dan
- cara menuliskan perintah penampilan isi ke layar.

Konsep Python

Variabel dan Penugasan Nilai

`nama_praktikan = "Aulia"` artinya: siapkan kotak berlabel `nama_praktikan`, lalu masukkan tulisan "Aulia" ke dalamnya. Kotak berlabel itulah **variabel**, dan tanda `=` disebut **penugasan** (*assignment*): perintah "isikan nilai di kanan ke dalam label di kiri", bukan pernyataan matematika "sama dengan".

Setelah diisi, label bisa dipakai di mana pun untuk memanggil isinya. Itulah sebabnya `print(f"...{nama_praktikan}...")` bisa menampilkan Aulia. Isi kotak juga boleh diganti kapan saja dengan penugasan baru, misalnya `nama_praktikan = "Bima"`; isi lama tertimpa oleh yang baru.

Di studio, variabel seperti kotak berlabel di rak: sekali kotak "data survei" diisi, siapa pun yang butuh tinggal menyebut labelnya tanpa memindahkan isinya ke sana-sini. Nanti kita akan sering menaruh hal-hal panjang (misalnya *path* berkas data) ke dalam variabel pendek agar tidak lelah mengetik.

Konsep Python

Tipe Data Skalar: String dan Angka

Perhatikan perbedaan dua macam isi kotak: "Aulia" ditulis **di antara tanda kutip**, sedangkan angka seperti 5 atau 2026 ditulis polos. Tanda kutip adalah penanda bahwa isinya adalah **teks**: Python menyebutnya **string** (*str*). Tanpa tanda kutip, Python membacanya sebagai **angka**: bilangan bulat (**integer**/*int*, misalnya 2026) atau bilangan berkoma (**float**, misalnya 197.22; perhatikan pemisah desimalnya titik, bukan koma).

Jenis-jenis isi ini disebut **tipe data**, dan yang berisi satu nilai tunggal seperti di atas disebut tipe data **skalar**. Tipe menentukan apa yang bisa dilakukan: angka bisa dihitung (`2 + 3` menghasilkan 5), sedangkan string dirangkai (`"20" + "26"` menghasilkan "2026", bukan 46!). Salah membedakan keduanya adalah sumber kekeliruan yang sangat umum. Kita akan bertemu pesan kesalahannya sebentar lagi.

Konsep Python

`print()` dan f-string

`print()` adalah perintah untuk **menampilkan** isi ke layar; petugas loket membacakan isi kotak yang kita sebut. Apa pun yang ditaruh di dalam tanda kurungnya akan dilaporkan ke terminal.

Bentuk `f"Halo, {nama_praktikan} dari {prodi}!"` disebut **f-string** (huruf *f* di depan tanda kutip artinya *formatted*). Di dalam f-string, bagian yang diapit kurung kurawal `{...}` akan **diganti dengan isi variabelnya** saat ditampilkan. Ini cara paling rapi untuk mencampur teks tetap dengan nilai variabel; tanpa f-string, kita harus menyambung teks potong demi potong.

1.3.f Menulis Perintah di Python

Seperti halnya bahasa manusia, dalam berkomunikasi kita menganut kaidah baku dalam menyampaikan gagasan kita lewatnya. Kaidah itu sesederhana penempatan subjek, predikat, objek, keterangan, dan pelengkap kalimat. Kalau kita asal mencampur kata seperti “Kamu buah lupa keranjang” dan menyampaikan itu ke orang lain, orang lain itu pasti akan menatap kita heran. Syukur-syukur dia bertanya kembali “Maksudnya?”. Akan tetapi, kalau kita mengikuti kaidah, pesan kita akan diterima lebih efektif. “Kamu lupa bawa keranjang untuk buahnya” akan cenderung dijawab “Oh ya, terima kasih sudah mengingatkan”, menandakan pesan kita sampai.

Sama halnya untuk komputer. Ada kaidah dalam menyampaikan perintah, yang kalau tidak diikuti akan muncul pesan *error*, selayaknya manusia yang bertanya kembali “maksudnya?” tadi. Untuk dapat menuliskan perintah yang efektif, pelajari 2 kotak konsep mengenai sintaks perintah dalam Python berikut.

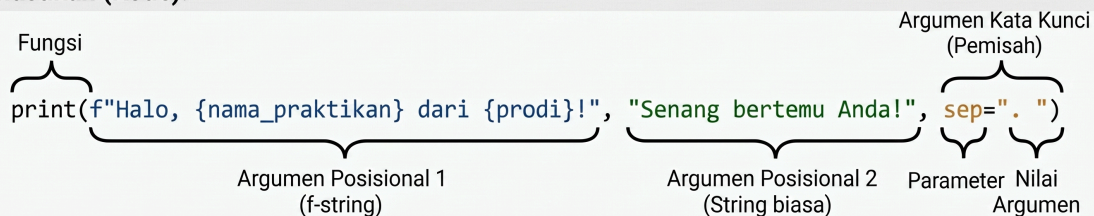
Konsep Python

Anatomi Sintaks Python

Perhatikan kembali baris `print(f"Halo, {nama_praktikan} dari {prodi}!")`. Baris itu punya beberapa bagian dengan nama masing-masing, dan mengenalinya akan sangat membantu saat nanti Anda membaca pesan galat atau dokumentasi resmi Python.

Sekarang perhatikan ilustrasi ini dan penjelasan bagian-bagiannya.

Masukan (Kode):



Dijalankan oleh Python

Keluaran (Output):

Halo, Angga dari PWK! . Senang bertemu Anda!

```
nama_praktikan = "Angga"
prodi = "PWK"
```

- `print` adalah sebuah **fungsi** (*function*) — perintah siap pakai yang sudah dibuatkan orang lain, tinggal dipanggil. Tanda kurung `()` setelah nama fungsi menandai **pemanggilan fungsi** (*function call*): “Tolong jalankan perintah ini sekarang.”
- Apa pun yang dituliskan di dalam tanda kurung itu disebut **argumen** (*argument*) — nilai yang Anda kirim ke fungsi supaya fungsi tahu apa yang harus dikerjakannya. Pada `print(f"Halo, {nama_praktikan} dari {prodi}!")`, argumennya adalah string hasil f-string itu sendiri. Karena argumen ini dikirim tanpa nama, ia disebut **argumen posisional** (*positional argument*): Python

mengenalinya semata dari urutan penulisan, bukan dari nama. Ini juga berlaku untuk "Senang bertemu Anda!".

- Sebagian fungsi menerima lebih dari satu hal, dan masing-masing tempat penerimanya sudah punya nama yang ditentukan sejak fungsi itu dibuat. Nama itu disebut **parameter** — label pada slot yang menunggu diisi. Jangan tertukar dengan argumen: **parameter adalah nama slotnya, argumen adalah isi yang Anda masukkan ke slot itu.**
 - Ketika Anda mengisi slot itu dengan menyebut namanya secara eksplisit, misalnya `sep="-"` pada `print("a", "b", sep="-")`, pasangan `nama_parameter=nilai` itu disebut **argumen kata kunci** atau **argumen bernama** (*keyword argument*). Bedanya dari argumen posisional: karena disertai nama, urutan penulisannya jadi bebas — Python tetap tahu ke slot mana nilai itu harus masuk.

Anda akan sering berjumpa argumen kata kunci mulai Modul 2, ketika `pd.read_csv(path_data, usecols=kolom_pilihan)` memakai parameter `usecols` untuk memilih kolom yang dibaca.

Konsep Python

Modul dan Notasi Titik

`import sys` di atas mendatangkan **modul** (*module*) bernama `sys` ke buku lapangan Anda — modul adalah kumpulan perintah dan informasi siap pakai yang ditulis orang lain, dikelompokkan dalam satu paket bernama.

Untuk mengambil sesuatu dari dalam modul, Python memakai tanda titik: `sys.executable`. Ini disebut **notasi titik** (*dot notation*) — “dari objek di sebelah kiri titik, ambil bagian di sebelah kanan titik.” Pada `sys.executable`, `executable` adalah informasi (alamat interpreter Python yang sedang aktif) yang menempel pada modul `sys`.

Notasi titik yang sama akan terus muncul di modul-modul berikutnya — misalnya `pd.read_csv(...)`, yang mengambil fungsi `read_csv` dari dalam modul `pd`.

Notasi titik juga dipakai untuk hal lain: memanggil kemampuan yang sudah menempel pada data yang Anda buat sendiri, misalnya `df.head()` yang akan Anda temui di Modul 2. Itu disebut **method** — sekilas mirip fungsi, tapi tidak berdiri sendiri seperti `print()`. Method ini akan dibahas begitu pertama kali dipakai.

1.3.g Membaca Pesan Kegagalan: Tiga Error Paling Umum dan Debugging Dasar

Cepat atau lambat (biasanya cepat), kode kita akan gagal berjalan dan Python menampilkan tulisan merah yang panjang. Jangan panik: pesan itu bukan kemarahan, melainkan **petugas loket yang balik bertanya** karena instruksi kita tidak bisa dia pahami atau kerjakan. Seperti peserta studio yang menerima catatan revisi dari asisten studio, perencana yang baik tidak takut pada catatan revisi; programmer yang baik tidak takut pada pesan *error*.

Kunci membacanya: lihat **baris paling bawah** terlebih dahulu. Di sanalah tertulis **nama jenis kesalahannya** beserta keterangan singkat, sedangkan baris-baris di atasnya (disebut *traceback*) menunjukkan di berkas dan baris ke berapa kesalahan terjadi. Tiga jenis inilah yang paling sering

menyapa pemula. Kita sengaja memproduksi ketiganya sekarang, supaya saat muncul sungguhan kita sudah kenal wajahnya.

Wajah pertama, `SyntaxError`: kalimat instruksinya cacat. Python punya tata bahasa. Lupa menutup tanda kutip atau tanda kurung membuat kalimat kita tidak selesai, dan Python menolak bahkan untuk mulai bekerja. Jalankan sel berikut yang lupa menutup tanda kutip dan kurung:

```
1 print("Halo, Studio!
```

```
SyntaxError: unterminated string literal (detected at line 1) (501600097.py, line 1)
Cell In[2], line 1
    print("Halo, Studio!
          ^
SyntaxError: unterminated string literal (detected at line 1)
```

Perhatikan baris terakhirnya: nama kesalahannya `SyntaxError`, dan tanda ^ menunjuk lokasi Python kebingungan. Perbaikannya: lengkapi pasangan tanda yang hilang. Setiap " dan (harus ada penutupnya.

Wajah kedua, `NameError`: menyebut label kotak yang tidak ada. Kesalahan ini muncul saat kita memanggil variabel yang belum pernah diisi, paling sering karena **salah ketik**. Di sel berikut, kita bermaksud memanggil `nama_praktikan` tetapi namanya kurang satu huruf:

```
1 print(nama_praktika)
```

```
NameError: name 'nama_praktika' is not defined
-----
NameError                                Traceback (most recent call last)
Cell In[3], line 1
----> 1 print(nama_praktika)

NameError: name 'nama_praktika' is not defined
```

Python menjawab `NameError: name 'nama_praktika' is not defined`: “tidak ada kotak berlabel itu di rak”. Perbaikannya: cocokkan ejaan nama variabel dengan yang didefinisikan, huruf demi huruf (huruf besar-kecil dibedakan!). Pada Python versi terbaru, sesekali pesan ini malah berbaik hati menebak maksud kita dengan saran `Did you mean: ...?`.

Wajah ketiga, `TypeError`: menyuruh dua tipe data yang tak cocok bekerja sama. Ingat pembahasan tipe data tadi: string dan angka punya kemampuan berbeda. Menyambung teks dengan angka memakai + membuat Python bingung harus menghitung atau merangkai:

```
1 "Kota Bandar Lampung berusia " + 342
```

```
TypeError: can only concatenate str (not "int") to str
-----
TypeError                                Traceback (most recent call last)
Cell In[4], line 1
----> 1 "Kota Bandar Lampung berusia " + 342

TypeError: can only concatenate str (not "int") to str
```

Baris terakhirnya `TypeError: can only concatenate str (not "int") to str`: “saya hanya bisa menyambung string dengan string, bukan dengan bilangan bulat”. Perbaikannya yang paling rapi sudah kita kenal: bungkus dengan f-string, `print(f"Kota Bandar Lampung berusia {342}")`, dan Python otomatis mengubah angkanya menjadi teks.

Dari tiga pengenalan ini, lahirlah rutinitas **debugging** (mencari dan memperbaiki kesalahan) yang akan kita pakai sepanjang semester:

1. Baca **baris paling bawah**: apa nama jenis kesalahannya?
2. Lihat **nomor baris** yang ditunjuk *traceback*: di mana lokasinya?
3. Perbaiki satu hal itu saja, lalu **jalankan ulang**.
4. Masih gagal? Ulangi dari langkah 1. Setiap pesan baru adalah petunjuk baru.

1.3.h Membandingkan Dua Berkas: Mengapa Satu File Harus Lengkap-Mandiri

Sekarang eksperimen kecil yang hasilnya akan kita tagih lagi di Modul 2. Buat **dua** berkas baru di folder *skrip*.

Berkas pertama, `hitung_lengkap.py`, berisi perhitungan kepadatan penduduk Kota Bandar Lampung *beserta* definisi variabelnya:

```
1 luas_wilayah_km2 = 197.22
2 jumlah_penduduk = 1166066
3
4 kepadatan = jumlah_penduduk / luas_wilayah_km2
5 print(f"Kepadatan penduduk: {kepadatan:.0f} jiwa per km persegi")
```

Berkas kedua, `hitung_pincang.py`, berisi **hanya** perhitungannya. Definisi variabelnya sengaja tidak ditulis:

```
1 kepadatan = jumlah_penduduk / luas_wilayah_km2
2 print(f"Kepadatan penduduk: {kepadatan:.0f} jiwa per km persegi")
```

Jalankan keduanya bergantian dari terminal:

```
1 python skrip/hitung_lengkap.py
2 python skrip/hitung_pincang.py
```

Hasilnya:

```
Kepadatan penduduk: 5913 jiwa per km persegi
Traceback (most recent call last):
  File "skrip/hitung_pincang.py", line 1, in <module>
    kepadatan = jumlah_penduduk / luas_wilayah_km2
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
NameError: name 'jumlah_penduduk' is not defined
```

Berkas pertama berhasil; berkas kedua gagal dengan `NameError`, yang merupakan wajah kedua yang baru saja kita kenal. Padahal `hitung_lengkap.py` baru saja dijalankan dan variabelnya baru saja diisi. Mengapa `hitung_pincang.py` tidak bisa ikut memakainya?

Karena **setiap kali python berkas.py dijalankan, komputer membuka “sesi kerja” yang benar-benar baru**: papan tulisnya kosong, raknya kosong, tidak ada ingatan apa pun dari eksekusi sebelumnya maupun dari berkas lain. Begitu skrip selesai, sesi itu ditutup dan seluruh isinya dibuang. Dua pelajaran penting:

1. **Satu berkas skrip harus lengkap-mandiri**: semua variabel yang dipakai harus didefinisikan di dalam berkas itu sendiri, seperti lembar instruksi kerja yang harus bisa dikerjakan petugas loket tanpa bertanya ke lembar lain.
2. Ingat baik-baik perilaku “tidak ada ingatan” ini. Pada Modul 2, kita akan bertemu Jupyter Notebook yang perilakunya **justru sebaliknya**, dan perbedaan itulah sumber kebingungan mahasiswa yang paling umum.

1.3.i Menyiapkan Buku Lapangan: Membuat Jupyter Notebook dan Menyambungkan Kernel

Peralatan terakhir yang kita siapkan hari ini adalah **Jupyter Notebook**: buku catatan lapangan seorang perencana-analis. Di lapangan, satu buku catatan memuat tulisan pengamatan, angka pengukuran, dan sketsa dalam halaman yang sama. Notebook persis seperti itu: dalam satu berkas `.ipynb` kita bisa menulis narasi, kode Python, sekaligus melihat hasil dan grafiknya. Hampir seluruh praktikum kita mulai Modul 2 akan tinggal di dalam notebook.

Buat notebook pertama Anda:

1. Di VS Code, klik kanan folder `modul` → **New File** → beri nama `buku-lapangan.ipynb`. VS Code otomatis membuka tampilan notebook yang tersusun atas **sel-sel**.
2. Di pojok kanan atas, klik **Select Kernel** → **Python Environments** → pilih `analitika-perkotaan`.

Langkah kedua itu yang paling penting. **Kernel** adalah mesin Python yang menjalankan **sel-sel notebook**. Kita baru saja menyambungkan notebook ke mesin dari *environment* buatan kita sendiri, lengkap dengan semua *library* yang tadi dipasang lewat `environment.yml`. Notebook tanpa kernel seperti buku lapangan tanpa surveyor: catatannya ada, tetapi tidak ada yang menghitung.

3. **Buktikan kernelnya benar**. Sama seperti papan nama terminal di langkah kelima tadi, nama kernel yang tampil juga baru sebatas klaim. Buat sel pertama berisi dua baris berikut, lalu jalankan dengan tombol ▶ di kiri sel (atau `Shift+Enter`):

```
1 import sys
2 print(sys.executable)
```

Path yang tercetak **harus** memuat potongan `envs\analitika-perkotaan`. Jika yang muncul lokasi lain, kembali ke langkah 2 dan pilih ulang kernelnya; bila tetap tidak berubah, laporkan ke dosen.

⚠ Nama Kernel Bisa Berbohong

Daftar kernel yang Anda lihat di VS Code tidak dibaca langsung dari daftar *environment* Conda. Daftar itu dibaca dari berkas-berkas kecil bernama `kernel.json`, dan di dalam berkas itu **nama yang ditampilkan** disimpan terpisah dari **Python yang benar-benar dijalankan**. Keduanya bisa tidak

cocok: sebuah kernel bertuliskan “Analitika Perkotaan” sepenuhnya sah menunjuk Python mana pun di komputer, termasuk Python lama dengan versi *library* yang berbeda.

Kasus semacam ini bisa saja terjadi: sebuah kernel bernama `analitika-perkotaan` diam-diam menjalankan Python lain dengan versi Pandas yang lebih tua, dan akibatnya baru diketahui setelah keluaran notebook dipenuhi peringatan yang tidak semestinya ada. Satu baris `print(sys.executable)` di awal akan menghemat berjam-jam penelusuran.

4. Ketik perintah berikut di sel kedua, lalu jalankan:

```
1 print("Buku lapangan siap!")
```

```
Buku lapangan siap!
```

Jika tulisan di atas muncul di bawah sel Anda, seluruh meja kerja digital kita sudah beres: direktori proyek tertata, *environment* terpasang, terminal sudah bisa diajak bicara, dan buku lapangan tersambung ke mesinnya. Bagaimana sel-sel notebook ini sesungguhnya bekerja, dan mengapa perilakunya berlawanan dengan dua berkas `.py` tadi, sengaja kita simpan sebagai pembuka Modul 2.

1.4 Latihan Mandiri

Kerjakan berurutan. Alurnya sama persis dengan instruksi yang baru kita lalui. Simpan semua berkasnya, karena akan diperiksa di awal pertemuan berikutnya.

1. **Menulis skrip.** Buat berkas baru `skrip/identitas.py` yang mendefinisikan **tiga variabel**: `nama` (string), `nim` (angka), dan `kelas_studio` (string), lalu menampilkan ketiganya ke terminal dalam **satu kalimat utuh** menggunakan `print()` dan f-string: “Izin memperkenalkan diri, saya <nama>, NIM <nim>, dari kelas studio <kelas_studio>. Mohon kerja samanya, ya!”
2. **Menjalankan dari terminal.** Jalankan skrip tersebut dari terminal yang berdiri di direktori proyek. Tuliskan (di komentar paling bawah berkas, diawali tanda `#`) perintah apa yang Anda pakai.
3. **Memproduksi tiga kegagalan.** Rusak skrip Anda secara sengaja, **satu kesalahan pada satu waktu** (perbaiki dulu sebelum membuat kesalahan berikutnya), sehingga ketiga *error* yang kita kenal muncul semua:
 - a. Hapus satu tanda kutip penutup → jalankan → amati **SyntaxError**.
 - b. Salah-ketikkan nama variabel di dalam `print()` → jalankan → amati **NameError**.
 - c. Ganti f-string dengan penyambungan langsung `"NIM saya " + nim` → jalankan → amati **TypeError**.

Untuk setiap kegagalan, catat di komentar: apa **nama error**-nya dan apa **baris terakhir** pesannya. Setelah selesai, kembalikan skrip ke kondisi benar.

4. **Menutup di buku lapangan.** Buka `modul/buku-lapangan.ipynb`, pastikan kernel `analitika-perkotaan` tersambung, lalu buat sel yang menampilkan kalimat identitas yang sama seperti langkah 1. Anda akan sadar harus **mendefinisikan ulang** ketiga variabelnya di dalam notebook. Jelaskan dalam satu kalimat (di sel teks/markdown) mengapa variabel dari `identitas.py` tidak bisa langsung dipakai di sini.
5. **Membuktikan ruang kerja.** Dengan papan nama (`analitika-perkotaan`) menyala, jalankan lagi ketiga perintah pembuktian dari langkah kelima:

```
1  where.exe python
2  python -c "import sys; print(sys.prefix)"
3  python -c "import sys; print(sys.path)"
```

Tempelkan ketiga keluarannya sebagai komentar (diawali tanda #) di bagian paling bawah `skrip/identitas.py`. Lalu tuliskan, masih sebagai komentar, satu kalimat yang menjawab: **“Python mana yang sesungguhnya menjalankan kode saya, dan dari mana saya tahu?”** Sebutkan pula apakah ruang kerja Anda memenuhi ketiga syarat lulus atau tidak; jika tidak, sebutkan syarat nomor berapa yang gagal. Ini bukan sekadar anjuran: ruang kerja yang tidak terbukti akan menyulitkan Anda di setiap modul setelah ini.

2 Pengolahan Data Terstruktur dengan Pandas

! Sub-CPMK Pertemuan Ini

Mahasiswa mampu mendemonstrasikan pengolahan data terstruktur dasar menggunakan Pandas, mengidentifikasi unit amatan dan unit analisis dari dataset perkotaan, serta memecahkan masalah kebingungan urutan eksekusi sel (*state* bersama) pada Jupyter Notebook.

2.1 Pendahuluan

<masukkan narasi yang menceritakan 'big why' mengelola data perkotaan yang terstruktur di sini>

2.2 Gambaran Umum Kasus

Pemerintah Kota Bandar Lampung hendak mengalokasikan bantuan pembangunan, dan pertanyaan pertamanya sederhana: kelurahan mana yang paling membutuhkan? Di *command center* kota, pertanyaan seperti ini tidak dijawab dengan tebakan atau kesan lapangan semata, melainkan dengan data yang dipantau layar demi layar oleh operator — berapa kepadatan penduduk tiap kelurahan, seberapa lengkap fasilitas pendidikan usia dininya, dan seterusnya. Data yang dipantau layar demi layar ini diproses sedemikian rupa sehingga dapat memenuhi kebutuhan. Pemrosesan data tersebut memungkinkan bila data kita mengorganisasinya dalam bentuk yang disebut dengan **data terstruktur**.

Kali ini kita berkenalan dengan **Pandas**, *library* Python untuk mengolah tabel rekapitulasi, sebagai alat kerja utama operator *command center* dalam memantau kondisi tiap kelurahan dan mengambil keputusan alokasi. Untuk praktik sekarang, kita akan menggunakan data PODES 2021 Kota Bandar Lampung, data dengan unit amatan kelurahan yang dikumpulkan BPS. Data untuk *command center* tentunya dikumpulkan dengan lebih *real-time*. Namun sebelum menyentuh data, kita tuntaskan dulu janji Modul 1: **membuktikan cara kerja sel Jupyter Notebook**, yang perilakunya berlawanan dengan dua berkas `.py` kita.

2.3 Instruksi

1. Buat *notebook* baru `modul/modul-02.ipynb` di direktori proyek Anda.
2. Sambungkan *notebook* ke *kernel* `analitika-perkotaan`.
3. Kerjakan seluruh instruksi di bawah ini secara berurutan.

2.3.a Jembatan Jupyter: Membuktikan Perilaku Sel dan State Bersama

Di Modul 1, `hitung_pincang.py` gagal karena setiap eksekusi `python berkas.py` adalah sesi kerja baru yang tidak mengingat apa pun. Sekarang kita reproduksi kasus yang sama di *notebook* — tetapi kali ini “berkas pincang”-nya kita pecah menjadi **dua sel**. Jalankan sel pertama:

Lalu jalankan sel kedua, yang hanya berisi perhitungan — persis si `hitung_pincang.py`:

```
1 luas_wilayah_km2 = 197.22
2
3 jumlah_penduduk = 1166066
```

Baris 1,3

Kedua variabel didefinisikan dan menyimpan nilai angka ke papan tulis *kernel*.

```
1 kepadatan = jumlah_penduduk / luas_wilayah_km2
2
3 print(f"Kepadatan penduduk: {kepadatan:.0f} jiwa per km persegi")
```

Baris 1

Variabel `kepadatan` dihitung dari dua variabel yang didefinisikan di sel sebelumnya.

Baris 3

`print()` menampilkan hasilnya dengan format satu digit di belakang koma (`:.0f`).

```
Kepadatan penduduk: 5913 jiwa per km persegi
```

Berhasil! Padahal sel kedua tidak mendefinisikan variabelnya sendiri. Inilah perbedaan mendasarnya: sel-sel notebook **bukan** berkas-berkas terpisah, melainkan potongan-potongan percakapan dalam **satu sesi yang sama** — seperti saat kita mengetik berurutan di mode interaktif `>>>` kemarin, hanya saja kini tiap potongannya tersimpan rapi.

Analogi studionya: *kernel* adalah **papan tulis studio** tempat kita menulis bersama. Di sel pertama, kita menuliskan dua angka di papan; di sel kedua kita tinggal menoleh ke papan yang sama untuk memakainya tanpa harus menulis ulang. Segala coretan yang telah kita tulis di papan — variabel yang pernah kita isi — akan tetap bertahan di sana dan siap kita gunakan di sel mana pun. Kondisi papan tulis pada satu waktu tertentu itulah yang disebut dengan *state* yang **dibagi bersama** oleh semua sel.

2.3.b Eksperimen Papan Tulis: Mengapa Urutan Eksekusi Sel Sangat Menentukan Hasil

Papan tulis bersama itu nyaman, tetapi menyimpan satu jebakan yang membingungkan hampir semua pemula. Mari kita masuk ke jebakannya dengan sadar. Jalankan sel berikut, yang mengganti isi variabel `luas` dengan angka yang jelas-jelas salah:

```
1 luas_wilayah_km2 = 20
```

Baris 1

Variabel `luas_wilayah_km2` ditimpa dengan angka yang sengaja salah untuk eksperimen.

Sekarang **kembalilah ke sel perhitungan kepadatan di atas dan jalankan ulang sel itu** (klik selnya, lalu `Shift+Enter`). Hasilnya melonjak menjadi lebih dari 58 ribu jiwa per km persegi — karena perhitungan menoleh ke papan tulis dan menemukan angka `20` yang baru. Di modul ini kita tuliskan ulang perhitungannya sebagai sel berikut agar lonjakannya terlihat:

```

1  kepadatan = jumlah_penduduk / luas_wilayah_km2
2
3  print(f"Kepadatan penduduk: {kepadatan:.0f} jiwa per km persegi")

```

Baris 1

Perhitungan yang sama persis menghasilkan angka berbeda karena papan tulis *kernel* kini berisi angka 20.

Baris 3

Hasilnya melonjak menjadi lebih dari 58 ribu jiwa per km persegi.

```
Kepadatan penduduk: 58303 jiwa per km persegi
```

Perhatikan dua hal:

1. Hasil sebuah sel **tidak ditentukan oleh posisinya di halaman**, melainkan oleh **urutan eksekusinya**. Sel yang tampak lebih atas bisa saja dijalankan belakangan dan memakai nilai yang lebih baru.
2. Angka dalam tanda kurung siku di kiri sel (misalnya [5]) adalah **nomor antrean eksekusi**. Kalau nomornya melompat-lompat tidak berurutan dari atas ke bawah, hasil yang tampil bisa jadi berasal dari papan tulis yang keadaannya sudah berubah.

⚠ Penting!

Papan Tulis Tidak Menghapus Dirinya Sendiri

Coretan di papan tulis studio akan bertahan sampai kita sendiri menghapusnya — meski kita sudah selesai bekerja. Begitu pula variabel di *kernel*: sekali terisi, ia bertahan dan memengaruhi sel mana pun yang dijalankan setelahnya, **meskipun sel yang mengisinya sudah kita hapus dari notebook**.

Karena itu, biasakan ritual studio ini: sebelum kita mengumpulkan atau mempercayai hasil akhir notebook kita, jalankan **Restart kernel & Run All** (hapus papan tulis, lalu kerjakan seluruh sel berurutan dari atas). Jika ada sel yang gagal, berarti notebook Anda belum lengkap-mandiri secara berurutan — persis prinsip berkas `.py` di Modul 1, kini dalam versi notebook.

Sebelum lanjut, kembalikan papan tulis ke angka yang benar:

```
1  luas_wilayah_km2 = 197.22
```

Baris 1

Variabel `luas_wilayah_km2` dikembalikan ke nilai semula.

Kemudian jalankan lagi sel berisi perhitungan `kepadatan` untuk mengembalikan nilai benarnya.

2.3.c Membuka Rekap Data: Membaca File CSV dengan Pandas

Sekarang kasus intinya. Data PODES kelurahan Bandar Lampung tersimpan sebagai berkas **CSV** (*comma-separated values*): berkas teks biasa yang tiap barisnya adalah satu catatan dan tiap kolomnya dipisahkan koma — bentuk paling sederhana dari **data terstruktur**, data yang tertata dalam baris dan kolom bermakna tetap. Baris pertamanya berisi nama-nama kolom.

1. Pada *side bar Explorer*, klik kanan berkas `bdL_kelurahan.csv`.
2. Klik **Open With...**, lalu pilih **Text Editor**. **. Ini sama seperti kita membukanya dengan Notepad. Isinya kira-kira:

```
kode_prov,nama_prov,kode_kab,nama_kab,kode_kec,nama_kec,kode_des,nama_des,...  
  
18,Lampung,71,Kota Bandar Lampung,10,Teluk Betung Barat,5,Bakung,...
```

Membaca teks seperti itu dengan mata tentu melelahkan. Di sinilah **Pandas** bekerja: ia memuat CSV menjadi tabel rekapitulasi yang rapi — persis lembar rekap kompilasi data studio di Excel — yang di Pandas disebut **DataFrame**.

Berkas kita berisi 34 kolom, padahal pertemuan ini hanya butuh sebagian. Maka sebelum membaca, kita siapkan dua variabel penampung: `kolom_pilihan` untuk daftar kolom yang ingin dibaca dan `path_data` untuk alamat berkasnya (ingat, notebook kita berada di folder `modul`, sehingga `path` relatifnya naik satu tingkat dulu dengan `..`). Dengan begitu, perintah bacanya sendiri tetap pendek:

```
1 import pandas as pd  
2  
3 kolom_pilihan = ["nama_kec", "nama_des", "status_daerah",  
4                  "kepadatan_pddk", "jml_tk_negeri", "jml_tk_swasta",  
5                  "jml_ra_ba_negeri", "jml_ra_ba_swasta"]  
6  
7 path_data = "../dataset-ganjil-2026-2027/bdL_kelurahan.csv"  
8  
9 df = pd.read_csv(path_data, usecols=kolom_pilihan)  
10 df.head()
```

Baris 1

`import pandas as pd` memanggil *library* Pandas dari rak *environment* kita dan memberinya nama panggilan pendek `pd` — cukup sekali di awal notebook.

Baris 9

`pd.read_csv(...)` membaca berkas pada variabel `path_data`, mengambil hanya kolom-kolom dalam variabel `kolom_pilihan` (lewat parameter `usecols=`), lalu hasilnya kita tampung di variabel `df`.

Baris 10

`df.head()` mengintip **lima baris pertama** tabel untuk memastikan bacaan kita benar.

Pelajari kotak Konsep Python tentang *method* dan tipe data *list* yang digunakan dalam perintah di atas.

Konsep Python

Metode: Fungsi yang Menumpang pada Data

Perhatikan `df.head()` pada perintah di atas. Sekilas mirip pemanggilan fungsi biasa — ada nama, ada tanda kurung — tapi ada satu perbedaan penting: `head` tidak berdiri sendiri seperti `print()` atau `pd.read_csv()`. Ia ditulis menempel pada `df`, dipisahkan tanda titik.

Perintah semacam ini disebut **metode** (*method*) — sederhananya, metode adalah **fungsi yang “menumpang” pada sebuah data**, bukan fungsi yang berdiri sendiri. Bayangkan bedanya seperti alat ukur lepas yang siapa saja bisa pakai (fungsi biasa, mis. `print()`) dengan kemampuan bawaan yang sudah melekat pada suatu berkas — semacam “cetak ringkasan diri sendiri” yang cuma bisa dijalankan oleh berkas itu, karena ia tahu isi dirinya.

Karena `df` adalah tabel data (*DataFrame*), `df.head()` berarti “hai `df`, tunjukkan lima baris pertama dirimu sendiri.” Metode tidak bisa dipanggil sendirian — menuliskan `head()` saja tanpa `df`. di depannya akan menimbulkan galat, karena ia hanya masuk akal ketika ditumpangkan ke data yang memang punya kemampuan itu.

Bandingkan dengan `pd.read_csv(...)` yang sudah Anda kenal dari sel di atas: itu tetap **fungsi biasa**, hanya kebetulan disimpan di dalam modul `pd` (lihat kembali callout “Modul dan Notasi Titik” di Modul 1) — dipanggil dengan pola `modul.fungsi()`. Method dipanggil dengan pola `objek.method()`, dan *objek*-nya adalah data yang sudah Anda buat sendiri, seperti `df`.

Anda akan sering menumpuk beberapa method sekaligus, misalnya `df.isna().sum()` — dibaca dari kiri ke kanan: “dari `df`, periksa mana yang kosong, lalu hitung jumlahnya.” Menumpuk method secara berurutan seperti ini disebut **method chaining**.

Konsep Python

Tipe Data list

Variabel `kolom_pilihan` adalah sebuah **list**: tipe data yang menampung **banyak nilai sekaligus dalam satu kotak, berurutan**. Penulisannya memakai kurung siku, dengan koma sebagai pemisah: `["nama_kec", "nama_des", ...]`. Kalau variabel biasa adalah satu kotak berlabel, *list* adalah **rak berlabel yang isinya bernomor urut**.

Nomor urut itu disebut **indeks**, dan — ini kekhasan Python — **dihitung mulai dari 0**: `kolom_pilihan[0]` berisi `"nama_kec"`, `kolom_pilihan[1]` berisi `"nama_des"`, dan seterusnya. Panjang *list* bisa ditanyakan dengan `len(kolom_pilihan)`.

list akan sangat sering kita pakai untuk menampung daftar nama kolom seperti ini — daripada mengetik daftar panjang berulang-ulang di banyak perintah, kita tulis sekali di satu variabel lalu sebut labelnya.

2.3.d Mengenal Struktur Tabel: Anatomi Baris, Kolom, dan Tipe Data DataFrame

Data yang baru masuk ke *command center* selalu kita periksa dulu kelengkapannya sebelum dipakai untuk pemantauan. Untuk *DataFrame*, pemeriksaan pertamanya adalah `info()`:

```
1 df.info()
```

Baris 1

`info()` mencetak ringkasan struktur tabel: jumlah baris, nama kolom, tipe data, dan cacah nilai terisi per kolom.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 126 entries, 0 to 125
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   nama_kec              126 non-null   object
1   nama_des              126 non-null   object
2   status_daerah         126 non-null   int64
3   jml_tk_negeri         126 non-null   int64
4   jml_tk_swasta         126 non-null   int64
5   jml_ra_ba_negeri      126 non-null   int64
6   jml_ra_ba_swasta      126 non-null   int64
7   kepadatan_pddk        126 non-null   int64
dtypes: int64(6), object(2)
memory usage: 8.0+ KB
```

Laporan itu padat informasi. Bacalah dari atas:

- **126 entries** — tabel berisi **126 baris**, dan setiap baris adalah **satu kelurahan**. Di sinilah bedanya dengan tabel Excel bebas: dalam data terstruktur, satu baris selalu mewakili satu satuan pencatatan yang sama.
- Daftar kolom beserta **Non-Null Count** — berapa baris yang **terisi** di tiap kolom. Semuanya **126 non-null**, artinya tidak ada data kosong pada kolom-kolom pilihan kita (kemewahan yang tidak selalu terjadi; menangani data kosong adalah porsi Modul 4).
- **Dtype** — **tipe data** tiap kolom, saudara dari tipe data skalar di Modul 1: **int64** untuk bilangan bulat, **float64** untuk bilangan berkoma, dan **object** untuk teks/string seperti nama kecamatan.

Untuk kolom-kolom angka, kita bisa langsung meminta ringkasan statistiknya:

```
1 df.describe()
```

Baris 1

`describe()` menghitung sekaligus: jumlah data (**count**), rata-rata (**mean**), simpangan baku (**std**), nilai terkecil-terbesar (**min**, **max**), dan kuartil (25%, 50%, 75%).

	status_daerah	jml_tk_negeri	jml_tk_swasta	jml_ra_ba_negeri \
count	126.000000	126.000000	126.000000	126.0
mean	1.015873	0.103175	2.595238	0.0
std	0.125483	0.330560	2.075297	0.0
min	1.000000	0.000000	0.000000	0.0
25%	1.000000	0.000000	1.000000	0.0
50%	1.000000	0.000000	2.000000	0.0
75%	1.000000	0.000000	4.000000	0.0
max	2.000000	2.000000	10.000000	0.0

	jml_ra_ba_swasta	kepadatan_pddk
count	126.000000	126.000000
mean	0.269841	9431.825397
std	0.662271	8428.116609

min	0.000000	627.000000
25%	0.000000	3466.750000
50%	0.000000	6450.500000
75%	0.000000	12693.500000
max	4.000000	43907.000000

Dari satu tabel ini saja sudah terbaca, misalnya, betapa timpangnya kepadatan penduduk antar-kelurahan: ada kelurahan yang hanya ratusan jiwa per km², ada yang lebih dari 40 ribu. Pendalaman statistik deskriptif semacam ini adalah menu utama Modul 4 — hari ini cukup berkenalan.

2.3.e Kelurahan vs Kecamatan: Membedakan Unit Amatan dan Unit Analisis

Perhatikan lagi: setiap baris tabel kita adalah satu **kelurahan** — pada level itulah BPS mencatat datanya. Dalam bahasa metodologi, kelurahan adalah **unit amatan** (*unit of observation*) dataset ini: satuan tempat data dicatat, seperti satu lembar kuesioner terisi untuk satu kelurahan.

Namun pertanyaan perencanaan tidak selalu dijawab pada level pencatatan. Ketika pimpinan bertanya “kecamatan mana yang paling minim fasilitas PAUD?”, kesimpulannya diambil pada level **kecamatan** — itulah **unit analisis** (*unit of analysis*): satuan tempat kesimpulan dibuat, seperti lembar rekapitulasi per wilayah yang merangkum tumpukan kuesioner. Unit amatan dan unit analisis boleh sama, boleh berbeda; yang fatal adalah tidak sadar sedang bekerja di level yang mana.

Mari kenali kedua level itu di data kita. Satu kolom di *DataFrame* dipanggil dengan `df["nama_kolom"]`, dan `nunique()` menghitung banyaknya **nilai unik** (nilai yang berbeda-beda) di dalamnya:

```
1 print(f"Jumlah kelurahan (unit amatan) : {df['nama_des'].nunique()}")
2
3 print(f"Jumlah kecamatan : {df['nama_kec'].nunique()}")
```

Baris 1

`nunique()` menghitung banyaknya **nilai unik** di sebuah kolom — berapa nama kelurahan yang berbeda-beda.

Baris 3

Kolom `nama_kec` dihitung terpisah untuk mengetahui berapa kecamatan yang menaungi kelurahan-kelurahan tersebut.

```
Jumlah kelurahan (unit amatan) : 126
Jumlah kecamatan : 20
```

126 kelurahan tersebar di 20 kecamatan. Lalu berapa kelurahan yang dinaungi masing-masing kecamatan? `value_counts()` menghitung **frekuensi kemunculan** tiap nilai pada sebuah kolom — alat andalan untuk meringkas **data kategorikal** (data berupa golongan/kategori, bukan angka hitungan):

```
1 df["nama_kec"].value_counts()
```

Baris 1

`value_counts()` menghitung frekuensi kemunculan tiap nilai pada kolom `nama_kec`, diurutkan dari yang terbanyak.

```

nama_kec
Kemiling          9
Panjang           8
Tanjung Karang Barat 7
Sukabumi          7
Kedamaian        7
Rajabasa          7
Tanjung Karang Pusat 7
Kedaton           7
Sukarame          6
Labuhan Ratu      6
Way Halim         6
Telukbetung Timur 6
Enggal            6
Teluk Betung Utara 6
Teluk Betung Selatan 6
Langkapura        5
Tanjung Senang    5
Tanjung Karang Timur 5
Bumi Waras        5
Teluk Betung Barat 5
Name: count, dtype: int64

```

Tabel kecil itu sudah merupakan rekapitulasi pada unit analisis kecamatan: Teluk Betung Selatan menaungi paling banyak kelurahan, dan seterusnya. Simpan baik-baik `value_counts()` di ingatan — di Latihan Mandiri ia akan dipakai untuk meringkas kategori yang lebih bermakna sosial-ekonomi, dan di Modul 3 kita akan membuat rekapitulasi per kecamatan yang lebih kaya menggunakan SQL.

2.3.f Merapikan Tabel: Membuat, Menyusun Ulang, dan Menghapus Kolom

Fasilitas pendidikan usia dini di data kita terpecah menjadi empat kolom: TK negeri, TK swasta, RA/BA negeri, dan RA/BA swasta (RA/BA adalah *raudatul athfal/bustanul athfal*, PAUD di bawah Kemenag). Untuk pertanyaan alokasi bantuan, yang lebih berguna adalah **totalnya**. Di layar *command center*, kita akan menambahkan satu kolom jumlahan di ujung tabel supaya totalnya langsung terlihat operator — di Pandas pun sama: menugaskan nilai ke nama kolom yang belum ada akan **membuat kolom baru**:

```

1 df["jml_paud"] = (
2     df["jml_tk_negeri"] + df["jml_tk_swasta"]
3     + df["jml_ra_ba_negeri"] + df["jml_ra_ba_swasta"]
4 )
5
6 df.head(3)

```

Baris 1

Kolom baru `jml_paud` muncul di **ujung kanan** tabel.

Baris 3

Penjumlahan antar-kolom bekerja **serentak untuk seluruh 126 baris** tanpa kita menghitung satu-satu.

	nama_kec	nama_des	status_daerah	jml_tk_negeri	\
0	Teluk Betung Barat	Bakung	1	0	
1	Teluk Betung Barat	Kuripan	1	0	
2	Teluk Betung Barat	Negeri Olok Gading	1	0	

	jml_tk_swasta	jml_ra_ba_negeri	jml_ra_ba_swasta	kepadatan_pddk	jml_paud
0	2	0	0	6479	2
1	0	0	0	16321	0
2	1	0	0	7761	1

Posisi di ujung kanan itu kurang enak dibaca — kolom terpenting itu layak berdiri di dekat identitas kelurahan. Menyusun ulang posisi kolom caranya sederhana: sodorkan **list** berisi urutan kolom yang kita mau (pemakaian ulang konsep *list* barusan):

```
1 urutan_kolom = ["nama_kec", "nama_des", "status_daerah",
2                 "kepadatan_pddk", "jml_paud",
3                 "jml_tk_negeri", "jml_tk_swasta",
4                 "jml_ra_ba_negeri", "jml_ra_ba_swasta"]
5
6 df = df[urutan_kolom]
7
8 df.head(3)
```

Baris 4

Urutan kolom yang diinginkan ditampung dalam variabel *list*.

Baris 6

`df[urutan_kolom]` menyusun ulang kolom sesuai urutan di *list* tersebut.

Baris 8

`head(3)` menampilkan tiga baris pertama untuk memeriksa hasil penyusunan ulang.

	nama_kec	nama_des	status_daerah	kepadatan_pddk	\
0	Teluk Betung Barat	Bakung	1	6479	
1	Teluk Betung Barat	Kuripan	1	16321	
2	Teluk Betung Barat	Negeri Olok Gading	1	7761	

	jml_paud	jml_tk_negeri	jml_tk_swasta	jml_ra_ba_negeri	jml_ra_ba_swasta
0	2	0	2	0	0
1	0	0	0	0	0
2	1	0	1	0	0

Terakhir, setelah totalnya ada, empat kolom pecahannya tidak kita perlukan lagi hari ini. Membiarkan kolom tak terpakai membuat tabel lebar dan mata cepat lelah — seperti membiarkan data usang menumpuk di layar *command center* yang seharusnya bersih untuk pemantauan. Kita buang dengan `drop()`, lagi-lagi lewat variabel *list* agar rapi:

```

1 kolom_dibuang = ["jml_tk_negeri", "jml_tk_swasta", "jml_ra_ba_negeri",
2 "jml_ra_ba_swasta"]
3 df = df.drop(columns=kolom_dibuang)
4 df.head(3)

```

Baris 3

`drop(columns=kolom_dibuang)` menghapus seluruh kolom yang disebutkan dalam variabel `kolom_dibuang` dari tabel; hasilnya ditugaskan kembali ke variabel `df` supaya perubahan tersimpan permanen.

2.3.g Menyaring Baris: Filtrasi Data Kelurahan Berdasarkan Kriteria Tertentu

Tabel sudah rapi; kini saatnya mengajukan pertanyaan sang pemberi kasus: **kelurahan mana yang paling membutuhkan bantuan fasilitas PAUD?** Tafsiran pertama yang paling sederhana: kelurahan yang fasilitas PAUD-nya sangat sedikit — katakanlah satu atau tidak ada sama sekali.

Menyaring baris di Pandas polanya `df[kondisi]`: “dari tabel `df`, ambil baris-baris yang memenuhi kondisi”. Kondisinya kita tulis sebagai perbandingan atas sebuah kolom:

```

1 kelurahan_minim_paud = df[df["jml_paud"] <= 1]
2
3 print(f"Jumlah kelurahan dengan PAUD <= 1: {len(kelurahan_minim_paud)}")
4
5 kelurahan_minim_paud.head()

```

Baris 1

Penyaringan `df[df["jml_paud"] <= 1]` menahan hanya baris yang kolom `jml_paud`-nya bernilai 0 atau 1.

Baris 3

`len()` menghitung berapa baris yang lolos saringan.

Baris 5

`head()` menampilkan lima baris pertama dari hasil saringan.

```
Jumlah kelurahan dengan PAUD <= 1: 37
```

	nama_kec	nama_des	status_daerah	kepadatan_pddk	\
1	Teluk Betung Barat	Kuripan	1	16321	
2	Teluk Betung Barat	Negeri Olok Gading	1	7761	
4	Teluk Betung Barat	Batu Putuk	1	2214	
8	Telukbetung Timur	Perwata	1	21630	
9	Telukbetung Timur	Way Tataan	1	687	

	jml_paud
1	0
2	1

4	1
8	0
9	0

Konsep Python

Operator Perbandingan

`df["jml_paud"] <= 1` memakai **operator perbandingan** — tanda yang membandingkan dua nilai dan menjawab benar (`True`) atau salah (`False`):

Operator	Arti	Contoh
<code>==</code>	sama dengan	<code>status == 1</code>
<code>!=</code>	tidak sama dengan	<code>nama != "Bakung"</code>
<code>></code>	lebih besar	<code>kepadatan > 10000</code>
<code><</code>	lebih kecil	<code>luas < 5</code>
<code>>=</code>	lebih besar atau sama	<code>jml_paud >= 2</code>
<code><=</code>	lebih kecil atau sama	<code>jml_paud <= 1</code>

Perhatikan: **sama dengan ditulis dua kali** (`==`), karena tanda `=` tunggal sudah dipakai untuk penugasan variabel (Modul 1). Menukar keduanya adalah salah ketik yang sangat umum.

Saat dipakai pada kolom *DataFrame*, perbandingan diuji **serentak ke seluruh baris**, menghasilkan deretan `True/False` sepanjang tabel — dan `df[...]` menahan hanya baris-baris yang `True`. Itulah mekanika di balik penyaringan barisan.

```
1 kriteria_padat = df["kepadatan_pddk"] > 10000
2
3 kriteria_minim = df["jml_paud"] <= 1
4
5 kelurahan_prioritas = df[kriteria_padat & kriteria_minim]
6
7 print(f"Jumlah kelurahan prioritas: {len(kelurahan_prioritas)}")
8
9 kelurahan_prioritas
```

Baris 1

Variabel `kriteria_padat` menampung kondisi kepadatan di atas 10.000.

Baris 3

Variabel `kriteria_minim` menampung kondisi PAUD paling banyak 1.

Baris 5

Operator `&` menggabungkan kedua kondisi: hanya baris yang memenuhi keduanya yang lolos.

Baris 7

`len()` menghitung cacah kelurahan prioritas.

Baris 9

Menampilkan seluruh tabel kelurahan prioritas.

Jumlah kelurahan prioritas: 18

	nama_kec	nama_des	status_daerah	kepadatan_pddk	\
1	Teluk Betung Barat	Kuripan	1	16321	
8	Telukbetung Timur	Perwata	1	21630	
10	Telukbetung Timur	Kota Karang Raya	1	23796	
11	Teluk Betung Selatan	Gedong Pakuon	1	13067	
12	Teluk Betung Selatan	Talang	1	19524	
14	Teluk Betung Selatan	Teluk Betung	1	25011	
31	Tanjung Karang Timur	Tanjung Agung	1	37717	
33	Tanjung Karang Timur	Sawah Lama	1	35775	
34	Tanjung Karang Timur	Sawah Brebes	1	26055	
51	Tanjung Karang Pusat	Kaliawi	1	25700	
54	Tanjung Karang Pusat	Kaliawi Persada	1	13159	
55	Enggal	Gunung Sari	1	13476	
56	Enggal	Tanjung Karang	1	13571	
86	Kedaton	Sukamenanti Baru	1	20233	
87	Kedaton	Penengahan	1	21014	
88	Kedaton	Penengahan Raya	1	11023	
120	Way Halim	Jagabaya I	1	19406	
123	Way Halim	Gunung Sulah	1	14823	

	jml_paud
1	0
8	0
10	0
11	0
12	0
14	0
31	1
33	1
34	1
51	1
54	1
55	0
56	0
86	1
87	1
88	0
120	0
123	1

Operator Boolean & dan |

Untuk menggabungkan dua kondisi, Python menyediakan **operator boolean** (dinamai dari aljabar *Boolean*, aljabar benar-salah):

- & berarti **dan** — hasilnya `True` hanya jika **kedua** kondisi benar.
- | berarti **atau** — hasilnya `True` jika **salah satu saja** kondisi benar.

Dalam Python murni kata kuncinya `and` dan `or`, tetapi **khusus untuk kolom Pandas kita wajib memakai lambang & dan |**, karena pengujian dilakukan serentak ke ratusan baris sekaligus. Satu aturan lagi: jika kondisinya ditulis langsung tanpa variabel penampung, tiap kondisi **harus dikurung**, misalnya `df[(df["kepadatan_pddk"] > 10000) & (df["jml_paud"] <= 1)]` — tanpa kurung, Python salah membaca urutannya. Menampung kondisi ke variabel seperti yang kita lakukan membuat kurung tak diperlukan dan kodenya jauh lebih terbaca.

2.4 Latihan Mandiri

Kerjakan di notebook baru `modul/latihan-02.ipynb` dengan *kernel* `analitika-perkotaan`. Alurnya sama persis dengan instruksi di atas; yang berganti hanya kolom yang dipakai — kali ini **pendidikan dasar** (SD dan SMP), ditambah satu kolom kategorikal.

1. **Jembatan sel.** Di dua sel terpisah: sel pertama mendefinisikan dua variabel angka apa saja versi Anda sendiri, sel kedua menghitung sesuatu dari keduanya. Buktikan keduanya berbagi papan tulis, lalu jalankan **Restart kernel & Run All** dan pastikan semuanya tetap berjalan berurutan.
2. **Membaca data.** Baca `bdl_kelurahan.csv` memakai variabel `path_data` dan `kolom_pilihan` yang berisi: `nama_kec`, `nama_des`, `status_daerah`, `kepadatan_pddk`, `lap_usaha_sebagian_warga`, `jml_sd_negeri`, `jml_sd_swasta`, `jml_smp_negeri`, `jml_smp_swasta`.
3. **Anatomi tabel.** Periksa dengan `info()` dan `describe()`. Sebutkan (di sel teks): berapa barisnya, kolom mana saja yang bertipe angka, dan adakah kolom yang datanya kosong.
4. **Unit amatan vs unit analisis.** Hitung banyaknya nilai unik kelurahan dan kecamatan, lalu buat rekapitulasi frekuensi **data kategorikal** `lap_usaha_sebagian_warga` (lapangan usaha sebagian besar warga) dengan `value_counts()`. Nilainya berupa kode kategori BPS — arti tiap kode dapat dilihat di kamus data PODES 2021 yang dibagikan dosen. Tuliskan satu kalimat: lapangan usaha kode apa yang paling banyak menjadi tumpuan warga kelurahan-kelurahan Bandar Lampung, dan pada unit apa (amatan atau analisis) kesimpulan kalimat Anda itu berdiri?
5. **Merapikan tabel.** Buat kolom baru `jml_dikdas` (jumlah fasilitas pendidikan dasar) dari penjumlahan empat kolom SD/SMP, susun ulang kolom sehingga `jml_dikdas` berdiri setelah `kepadatan_pddk`, lalu hapus empat kolom pecahannya memakai variabel `list`.
6. **Menyaring baris.** Temukan **kelurahan prioritas fasilitas pendidikan dasar**: kepadatan penduduk di atas **15.000 jiwa/km²** **dan** `jml_dikdas` paling banyak **2**. Berapa kelurahan yang lolos? Tutup notebook Anda dengan satu paragraf sel teks yang menafsirkan daftar tersebut sebagaimana laporan singkat kepada pimpinan.

3 Pengolahan Data Terstruktur dengan SQL (SQLite)

! Sub-CPMK Pertemuan Ini

Mahasiswa mampu mengoperasikan basis data relasional SQLite menggunakan kueri SQL dasar, menerapkan operasi `JOIN` untuk menghubungkan tabel relasional kelurahan-kecamatan, dan mengintegrasikan kueri SQL ke dalam *DataFrame* Pandas.

3.1 Pendahuluan

<masukkan narasi yang menceritakan 'big why' mengelola basis data perkotaan yang relasional di sini>

3.2 Gambaran Umum Kasus

Di command center kota, layar pemantauan tidak boleh menampilkan data yang saling bertentangan. Bila kepadatan penduduk suatu kelurahan tersimpan berulang-ulang pada banyak baris data dengan nama kecamatan yang dieja berbeda-beda pada tiap salinannya, operator berisiko mengambil keputusan dari angka yang sudah tidak konsisten. Risiko itu makin nyata di Indonesia, tempat pemekaran wilayah adalah isu yang sering diangkat (Fokus: Geliat Wacana Pemekaran Daerah — CNN Indonesia): begitu satu kecamatan mekar, seluruh salinan namanya yang bertebaran di berkas tunggal harus dimutakhirkan serentak, dan celah pembaruan itulah yang membuat angka di layar operator meleset dari kondisi lapangan.

Sebagai operator command center, kita membutuhkan cara penyimpanan yang lebih tertib: **basis data relasional**. Menggunakan data profil kelurahan Kota Bandar Lampung dari PODES 2021 (BPS), kita akan memindahkan data kelurahan ke dalam basis data SQLite yang telah memuat tabel kecamatan, mengajukan pertanyaan kependudukan dan fasilitas pendidikan lewat kueri SQL — dari penyaringan sederhana hingga penggabungan kelurahan dengan kecamatan induknya — lalu menarik hasilnya kembali ke Pandas untuk dipantau lebih lanjut di layar kerja kita.

3.3 Instruksi

1. Buat *notebook* baru `modul/modul-03.ipynb` di direktori proyek Anda.
2. Sambungkan *notebook* ke *kernel* `analitika-perkotaan`.
3. Kerjakan seluruh instruksi di bawah ini secara berurutan.

3.3.a Lemari Arsip Digital: Pengenalan SQLite dan Koneksi Database

Sebelum memulai praktik, pahami dulu dua konsep dasarnya: basis data relasional dan SQLite. Konsep SQL sendiri baru dijelaskan begitu Anda benar-benar mengirim kueri pertama ke basis data.

3.3.a.a Apa itu Basis Data Relasional?

Basis data relasional (*relational database*) adalah sistem penyimpanan data digital yang mengorganisasi data ke dalam kumpulan tabel dua dimensi (baris dan kolom) yang saling berhubungan — saling **berelasi** — satu sama lain. Model ini pertama kali dirancang oleh Edgar F. Codd pada tahun 1970, dengan tiga karakteristik utama:

- **Struktur tabular** — setiap tabel mewakili satu entitas tertentu; misalnya satu tabel khusus data kelurahan dan satu tabel khusus data kecamatan.
- **Kunci relasi** (*key*) — hubungan antar-tabel diikat oleh kolom pengenal unik yang sama; tabel kelurahan terhubung ke tabel kecamatan melalui kolom kunci wilayah `kode_kec`.
- **Efisiensi dan integritas** — dengan memisahkan entitas ke tabel-tabel yang berelasi, kita menghindari redundansi (penulisan data berulang) dan menjaga data tetap konsisten ketika terjadi pemutakhiran wilayah.

Dalam analisis perkotaan, perencana sering bekerja dengan banyak tabel yang saling berhubungan — data kependudukan, fasilitas, hingga ekonomi lintas proyek dan lintas instansi. Dibandingkan menumpuk semuanya di berkas-berkas CSV terpisah, basis data relasional menawarkan penyimpanan yang efisien, kueri lintas tabel, dan penghindaran redundansi.

3.3.a.b Mengenal SQLite

SQLite adalah basis data relasional yang unik karena bersifat **serverless**. Berdasarkan dokumentasi resmi SQLite.org, SQLite tidak memerlukan proses server latar belakang (*daemon*) ataupun koneksi jaringan: mesin SQLite adalah *library* yang berjalan langsung di dalam proses aplikasi kita (*in-process*). Seluruh isi basis data — berapa pun banyak tabelnya — disimpan dalam **satu file biner tunggal** di *disk* lokal, sehingga bersifat *zero-configuration*, portabel, dan mudah disalin antarfolder.

Di sinilah jawaban teka-teki penting modul ini: **satu file `.sqlite` adalah satu basis data utuh, bukan satu tabel**. Berbeda dengan CSV yang satu file-nya hanya memuat satu tabel, satu file `.sqlite` adalah satu lemari arsip lengkap yang lacinya (tabelnya) bisa lebih dari satu.

i Pengetahuan Komputer

Basis Data Klien-Server vs Serverless

Basis data kelas industri seperti **PostgreSQL** bekerja dengan arsitektur **klien-server**: data dikelola oleh satu program server yang berjalan terus-menerus, dan analis mengaksesnya lewat jaringan memakai akun serta izin akses. Arsitektur ini unggul untuk data besar yang diakses banyak orang serentak — tetapi merepotkan untuk belajar karena menuntut instalasi dan konfigurasi server.

SQLite memangkas semua itu: tanpa server, tanpa akun, cukup satu file. Karena kesederhanaannya, SQLite menjadi basis data yang paling banyak terpasang di dunia — tertanam di ponsel, peramban, hingga berbagai aplikasi sehari-hari.

Satu klasifikasi lagi: SQLite dan PostgreSQL sama-sama berorientasi baris (*row-store*), menyimpan data per baris di *disk* — efisien untuk pencatatan dan pemutakhiran. Kontrasnya adalah basis data analitik berorientasi kolom (*column-store*) seperti BigQuery dan Snowflake, yang menyimpan data per kolom sehingga sangat cepat merekap dataset berskala raksasa.

Praktik kita mulai.

```

1  import sqlite3
2  import pandas as pd
3
4  path_db = "../dataset-ganjil-2026-2027/bd1.sqlite"
5  koneksi = sqlite3.connect(path_db)

```

Baris 1

Muat *library* `sqlite3` — pustaka standar bawaan Python, tanpa instalasi tambahan

Baris 2

Muat juga `pandas` untuk mengelola basis data tadi sebagai data terstruktur

Baris 4

Jalur ke file `bd1.sqlite` sebagai lemari arsip kita simpan ke variabel `path_db`.

Baris 5

Fungsi `sqlite3.connect(...)` membuka jalur komunikasi ke lemari arsip kita (variabel `path_db`) dan menampungnya di variabel `koneksi`. Semua perintah SQL kita selanjutnya akan lewat jalur ini.

⚠ Penting!

Path Salah Tidak Menimbulkan Galat

Jika *path* yang diberikan ke `sqlite3.connect(...)` keliru, Python **tidak akan protes** — ia justru membuat file basis data baru yang kosong di lokasi keliru tersebut. Maka bila inspeksi pada langkah berikutnya tidak menemukan tabel apa pun, jangan lanjut: periksa kembali variabel `path_db` Anda, hapus file kosong yang telanjur terbuat, lalu ulangi.

3.3.a.c Inspeksi Isi Basis Data

Karena satu file dapat berisi banyak tabel, kebiasaan pertama saat menerima file SQLite adalah **membuka lemarinya dan membaca daftar lacinya**. SQLite menyediakan tabel sistem bernama `sqlite_master` (nama barunya: `sqlite_schema`) yang mencatat metadata seluruh objek di dalam basis data. Tiga kolom pentingnya: `type` (jenis objek: `table`, `index`, `view`, `trigger`), `name` (nama objek), dan `sql` (pernyataan `CREATE TABLE` asli pembuat objek itu).

Jalankan kueri pertama Anda untuk mendaftar semua tabel:

```

1  kueri_daftar_tabel = "SELECT name FROM sqlite_master WHERE type='table'"
2
3  daftar_tabel = (
4      koneksi
5      .execute(kueri_daftar_tabel)
6      .fetchall()
7  )
8  print(daftar_tabel)

```

Baris 5

`koneksi.execute(...)` mengirim kueri ke basis data.

Baris 6

`.fetchall()` mengambil seluruh baris hasilnya sebagai sebuah *list* (Modul 2).

```
[('kecamatan',)]
```

Setiap baris hasil tampil sebagai satu kelompok nilai di dalam tanda kurung. Hasilnya satu baris saja: lemari arsip kita baru berisi satu laci, tabel `kecamatan`. `.execute(...)` dan `.fetchall()` di sini adalah **method** — bukan fungsi berdiri sendiri seperti `print()`, melainkan kemampuan yang menempel pada *koneksi*. Menuliskan dua method berurutan seperti ini disebut **method chaining**, persis seperti pola `df.isna().sum()` di Modul 2.

i Pengetahuan Komputer

Apa itu SQL?

SQL (*Structured Query Language*) adalah bahasa standar untuk berkomunikasi dengan basis data relasional. Jika basis data adalah **lemari arsip digital** studio PWK, maka SQL adalah bahasa perintah tertib yang kita gunakan untuk meminta petugas arsip melakukan tindakan tertentu — mengambil berkas kelurahan tertentu, mencatat berkas baru, atau merekap fasilitas umum per kecamatan. Kueri `SELECT name FROM sqlite_master WHERE type='table'` yang baru saja Anda jalankan adalah salah satu bentuknya: permintaan tertulis kepada petugas arsip untuk membacakan daftar laci yang tersedia.

Lalu di mana posisinya terhadap Pandas yang baru kita pelajari? Tabel berikut membandingkan keduanya *head-to-head* (diacu dari *SQL for Data Analysis*, Tanimura, 2021):

Dimensi Perbandingan	SQL	Python (Pandas)
Lokasi & beban komputer	Pengolahan dilakukan langsung oleh sistem basis data di pusat penyimpanan; komputer kita tidak terbebani saat datanya sangat besar.	Pengolahan dilakukan sepenuhnya di memori komputer kita sendiri; komputer bekerja lebih keras jika datanya sangat besar.
Kekakuan struktur data	Data harus ditata rapi sejak awal dalam tabel-tabel yang saling terhubung — seperti sekat permanen di laci arsip.	Bentuk data lebih fleksibel dan bebas diubah di tengah jalan, tetapi perintah kodenya lebih panjang dan detail.
Mekanisme perulangan data	Otomatis memproses seluruh baris sekaligus; tidak perlu perintah membaca baris demi baris.	Kadang perlu perintah khusus agar komputer membaca data baris demi baris secara berurutan.
Kelengkapan analisis	Sangat cepat untuk menyaring, menyatukan, dan merekap tabel, tetapi tidak bisa dipakai membangun model simulasi kota yang rumit.	Ekosistem alat bantunya melimpah: grafik, visualisasi peta, hingga model simulasi masa depan kota.

Keduanya bukan pesaing, melainkan pasangan kerja: SQL menyiapkan dan merekap data di lemari arsip, Pandas mengolah hasil rekapnya lebih lanjut. Persis alur itulah yang kita jalani pada modul ini.

Untuk membaca struktur kolom sebuah tabel, gunakan perintah sistem `PRAGMA table_info(nama_tabel)`. Ia mengembalikan rincian setiap kolom: nomor urut, nama kolom, tipe data, wajib-tidaknya terisi (`notnull`), nilai bawaan, dan status kunci primer (`pk`). Bentuk kueri standarnya juga tersedia: `SELECT * FROM pragma_table_info('nama_tabel')`.

```
1 struktur_kecamatan = koneksi.execute("PRAGMA table_info(kecamatan)").fetchall()
2 print(struktur_kecamatan)
```

Baris 1

`PRAGMA table_info(...)` mengembalikan rincian kolom tabel: nomor urut, nama, tipe data, dan status kunci primer.

Baris 2

`print()` menampilkan hasilnya.

```
[(0, 'kode_kec', 'INTEGER', 0, None, 0), (1, 'nama_kec', 'TEXT', 0, None, 0)]
```

Tabel `kecamatan` berisi dua kolom: `kode_kec` bertipe `INTEGER` (bilangan bulat) dan `nama_kec` bertipe `TEXT` (teks). Sekarang intip isinya — perintah `SELECT` akan kita bedah tuntas pada bagian Kueri Dasar; untuk sekarang cukup baca `SELECT * FROM kecamatan LIMIT 5` sebagai “ambilkan semua kolom dari tabel kecamatan, lima baris pertama saja”:

```
1 print(koneksi.execute("SELECT * FROM kecamatan LIMIT 5").fetchall())
```

Baris 1

Kueri `SELECT * FROM kecamatan LIMIT 5` mengambil semua kolom dari tabel kecamatan, dibatasi lima baris pertama; hasilnya langsung dicetak.

```
[(10, 'Teluk Betung Barat'), (11, 'Telukbetung Timur'), (20, 'Teluk Betung Selatan'),
(21, 'Bumi Waras'), (30, 'Panjang')]
```

Kecamatan-kecamatan Kota Bandar Lampung tercatat rapi di laci ini, masing-masing dengan kode wilayah resmi BPS-nya.

Pengetahuan VS Code

Inspeksi Visual dengan SQLite Explorer

File `.sqlite` adalah file biner — jika dibuka langsung di editor, isinya tampak seperti karakter acak. Agar dapat menjelajahinya secara visual, pasang ekstensi **SQLite** (penerbit: alexcvzz) dari panel Extensions VS Code, lalu ikuti langkah berikut:

1. Buka **Command Palette** (`Ctrl+Shift+P`)
2. Jalankan perintah **SQLite: Open Database**
3. Pilih file `bd1.sqlite`
4. Bentangkan panel **SQLITE EXPLORER** yang muncul di sisi kiri bawah
5. Klik ikon panah di samping nama tabel untuk menampilkan isinya (*Show Table*)

Ini adalah padanan visual dari kueri `sqlite_master` dan `PRAGMA table_info` yang barusan Anda jalankan — dua cara membaca lemari arsip yang sama.

3.3.b Menyusun Laci Data: Membuat Tabel dan Mengisi Data dari Pandas

Laci kelurahan belum ada — kita yang akan menatanya sendiri dari berkas `bdL_kelurahan.csv` yang sudah kita kenal di Modul 2. Baca kembali berkas itu memakai pola variabel `path_data` dan `kolom_pilihan`; kali ini kolom yang dibutuhkan adalah kunci wilayah, identitas kelurahan, kepadatan penduduk, dan fasilitas TK:

```
1 path_data = "../dataset-ganjil-2026-2027/bdL_kelurahan.csv"
2 kolom_pilihan = ["kode_kec", "nama_kec", "nama_des",
3                 "kepadatan_pddk", "jml_tk_negeri", "jml_tk_swasta"]
4
5 df_kelurahan = pd.read_csv(path_data, usecols=kolom_pilihan)
6 df_kelurahan.head()
```

Baris 1,3

Variabel `path_data` dan `kolom_pilihan` menyimpan alamat berkas dan daftar kolom yang dibutuhkan.

Baris 5

`pd.read_csv()` membaca CSV dengan hanya kolom yang dipilih.

Baris 6

`head()` menampilkan lima baris pertama untuk pemeriksaan.

	kode_kec	nama_kec	nama_des	jml_tk_negeri	\
0	10	Teluk Betung Barat	Bakung	0	
1	10	Teluk Betung Barat	Kuripan	0	
2	10	Teluk Betung Barat	Negeri Olok Gading	0	
3	10	Teluk Betung Barat	Sukarame Ii	0	
4	10	Teluk Betung Barat	Batu Putuk	0	

	jml_tk_swasta	kepadatan_pddk
0	2	6479
1	0	16321
2	1	7761
3	3	1514
4	1	2214

Perhatikan kolom `nama_kec`: nama kecamatan yang sama tertulis berulang pada setiap baris kelurahannya. Buktikan skala redundansinya dengan dua alat Modul 2:

```
1 print(len(df_kelurahan))
2 print(df_kelurahan["nama_kec"].nunique())
```

Baris 1

`len()` menghitung total baris.

Baris 2

`nunique()` menghitung banyaknya nama kecamatan yang berbeda.

126

20

Dari 126 baris, hanya ada 20 nama kecamatan unik — artinya setiap nama kecamatan ditulis berulang rata-rata enam kali. Padahal daftar resmi kecamatan sudah terstruktur rapi di tabel `kecamatan`. Menyimpan `nama_kec` sekali lagi di tabel kelurahan berarti memboroskan ruang dan membuka risiko ketidakkonsistenan bila ada pemutakhiran nama.

Maka lakukan **normalisasi** sederhana: buang kolom `nama_kec` (cukup kunci `kode_kec` yang tinggal, sebagai pengikat relasi), lalu tulis *DataFrame*-nya menjadi tabel baru bernama `kelurahan` menggunakan `to_sql`:

```
1 df_kelurahan = df_kelurahan.drop(columns=["nama_kec"])
2 df_kelurahan.to_sql(
3     "kelurahan",
4     koneksi,
5     if_exists="replace",
6     index=False,
7 )
```

Baris 3

Pandas membuatkan tabel `kelurahan` di dalam basis data dan menyalin seluruh barisnya — angka yang tampil di bawah sel adalah jumlah baris yang tersalin.

Baris 5

Argumen `if_exists="replace"` menentukan sikap ketika tabel bernama sama sudah ada: hapus dan tulis ulang.

Baris 6

Argumen `index=False` mencegah nomor indeks *DataFrame* ikut tersimpan sebagai kolom tersendiri.

126

Penting!

Galat `table kelurahan already exists`

Tanpa argumen `if_exists="replace"`, perilaku bawaan `to_sql` adalah argumen `if_exists="fail"`: eksekusi kedua atas sel yang sama akan gagal dengan galat `ValueError: Table 'kelurahan' already exists`, karena lacinya memang sudah terisi. Ingat ritual **Restart kernel & Run All** dari Modul 2 — tanpa `replace`, notebook Anda tidak akan lolos ritual itu. Karena tabel ini milik kita dan sumber kebenarannya adalah berkas CSV, menghapus lalu menulis ulang adalah sikap yang tepat di sini.

3.3.b.a Verifikasi Hasil Pengisian Data

Buktikan bahwa lemari arsip kini berisi dua laci, lalu periksa struktur laci baru itu. Variabel `kueri_daftar_tabel` masih tersimpan di papan tulis *kernel* — pakai ulang saja:

```
1 print(koneksi.execute(kueri_daftar_tabel).fetchall())
2 print(koneksi.execute("PRAGMA table_info(kelurahan)").fetchall())
```

Baris 1

Variabel `kueri_daftar_tabel` dipakai ulang untuk membuktikan bahwa lemari arsip kini berisi dua laci.

Baris 2

`PRAGMA table_info(kelurahan)` memeriksa struktur kolom tabel baru.

```
[('kecamatan',), ('kelurahan',)]
[(0, 'kode_kec', 'INTEGER', 0, None, 0), (1, 'nama_des', 'TEXT', 0, None, 0), (2, 'jml_tk_negeri', 'INTEGER', 0, None, 0), (3, 'jml_tk_swasta', 'INTEGER', 0, None, 0), (4, 'kepadatan_pddk', 'INTEGER', 0, None, 0)]
```

Dua tabel dalam satu file — persis seperti janji konsep di awal. Perhatikan pula tipe datanya: `to_sql` menerjemahkan tipe Pandas secara otomatis menjadi `INTEGER` untuk kolom bilangan bulat, `REAL` untuk bilangan desimal, dan `TEXT` untuk teks. Silakan segarkan panel SQLite Explorer Anda (ikon *refresh*) — tabel `kelurahan` kini juga tampil di sana.

⚠ Penting!

Basis Data Ini Dipakai Lagi di Modul-Modul Berikutnya

Lemari arsip `bd1.sqlite` yang baru Anda tata bukan sekadar latihan sekali pakai: modul-modul berikutnya akan **membuka kembali basis data ini** sebagai sumber data praktik. Jangan menghapus atau memindahkan file tersebut dari direktori dataset, dan pastikan tabel `kelurahan` benar-benar terbentuk (lolos verifikasi di atas) sebelum melanjutkan. Jika di kemudian hari file ini rusak atau terhapus, tabel `kelurahan` selalu dapat ditata ulang dari berkas CSV sumbernya dengan mengulang bagian ini.

3.3.c Kueri Dasar: Mengambil Data dengan `SELECT` dan Menyaring dengan `WHERE`

Lemari sudah tertata; saatnya belajar meminta petugas arsip bekerja. Kueri pengambilan data tersusun dari klausa-klausa: `SELECT` (kolom apa yang diambil), `FROM` (dari tabel mana), `WHERE` (baris mana yang lolos saringan), `ORDER BY` (diurutkan menurut apa; tambahkan `DESC` untuk urutan menurun), dan `LIMIT` (berapa baris maksimal yang dikembalikan). Klausa `WHERE` adalah padanan penyaringan baris Pandas pada Modul 2 — operator perbandingannya (`>`, `<=`, dan kawan-kawannya) bekerja serupa.

Dua bekal tambahan dari Tanimura (2021, Bab 2) untuk menghadapi data lapangan:

1. **Penanganan nilai kosong** — data yang hilang direpresentasikan sebagai `NULL`, dan menyaringnya memakai operator khusus `IS NULL` atau `IS NOT NULL` (bukan `= NULL`). Kolom-kolom PODES yang

kita pakai kebetulan terisi lengkap, tetapi saringan ini adalah sabuk pengaman standar ketika bekerja dengan data instansi yang kelengkapannya belum tentu terjamin.

2. **Konversi tipe data** (*type casting*) — SQLite bertipe dinamis; untuk perhitungan yang konsisten kita bisa memaksa tipe tertentu secara eksplisit dengan `CAST(kolom AS tipe)`. Basis data lain seperti PostgreSQL menyediakan jalan pintas berupa operator `::`.

Susun kueri untuk menemukan kelurahan terpadat: kepadatan di atas 10.000 jiwa/km², urut dari yang terpadat, sepuluh teratas. Kueri sepanjang ini tidak nyaman ditulis sebagai satu baris — tampung dalam string multi-baris:

```
1 kueri_dasar = """
2 SELECT
3     nama_des,
4     kode_kec,
5     CAST(kepadatan_pddk AS REAL) AS kepadatan
6 FROM
7     kelurahan
8 WHERE
9     kepadatan_pddk IS NOT NULL
10    AND kepadatan_pddk > 10000
11 ORDER BY
12     kepadatan DESC
13 LIMIT 10
14 """
15
16 hasil_dasar = koneksi.execute(kueri_dasar).fetchall()
17 print(hasil_dasar)
```

Baris 14

Kueri ditampung di variabel *string* multi-baris: `SELECT` memilih kolom, `WHERE` menyaring, `ORDER BY DESC` mengurutkan menurun, `LIMIT 10` membatasi.

Baris 16

`execute()` mengirim kueri ke basis data, `fetchall()` mengambil seluruh baris hasilnya.

Baris 17

`print()` mencetak hasilnya.

```
[('Kota Karang', 11, 43907.0), ('Tanjung Agung', 40, 37717.0), ('Sawah Lama', 40, 35775.0), ('Sukajawa', 70, 31830.0), ('Sukamenanti', 80, 28790.0), ('Kebon Jeruk', 40, 26592.0), ('Sawah Brebes', 40, 26055.0), ('Kaliawi', 60, 25700.0), ('Teluk Betung', 20, 25011.0), ('Kota Karang Raya', 11, 23796.0)]
```

Sepuluh kelurahan terpadat Bandar Lampung kini di tangan Anda. Klausula `AS kepadatan` pada `SELECT` memberi nama panggilan (*alias*) untuk kolom hasil — nama itulah yang dikenali `ORDER BY`.

String Multi-baris (Triple-Quoted)

Di Modul 1, string selalu kita tulis satu baris dengan sepasang tanda kutip. Python menyediakan bentuk panjangnya: **string multi-baris**, diapit tiga tanda kutip (""" di awal dan di akhir). Segala yang berada di antaranya — ganti baris, indentasi, spasi — tersimpan apa adanya sebagai **satu nilai string biasa**.

Bayangkan peserta studio yang menulis instruksi kerja panjang di selembar kertas kalkir: baris baru dan indentasinya tetap ikut tersalin apa adanya selama masih di lembar yang sama. Begitu pula tanda kutip tiga itu — ia hanyalah pembatas awal dan akhir “lembar kalkir” kita, sedangkan Python membaca seluruh isinya sebagai satu potongan teks utuh.

Itulah sebabnya variabel `kueri_dasar` dapat kita tata bertingkat agar enak dibaca tanpa menimbulkan galat sintaksis: bagi Python ia tetap satu string utuh yang tinggal dikirim ke basis data. Pola ini akan sering kembali setiap kali kita menampung teks panjang di dalam kode.

i Pengetahuan Komputer

KAPITAL, Indentasi, dan Urutan Evaluasi

Bagi basis data, `select` dan `SELECT` sama saja. Penulisan kata kunci dengan huruf KAPITAL dan pemenggalan klausa dengan indentasi adalah **konvensi gaya** antarmanusia (Tanimura, 2021, Bab 8): mata cepat menangkap kerangka kueri, dan kueri panjang tetap terbaca.

Trivia keduanya lebih dalam: urutan penulisan klausa bukanlah urutan kerjanya. Secara internal, basis data mengevaluasi `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`. Jadi meski `SELECT` ditulis paling atas, ia justru dievaluasi belakangan — petugas arsip memilih laci dulu, menyaring berkasnya, baru memutuskan kolom mana yang disalin ke laporan.

```
1  kueri_gabung = """
2  SELECT DISTINCT
3      kel.nama_des,
4      kec.nama_kec,
5      kel.kepadatan_pddk
6  FROM
7      kelurahan AS kel
8  LEFT JOIN
9      kecamatan AS kec ON kel.kode_kec = kec.kode_kec
10 ORDER BY
11     kepadatan_pddk DESC
12 LIMIT 10
13 """
14
15 hasil_gabung = koneksi.execute(kueri_gabung).fetchall()
16 print(hasil_gabung)
```

Baris 13

Kueri `LEFT JOIN` menggabungkan tabel `kelurahan` dengan `kecamatan` lewat kunci `kode_kec`; alias `kel` dan `kec` mempersingkat penulisan.

Baris 15

`execute()` dan `fetchall()` menjalankan kueri dan mengambil hasilnya.

Baris 16

`print()` mencetak hasil gabungan.

```
[('Kota Karang', 'Telukbetung Timur', 43907), ('Tanjung Agung', 'Tanjung Karang Timur', 37717), ('Sawah Lama', 'Tanjung Karang Timur', 35775), ('Sukajawa', 'Tanjung Karang Barat', 31830), ('Sukamenanti', 'Kedaton', 28790), ('Kebon Jeruk', 'Tanjung Karang Timur', 26592), ('Sawah Brebes', 'Tanjung Karang Timur', 26055), ('Kaliawi', 'Tanjung Karang Pusat', 25700), ('Teluk Betung', 'Teluk Betung Selatan', 25011), ('Kota Karang Raya', 'Telukbetung Timur', 23796)]
```

Sekarang setiap kelurahan terpadat tampil bersama nama kecamatan induknya — dua laci menyatu dalam satu laporan.

i Pengetahuan Komputer

Kunci Primer dan Kunci Asing

Pada tabel `kecamatan`, kolom `kode_kec` berperan sebagai **kunci primer** (*primary key*): pengenalan unik yang nilainya pantang kembar — satu kode, satu kecamatan. Pada tabel `kelurahan`, kolom `kode_kec` yang sama berperan sebagai **kunci asing** (*foreign key*): penunjuk ke baris tabel lain yang boleh berulang — banyak kelurahan menunjuk satu kecamatan induk yang sama.

Mengapa mengikat relasi dengan kode, bukan dengan nama? Karena nama itu rapuh: ejaannya bisa berbeda antardokumen (misalnya “Teluk Betung” dan “Telukbetung”) dan namanya bisa berganti saat pemekaran wilayah, sedangkan kode wilayah resmi BPS dijaga konsisten. Inilah alasan kolom `nama_kec` aman kita buang dari tabel kelurahan: relasinya sudah dijamin oleh kunci.

3.3.d Agregasi Kota: Mengelompokkan Data dengan **GROUP BY**

Perencana jarang berhenti di daftar kelurahan; laporan akhirnya biasanya berupa rekap pada unit analisis kecamatan (Modul 2). Di SQL, rekap seperti itu dibuat dengan klausa **GROUP BY**, yang mengelompokkan baris-baris bernilai sama, lalu **fungsi agregasi** yang merangkum tiap kelompok menjadi satu angka: **AVG** (rata-rata), **SUM** (jumlah total), **COUNT** (cacah baris), **MIN**, dan **MAX**. `value_counts()` yang Anda pakai di Modul 2 sesungguhnya adalah **GROUP BY + COUNT** versi Pandas — dan inilah “rekapitulasi per kecamatan yang lebih kaya” yang dijanjikan modul lalu.

Aturan emasnya: **setiap kolom pada SELECT yang tidak dibungkus fungsi agregasi wajib dicantumkan di GROUP BY** — petugas arsip harus tahu persis menurut apa berkas dikelompokkan.

3.3.d.a Penanganan Nilai **NULL** dengan **COALESCE**

Satu jebakan aritmetika SQL: angka yang dijumlahkan dengan **NULL** hasilnya **NULL**. Bila salah satu dari `jml_tk_negeri + jml_tk_swasta` kosong, seluruh hasil penjumlahannya ikut kosong. Penangkalnya adalah fungsi **COALESCE(kolom, nilai_cadangan)** (Tanimura, 2021, Bab 2): ia memeriksa nilai dari kiri ke kanan dan mengembalikan yang pertama tidak-**NULL** — sehingga `COALESCE(jml_tk_negeri, 0)` berarti “pakai nilainya, atau 0 bila kosong”. Kolom TK kita memang terisi lengkap, tetapi membiasakan **COALESCE** pada penjumlahan lintas kolom adalah disiplin yang menyelamatkan ketika data instansi datang bolong-bolong.

Susun rekapnya: rata-rata jumlah TK (negeri + swasta) per kecamatan, beserta cacah kelurahannya, urut dari yang tertinggi:

```
1 kueri_rekap = ""
2 SELECT
3     kec.nama_kec,
4     COUNT(kel.nama_des) AS jumlah_kelurahan,
5     AVG(COALESCE(kel.jml_tk_negeri, 0) + COALESCE(kel.jml_tk_swasta, 0)) AS
rata_rata_tk
6 FROM
7     kecamatan AS kec
8 LEFT JOIN
9     kelurahan AS kel ON kec.kode_kec = kel.kode_kec
10 GROUP BY
11     kec.nama_kec
12 ORDER BY
13     rata_rata_tk DESC
14 ""
15
16 hasil_rekap = koneksi.execute(kueri_rekap).fetchall()
17 print(hasil_rekap)
```

Baris 14

Kueri `GROUP BY` mengelompokkan baris berdasarkan `kec.nama_kec`, lalu fungsi agregasi `COUNT` dan `AVG` merangkum tiap kelompok.

Baris 16

`execute()` dan `fetchall()` menjalankan kueri.

Baris 17

`print()` mencetak rekap 20 kecamatan.

```
[('Sukarame', 6, 5.833333333333333), ('Labuhan Ratu', 6, 4.166666666666667),
('Kedamaian', 7, 4.142857142857143), ('Rajabasa', 7, 3.857142857142857), ('Way
Halim', 6, 3.0), ('Sukabumi', 7, 2.857142857142857), ('Langkapura', 5, 2.6), ('Teluk
Betung Utara', 6, 2.5), ('Kemiling', 9, 2.4444444444444446), ('Tanjung Karang
Barat', 7, 2.4285714285714284), ('Kedaton', 7, 2.4285714285714284), ('Panjang', 8,
2.375), ('Teluk Betung Selatan', 6, 2.3333333333333335), ('Tanjung Karang Pusat', 7,
2.2857142857142856), ('Tanjung Senang', 5, 2.0), ('Tanjung Karang Timur', 5, 2.0),
('Telukbetung Timur', 6, 1.8333333333333333), ('Enggal', 6, 1.5), ('Teluk Betung
Barat', 5, 1.4), ('Bumi Waras', 5, 1.2)]
```

Perhatikan arah `JOIN`-nya kita balik — kecamatan di kiri — agar seluruh kecamatan pasti tampil di rekap, termasuk seandainya ada kecamatan yang tidak punya baris kelurahan.

Rekap 20 kecamatan itu benar isinya, tetapi bentuknya masih deretan baris polos hasil `fetchall()`: sulit dibaca, dan belum bisa diolah lanjut. Kita butuh jembatan kembali ke perkakas yang sudah kita kuasai.

3.3.e Jembatan SQL-Pandas: Membaca Hasil Kueri ke DataFrame Pandas

Jembatan itu bernama `pd.read_sql(kueri, koneksi)`: kirimkan kueri lewat koneksi, dan hasilnya langsung tersaji sebagai *DataFrame* — lengkap dengan nama kolom sesuai *alias* pada `SELECT`. Jalankan ulang variabel `kueri_rekap` yang masih tersimpan di papan tulis *kernel*:

```
1 df_rekap = pd.read_sql(kueri_rekap, koneksi)
2 df_rekap.head(10)
```

Baris 1

`pd.read_sql()` mengirim kueri lewat koneksi dan langsung mengembalikan hasilnya sebagai *DataFrame*.

Baris 2

`head(10)` menampilkan sepuluh baris pertama.

	nama_kec	jumlah_kelurahan	rata_rata_tk
0	Sukarame	6	5.833333
1	Labuhan Ratu	6	4.166667
2	Kedamaian	7	4.142857
3	Rajabasa	7	3.857143
4	Way Halim	6	3.000000
5	Sukabumi	7	2.857143
6	Langkapura	5	2.600000
7	Teluk Betung Utara	6	2.500000
8	Kemiling	9	2.444444
9	Tanjung Karang Barat	7	2.428571

Bandingkan dengan keluaran `fetchall()` di atas: isinya sama, wujudnya berbeda — kini berupa tabel rapi yang siap disaring, direkap ulang, atau digabung dengan data lain memakai seluruh kemampuan Pandas dari Modul 2. Terbacalah alur lengkap modul ini: `CSV` → `to_sql` → **basis data SQLite** → **kueri SQL** → `read_sql` → ***DataFrame***. Lemari arsip menertibkan penyimpanan; Pandas melanjutkan analisis.

3.3.e.a Menutup Sesi Kerja Basis Data

Tutup koneksi setiap kali selesai bekerja:

```
1 koneksi.close()
```

Baris 1

`close()` menutup koneksi ke basis data agar berkas tidak terkunci.

 **Penting!**

Tutup Koneksi Sebelum Pulang

Koneksi yang dibiarkan terbuka dapat mengunci file basis data — program lain (termasuk SQLite Explorer) bisa tertolak mengaksesnya, dan penulisan yang terputus di tengah berisiko merusak file. Ibaratnya laci lemari arsip: selesai bekerja, dorong lacinya sampai tertutup. Setelah

`koneksi.close()`, variabel `koneksi` tidak dapat dipakai lagi; jika masih ingin mengueri, buka koneksi baru dengan `sqlite3.connect(...)`.

3.4 Latihan Mandiri

Kerjakan di notebook baru `modul/latihan-03.ipynb` dengan *kernel* `analitika-perkotaan`. Alurnya sama persis dengan instruksi di atas; yang berganti hanya fasilitasnya — dari TK ke **SMP**.

1. **Buka dan inspeksi.** Buka koneksi ke `bd1.sqlite` memakai variabel `path`, lalu daftar seluruh tabelnya dengan kueri `sqlite_master`.
2. **Tata ulang laci kelurahan.** Baca `bd1_kelurahan.csv` memakai variabel `path_data` dan `kolom_pilihan` berisi: `kode_kec`, `nama_des`, `jml_smp_negeri`, `jml_smp_swasta`. Tulis ke tabel `kelurahan` dengan `to_sql` — pikirkan sendiri argumen `if_exists` yang tepat, mengingat tabel itu sudah ada.
3. **Saring dengan kueri dasar.** Dalam string multi-baris, susun kueri yang menampilkan kelurahan yang sama sekali belum memiliki SMP (jumlah SMP negeri ditambah swasta sama dengan nol) — amankan penjumlahannya dengan `COALESCE`.
4. **Rekap per kecamatan.** Susun kueri `LEFT JOIN + GROUP BY` untuk menghitung rata-rata jumlah SMP (negeri + swasta) per kecamatan,urut dari rata-rata **terendah** sehingga kecamatan yang paling minim fasilitas tampil teratas.
5. **Tarik ke Pandas.** Jalankan kueri rekap tersebut lewat `pd.read_sql`, tampung hasilnya di `DataFrame` `df_latihan`, lalu tampilkan.
6. **Interpretasi.** Di sel teks, tulis dua-tiga kalimat: kecamatan mana yang rata-rata SMP-nya paling rendah dan paling tinggi, dan apa maknanya bagi pemerataan fasilitas pendidikan dasar di Bandar Lampung.
7. **Tutup dan uji.** Tutup koneksi, lalu jalankan **Restart kernel & Run All** — seluruh sel harus lolos berurutan.

4 Exploratory Data Analysis (EDA)

! Sub-CPMK Pertemuan Ini

Mahasiswa mampu melakukan *Exploratory Data Analysis* (EDA) secara statistik deskriptif pada data perkotaan, mendeteksi anomali data (*outlier*), menangani nilai kosong (*missing values*), dan menganalisis korelasi kasar antarvariabel perkotaan menggunakan Python.

4.1 Pendahuluan

<masukkan narasi yang menceritakan 'big why' mengeksplorasi karakter dan kualitas data perkotaan sebelum menganalisisnya di sini>

4.2 Gambaran Umum Kasus

Bayangkan kursi Anda adalah kursi operator di *command center* Kota Bandar Lampung: layar-layar di depan Anda menampilkan kepadatan penduduk, ketersediaan fasilitas pendidikan, dan berbagai indikator lain per kelurahan, dan dari angka-angka itulah keputusan pengelolaan kota diambil. Seorang operator yang cermat tidak langsung mempercayai apa pun yang tampil di layar: sebelum satu angka pun dipakai memutuskan sesuatu, ia memeriksa dulu data di baliknya — adakah data yang kosong, adakah angka yang janggal jauh dari kewajaran, dan pola awal apa yang mulai terlihat. Keputusan yang dibangun di atas data yang belum diperiksa ibarat radar yang membaca sinyal yang keliru: keputusan yang diambil bisa salah tanpa satu pun peringatan yang berbunyi.

Kebiasaan memeriksa itulah yang dalam analitika perkotaan disebut **Exploratory Data Analysis (EDA)**: eksplorasi awal untuk mengenali karakter dan menguji kualitas data sebelum data itu dipakai merumuskan apa pun. Menggunakan profil kelurahan Kota Bandar Lampung dari PODES 2021 (BPS) yang tersimpan dalam basis data hasil kerja kita di Modul 3, kita akan mengenali karakter data numerik lewat statistik deskriptif, membaca frekuensi data kategorikal, memburu nilai-nilai ekstrem (*outlier*) dengan metode IQR, menemukan dan menangani data kosong (*missing values*), memeriksa hubungan kasar antarvariabel lewat korelasi, lalu menutupnya dengan sintesis: seperti apa sesungguhnya karakter sosio-demografi kota ini menurut datanya sendiri.

4.3 Instruksi

1. Buat *notebook* baru `modul/modul-04.ipynb` di direktori proyek Anda.
2. Sambungkan *notebook* ke *kernel* `analitika-perkotaan`.
3. Kerjakan seluruh instruksi di bawah ini secara berurutan.

```
1 import sqlite3
2 import pandas as pd
3
4 path_db = "../dataset-ganjil-2026-2027/bdl.sqlite"
5 koneksi = sqlite3.connect(path_db)
6
7 path_data = "../dataset-ganjil-2026-2027/bdl_kelurahan.csv"
```

```

8 df_csv = pd.read_csv(path_data)
9 df_csv = df_csv.drop(columns=["nama_kec"])
10 df_csv.to_sql("kelurahan", koneksi, if_exists="replace", index=False)

```

Baris 1-2

Mengimpor pustaka `sqlite3` dan `pandas`.

Baris 4-5

Menentukan jalur (*path*) berkas basis data dan membuka koneksi ke `bdl.sqlite`.

Baris 7-8

Menentukan lokasi berkas CSV dan membaca seluruh datanya ke dalam *DataFrame*.

Baris 9

Membuang kolom `nama_kec` dari *DataFrame* agar tidak duplikat dengan kolom dari tabel `kecamatan` saat digabungkan nanti.

Baris 10

Menyalin seluruh data *DataFrame* ke dalam tabel `kelurahan` di basis data SQLite (opsi `if_exists="replace"` menimpa tabel lama jika sudah ada, dan `index=False` mencegah penambahan kolom indeks bawaan).

126

Angka 126 menandakan seluruh baris kelurahan tersalin. Perhatikan bedanya dengan Modul 3: `pd.read_csv(path_data)` tanpa argumen `usecols` membaca **semua** kolom yang ada — itulah profil lengkap yang kita butuhkan.

⚠ Penting!

Jika Inspeksi Tidak Menemukan Tabel `kecamatan`

Langkah di atas menganggap `bdl.sqlite` Anda sehat: berisi tabel `kecamatan` bawaan kelas. Ingat perilaku `sqlite3.connect(...)` dari Modul 3 — *path* yang keliru tidak menimbulkan galat, melainkan diam-diam membuat basis data baru yang kosong. Bila kueri `JOIN` di bawah nanti menghasilkan kolom `nama_kec` yang kosong seluruhnya, berhenti: periksa variabel `path_db` Anda, hapus file kosong yang telanjur terbuat, dan bila perlu ulangi bagian awal Modul 3.

Sekarang tarik profil lengkap itu ke *DataFrame*, sekaligus menjemput nama kecamatan induk dari laci sebelah dengan `LEFT JOIN` — persis pola Modul 3, ditampung dalam string multi-baris:

```

1 kueri_profil = """
2 SELECT
3     kel.*,
4     kec.nama_kec
5 FROM
6     kelurahan AS kel
7 LEFT JOIN
8     kecamatan AS kec ON kel.kode_kec = kec.kode_kec
9 """

```

```

10
11 df = pd.read_sql(kueri_profil, koneksi)
12 print(df.shape)

```

Baris 3

`kel.*` pada `SELECT` berarti “seluruh kolom dari tabel `kelurahan`” — sehingga kita tidak perlu mengetik puluhan nama kolom satu per satu.

Baris 11

`pd.read_sql()` menjalankan kueri penggabungan (`JOIN`) melalui koneksi aktif dan langsung memuat hasilnya ke dalam `DataFrame`.

Baris 12

`df.shape` menampilkan dimensi data (jumlah baris dan kolom) untuk memastikan seluruh data kelurahan beserta atributnya telah termuat.

```
(126, 34)
```

Hasilnya 126 baris dan 34 kolom: seluruh profil kelurahan, plus `nama_kec` hasil `JOIN`.

Kini kenali karakternya. Metode `describe()` merangkum kolom-kolom numerik menjadi delapan angka: cacah data terisi (`count`), rata-rata (`mean`), simpangan baku (`std`), nilai terkecil (`min`), kuartil bawah (25%), median (50%), kuartil atas (75%), dan nilai terbesar (`max`). Tampung dulu kolom-kolom yang ingin diperiksa dalam variabel `list` (pola Modul 2) supaya daftar yang sama bisa dipakai ulang sepanjang modul:

```

1 kolom_numerik = ["kepadatan_pddk", "jml_tk_negeri", "jml_tk_swasta",
2                  "jml_ra_ba_swasta"]
3 df[kolom_numerik].describe()

```

Baris 1

Mendefinisikan daftar nama kolom yang berisi data bertipe numerik.

Baris 3

Menampilkan statistik deskriptif ringkas untuk kolom-kolom numerik tersebut menggunakan metode `.describe()`.

	kepadatan_pddk	jml_tk_negeri	jml_tk_swasta	jml_ra_ba_swasta
count	126.000000	126.000000	126.000000	126.000000
mean	9431.825397	0.103175	2.595238	0.269841
std	8428.116609	0.330560	2.075297	0.662271
min	627.000000	0.000000	0.000000	0.000000
25%	3466.750000	0.000000	1.000000	0.000000
50%	6450.500000	0.000000	2.000000	0.000000
75%	12693.500000	0.000000	4.000000	0.000000
max	43907.000000	2.000000	10.000000	4.000000

Bacalah tabel itu seperti perencanaan membaca rekap survei:

- **count = 126 di semua kolom** — tidak ada data kosong pada keempat kolom ini; kelengkapan kolom lain kita periksa belakangan.
- **kepadatan_pddk** — rata-ratanya sekitar 9.432 jiwa/km², tetapi simpangan bakunya hampir sama besar (± 8.428): sebaran antarkelurahan sangat lebar. Rentangnya membentang dari 627 sampai 43.907 jiwa/km², dan median (6.450) jauh di bawah rata-rata — tanda distribusinya **menceng**: segelintir kelurahan yang luar biasa padat menyeret rata-rata ke atas. Nilai-nilai ekstrem inilah yang akan kita buru pada bagian *outlier*.
- **jml_tk_negeri** — median bahkan kuartil atasnya 0: lebih dari tiga perempat kelurahan tidak memiliki satu pun TK negeri; layanan TK di kota ini nyaris sepenuhnya ditopang swasta.

Satu catatan penyaringan kolom: profil PODES juga memuat `jml_ra_ba_negeri`, tetapi seluruh 126 nilainya seragam 0 — kolom seragam tidak menyumbang informasi bagi statistik deskriptif, maka tidak kita ikutkan di variabel `kolom_numerik`. Temuan “kolom seragam” semacam ini tetap layak dicatat; ia akan kembali di bagian sintesis.

4.3.a Data Kategorikal: Melihat Frekuensi dan Modus dengan `value_counts()`

Tidak semua kolom angka bermakna besar. Pada profil PODES, banyak kolom berisi **kode kategori** — angkanya adalah label, bukan jumlah, sehingga merata-ratakannya tidak bermakna. Perlakuan yang tepat untuk data kategorikal adalah menghitung **frekuensi** tiap kategori dengan `value_counts()` (Modul 2), dan kategori terbanyaknya disebut **modus**. Periksa dua kolom: topografi wilayah (`topo`) dan status daerah saat pencacahan (`status_daerah`):

```
1 print(df["topo"].value_counts())
2 print(df["status_daerah"].value_counts())
```

Baris 1

Menghitung frekuensi kemunculan tiap nilai kategori pada kolom `topo` (topografi).

Baris 2

Menghitung frekuensi kemunculan tiap nilai kategori pada kolom `status_daerah` (status wilayah pesisir/non-pesisir).

```
topo
3    77
2    49
Name: count, dtype: int64
status_daerah
1    124
2     2
Name: count, dtype: int64
```

Arti tiap kode tercantum dalam kamus data PODES 2021 yang dibagikan dosen. Untuk `topo`: kode 3 adalah **dataran** dan kode 2 adalah **lembah/daerah aliran sungai** (kode 1, lereng/puncak, tidak muncul di data). Untuk `status_daerah`: kode 1 **perkotaan**, kode 2 **perdesaan**.

Terbaca dua hal. Pertama, modus topografi Bandar Lampung adalah dataran (77 kelurahan), tetapi porsi kelurahan lembah/DAS-nya besar juga (49 kelurahan) — hampir empat dari sepuluh. Kedua, yang lebih menarik: dua kelurahan di kota ini masih tercacah **perdesaan**. Temuan kecil semacam inilah yang sering

luput bila kita tidak menyempatkan EDA. (Pandas juga menyediakan metode `mode()` untuk mengambil modus secara langsung; pada keluaran `value_counts()` modus selalu baris teratas, karena hasilnya terurut dari frekuensi terbesar.)

4.3.b Menemukan Keanehan: Deteksi Outlier (Nilai Ekstrem) Menggunakan Metode IQR

Statistik deskriptif tadi sudah membisikkan adanya nilai-nilai ekstrem. Sekarang kita tangkap mereka secara tegas dengan **metode IQR** (*Interquartile Range*), resep klasik John Tukey yang hanya membutuhkan dua angka dari tabel `describe()` tadi: kuartil bawah (Q1) dan kuartil atas (Q3). Jarak keduanya, $IQR = Q3 - Q1$, adalah rentang “separuh data yang di tengah”. Sebuah nilai dicap *outlier* bila jatuh di luar **pagar**: lebih kecil dari $Q1 - 1,5 \times IQR$, atau lebih besar dari $Q3 + 1,5 \times IQR$.

Hitung pagarnya untuk kepadatan penduduk — metode `quantile()` menerima posisi kuartil dalam pecahan (0.25 untuk Q1, 0.75 untuk Q3):

```
1 q1 = df["kepadatan_pddk"].quantile(0.25)
2 q3 = df["kepadatan_pddk"].quantile(0.75)
3 iqr = q3 - q1
4 batas_bawah = q1 - 1.5 * iqr
5 batas_atas = q3 + 1.5 * iqr
6
7 print(f"Q1 = {q1}, Q3 = {q3}, IQR = {iqr}")
8 print(f"Pagar bawah = {batas_bawah}, pagar atas = {batas_atas}")
```

Baris 1-2

Menghitung kuartil pertama (Q1) dan kuartil ketiga (Q3) untuk kolom kepadatan penduduk.

Baris 3-5

Menghitung rentang interkuartil (IQR), lalu menentukan ambang pagar bawah dan pagar atas.

Baris 7-8

Menampilkan nilai pagar bawah dan pagar atas ke layar.

```
Q1 = 3466.75, Q3 = 12693.5, IQR = 9226.75
Pagar bawah = -10373.375, pagar atas = 26533.625
```

Pagar bawahnya negatif — dan karena kepadatan penduduk mustahil negatif, otomatis tidak ada *outlier* di sisi bawah. Perburuan tinggal ke sisi atas: kelurahan mana yang kepadatannya menembus 26.533 jiwa/km²? Saring dengan filtrasi baris Modul 2:

```
1 outlier_kepadatan = df[(df["kepadatan_pddk"] < batas_bawah) |
(df["kepadatan_pddk"] > batas_atas)]
2 outlier_kepadatan[["nama_des", "nama_kec", "kepadatan_pddk"]]
```

Baris 1

Menyaring baris DataFrame yang nilai kepadatan penduduknya berada di luar batas bawah dan atas, lalu menyimpannya dalam variabel `outlier_kepadatan`.

Baris 2

Menampilkan kolom nama desa dan kepadatan penduduk khusus untuk baris yang merupakan *outlier* (pencilan).

	nama_des	nama_kec	kepadatan_pddk
7	Kota Karang	Telukbetung Timur	43907
31	Tanjung Agung	Tanjung Karang Timur	37717
32	Kebon Jeruk	Tanjung Karang Timur	26592
33	Sawah Lama	Tanjung Karang Timur	35775
63	Sukajawa	Tanjung Karang Barat	31830
82	Sukamenanti	Kedaton	28790

Enam kelurahan tertangkap pagar, dipuncaki Kota Karang (Telukbetung Timur) dengan 43.907 jiwa/km² — hampir tujuh kali lipat median kota. Ingat: *outlier* bukan otomatis data yang salah. Kepadatan setinggi itu bisa jadi kenyataan permukiman pesisir yang memang sesak — justru karena itulah ia perlu ditandai: diperiksa kebenarannya, dan bila benar, diberi perhatian khusus dalam perencanaan.

Satu kolom sudah; tetapi memeriksa kolom-kolom lain dengan menyalin-tempel blok kode yang sama berulang kali jelas bukan cara kerja yang tertib. Serahkan pengulangannya kepada Python — jalankan pemeriksaan IQR untuk **setiap** kolom dalam `kolom_numerik` sekaligus, lengkap dengan penilaian otomatis atas hasilnya:

```
1  for kolom in kolom_numerik:
2      q1 = df[kolom].quantile(0.25)
3      q3 = df[kolom].quantile(0.75)
4      iqr = q3 - q1
5      batas_bawah = q1 - 1.5 * iqr
6      batas_atas = q3 + 1.5 * iqr
7
8      penanda_outlier = (df[kolom] < batas_bawah) | (df[kolom] > batas_atas)
9      jumlah_outlier = penanda_outlier.sum()
10
11     if jumlah_outlier == 0:
12         keterangan = "aman"
13     elif jumlah_outlier <= 5:
14         keterangan = "perlu dicermati"
15     else:
16         keterangan = "perlu investigasi"
17
18     print(f"{kolom}: {jumlah_outlier} kelurahan outlier - {keterangan}")
```

Baris 8

Variabel `penanda_outlier` adalah deretan nilai benar/salah hasil filtrasi Modul 2.

Baris 9

`.sum()` menghitung banyaknya `True` di dalamnya (Python menghitung `True` sebagai 1).

```
kepadatan_pddk: 6 kelurahan outlier - perlu investigasi
jml_tk_negeri: 12 kelurahan outlier - perlu investigasi
jml_tk_swasta: 1 kelurahan outlier - perlu dicermati
jml_ra_ba_swasta: 22 kelurahan outlier - perlu investigasi
```

Empat kolom diperiksa oleh satu blok kode. Hasilnya patut direnungkan: `jml_ra_ba_swasta` dilaporkan punya 22 “outlier” dan `jml_tk_negeri` 12 — padahal kita tahu dari `describe()` bahwa mayoritas nilai kedua kolom itu 0, sehingga Q1 dan Q3-nya sama-sama 0, IQR-nya 0, dan **setiap nilai di atas nol otomatis dicap outlier**. Pelajarannya: keluaran metode adalah alarm awal, bukan vonis — penilaian akhir tetap di tangan perencana yang memahami konteks datanya.

Konsep Python

Perulangan `for` dan `range()`

Baris `for kolom in kolom_numerik:` memerintahkan Python: “untuk setiap isi *list* `kolom_numerik`, jalankan blok di bawah ini satu kali.” Pada setiap putaran, variabel `kolom` bergiliran menampung satu isi *list* — mula-mula “`kepadatan_pddk`”, lalu “`jml_tk_negeri`”, dan seterusnya sampai isi *list* habis. Yang termasuk “blok di bawah ini” adalah semua baris yang **berindentasi** (menjorok) di bawah baris `for` — begitu indentasi berakhir, berakhir pula wilayah pengulangan.

Ini persis ritme kompilasi data di studio: satu daftar periksa yang sama diterapkan ke setiap kuesioner dalam tumpukan, kuesioner demi kuesioner, sampai tumpukan habis — tanpa menulis ulang daftar periksanya untuk tiap lembar.

Bila yang dibutuhkan bukan “untuk setiap isi *list*” melainkan “ulangi sekian kali”, pasangkan `for` dengan `range()`: `for i in range(4):` mengulang blok empat putaran, dengan `i` bernilai 0, 1, 2, 3 secara bergiliran (ingat, Python menghitung dari nol). Bentuk inilah yang akan sering muncul ketika kita menjelajah struktur data yang lebih dalam di modul-modul lanjut.

Konsep Python

Percabangan `if-elif-else`

Blok `if-elif-else` membuat kode **memilih jalan**: Python memeriksa kondisi satu per satu dari atas — kondisi `if` dulu, lalu tiap `elif` (singkatan *else if*) — dan begitu menemukan kondisi pertama yang benar, hanya blok cabang itulah yang dijalankan; sisanya dilewati. Cabang `else` adalah penampung terakhir bila tidak ada satu pun kondisi yang benar. Kondisinya sendiri dibangun dari perkakas yang sudah Anda kuasai di Modul 2: operator perbandingan (`==`, `<=`, `>`) dan operator boolean.

Analoginya adalah momen menyortir kuesioner di studio: tiap lembar hasil survei Anda periksa dengan urutan pertanyaan yang tetap — terisi lengkap? masukkan ke tumpukan siap-olah; kurang satu-dua isian? beri catatan perbaikan; rusak parah? sisihkan untuk survei ulang. Satu lembar hanya berakhir di satu tumpukan, sebagaimana satu putaran percabangan hanya menjalankan satu cabang.

Perhatikan pula bahwa percabangan tadi bekerja **di dalam** perulangan: penilaian yang sama diterapkan pada setiap kolom yang digilir `for`. Memadukan keduanya — mengulang sambil memilih — adalah pola alur kendali yang akan terus Anda pakai sampai akhir buku ini.

4.3.c Mengisi Lembar Kosong: Identifikasi dan Penanganan Missing Values

Setelah nilai yang “terlalu ada”, giliran nilai yang **tidak ada**. Dalam *DataFrame*, isian yang kosong ditandai `NaN` (*Not a Number*), dan metode `isna()` memeriksanya sel demi sel. Rangkakan dengan `.sum()` untuk merekap jumlah kekosongan per kolom, lalu — karena kolomnya 34 — saring rekap itu agar hanya kolom yang benar-benar bermasalah yang tampil:

```
1 jumlah_kosong = df.isna().sum()
2 jumlah_kosong[jumlah_kosong > 0]
```

Baris 1

Menghitung jumlah nilai yang kosong (`NA/NaN`) pada tiap kolom.

Baris 2

Menampilkan hanya kolom-kolom yang memiliki nilai kosong lebih dari nol.

```
dm_permukiman_sutet    119
jml_bangunan_sutet     123
dtype: int64
```

Dari 34 kolom, hanya dua yang berlubang — tetapi lubangnya menganga: `dm_permukiman_sutet` kosong pada 119 baris dan `jml_bangunan_sutet` pada 123 baris, dari total 126. Jembatan kecil ke Modul 3: `NaN` di Pandas adalah kembaran `NULL` di SQL — kekosongan yang sama yang dulu kita saring dengan `IS NULL`. Buktikan kesetaraannya dengan bertanya langsung ke basis data:

```
1 kueri_kosong = """
2 SELECT
3     COUNT(*) AS jumlah_baris_kosong
4 FROM
5     kelurahan
6 WHERE
7     jml_bangunan_sutet IS NULL
8 """
9
10 pd.read_sql(kueri_kosong, koneksi)
```

Baris 1,8

Menyiapkan kueri SQL untuk menghitung jumlah kelurahan (baris) yang jumlah bangunan di bawah SUTET-nya tidak diketahui (`NULL`).

Baris 10

Menjalankan kueri dan menampilkan total kelurahannya.

	jumlah_baris_kosong
0	123

Angka yang sama, 123 — dua kacamata, satu kenyataan.

Sebelum menambal, **diagnosis dulu penyebab kosongnya**. Ketiga kolom ini berkaitan: `dm_dilalui_sutet` mencatat apakah wilayah kelurahan dilalui jaringan listrik tegangan tinggi (SUTET/

SUTT/SUTTAS; kode 1 ada, kode 2 tidak — lihat kamus data PODES 2021), `dm_permukiman_sutet` mencatat ada-tidaknya permukiman di bawah jaringan itu, dan `jml_bangunan_sutet` menghitung bangunannya. Pertanyaan lanjutan tentu hanya diisi bila jawaban pertanyaan awalnya “ada”. Buktikan dengan filtrasi silang:

```
1 df[df["dm_dilalui_sutet"] == 1][["nama_des", "dm_permukiman_sutet",  
  "jml_bangunan_sutet"]]
```

Baris 1

Memfilter DataFrame untuk kelurahan yang dilalui SUTET, kemudian hanya menampilkan kolom nama desa, jenis permukiman SUTET, dan jumlah bangunan di bawah SUTET.

	nama_des	dm_permukiman_sutet	jml_bangunan_sutet
4	Batu Putuk	2.0	NaN
11	Gedong Pakuon	2.0	NaN
22	Srengsem	2.0	NaN
68	Sumber Agung	1.0	12.0
109	Korpri Jaya	1.0	4.0
111	Korpri Raya	1.0	25.0
112	Sukarame Baru	2.0	NaN

Tepat 7 kelurahan dilalui SUTET — dan hanya pada merekalah `dm_permukiman_sutet` terisi; `jml_bangunan_sutet` bahkan hanya terisi pada 3 kelurahan yang permukimannya memang ada di bawah jaringan (kode 1). Kekosongan seperti ini disebut **kosong struktural**: bukan petugas lupa mencatat, melainkan pertanyaannya memang tidak berlaku bagi 119 kelurahan lain. Diagnosis ini menentukan resep penanganannya.

Pandas menyediakan dua penanganan pabrikan: `dropna()` membuang setiap baris yang memuat kekosongan, dan `fillna(nilai)` menambal kekosongan dengan nilai tertentu. Coba dulu yang pertama — tetapi periksa dampaknya lewat `.shape` sebelum benar-benar memakainya:

```
1 print(df.shape)  
2 print(df.dropna().shape)
```

Baris 1

Menampilkan jumlah baris dan kolom DataFrame sebelum metode penghapusan dilakukan.

Baris 2

Menampilkan jumlah baris dan kolom setelah semua baris yang mengandung sel kosong dihapus dengan `dropna()`.

```
(126, 34)  
(3, 34)
```

 Penting!

dropna() Bukan Jawaban Otomatis

Membuang semua baris yang berlubang menyisakan **3 kelurahan dari 126** — kita kehilangan 97% kota hanya karena dua kolom yang kosongnya pun struktural. Baris di atas sengaja tidak menimpa variabel `df`: `df.dropna()` menghasilkan *DataFrame* baru, dan selama hasilnya tidak ditugaskan kembali ke variabel, data asli tetap utuh. Jadikan `.shape` kebiasaan wajib sebelum memutuskan membuang baris — dan buang hanya bila kekosongannya sedikit serta barisnya memang tidak terpakai. `.dropna()` di sini adalah **method** milik `df` — sama seperti `.head()` yang Anda pakai di Modul 2, ia tidak berdiri sendiri dan hanya masuk akal ditumpangkan pada data.

Untuk kasus kita, resep yang masuk akal justru menambal. Pada `jml_bangunan_sutet`, kosong struktural bermakna pasti: tidak dilalui SUTET, atau tidak ada permukiman di bawahnya — artinya **nol bangunan**. Tambah dengan `fillna(0)`:

```
1 df["jml_bangunan_sutet"] = df["jml_bangunan_sutet"].fillna(0)
2
3 print(df["jml_bangunan_sutet"].isna().sum())
```

Baris 1

Mengisi kekosongan (NaN) pada kolom `jml_bangunan_sutet` dengan angka 0.

Baris 3

Mengecek ulang sisa kolom kosong untuk memverifikasi kekosongan sudah ditambah.

```
0
```

Kekosongannya tuntas. Adapun `dm_permukiman_sutet` kita **biarkan kosong**: ia kolom kode kategori (1/2), dan menambalnya dengan angka karangan justru memalsukan jawaban “pertanyaan ini tidak berlaku”. Tidak semua lubang harus ditambah — yang wajib adalah *mengetahui* lubangnya dan *sadar* memutuskan perlakuannya.

Rumuskan keputusan itu secara tertib untuk ketiga kolom SUTET sekaligus — perulangan dan percabangan yang baru Anda pelajari kini bekerja berpasangan:

```
1 kolom_sutet = ["dm_dilalui_sutet", "dm_permukiman_sutet", "jml_bangunan_sutet"]
2
3 for kolom in kolom_sutet:
4     persen_kosong = df[kolom].isna().sum() / len(df) * 100
5
6     if persen_kosong == 0:
7         keputusan = "lengkap, siap dianalisis"
8     elif persen_kosong < 50:
9         keputusan = "tambah dengan fillna"
10    else:
11        keputusan = "periksa: kemungkinan kosong struktural"
12
13    print(f"{kolom}: {persen_kosong:.1f}% kosong - {keputusan}")
```

Baris 1

Membuat daftar kolom yang berkaitan dengan data SUTET.

Baris 3

Melakukan perulangan melalui tiap kolom dalam daftar tersebut.

Baris 4

Menghitung persentase baris yang kosong pada kolom tersebut.

Baris 6,8,10

Memeriksa kondisi apakah tidak ada yang kosong, sebagian kosong, atau seluruhnya kosong.

Baris 13

Menampilkan hasil diagnosa berdasarkan persentase kekosongan tersebut.

```
dm_dilalui_sutet: 0.0% kosong - lengkap, siap dianalisis
dm_permukiman_sutet: 94.4% kosong - periksa: kemungkinan kosong struktural
jml_bangunan_sutet: 0.0% kosong - lengkap, siap dianalisis
```

Perhatikan `jml_bangunan_sutet` kini dilaporkan lengkap — buah dari tambalan barusan. (Notasi `:.1f` di dalam f-string Modul 1 sekadar merapikan tampilan angka menjadi satu digit di belakang koma.)

4.3.d Hubungan Awal: Analisis Korelasi Kasar Antarvariabel Numerik

Karakter tiap kolom sudah dikenali satu per satu; pemeriksaan pamungkas EDA adalah melihat **hubungan antar-kolom**. Ukuran kasarnya adalah **koefisien korelasi** (r): angka antara -1 dan $+1$ yang menunjukkan arah dan kekuatan hubungan garis lurus dua variabel — mendekati $+1$ berarti keduanya cenderung naik bersama, mendekati -1 berarti yang satu naik ketika yang lain turun, dan di sekitar 0 berarti nyaris tak ada pola bersama. Metode `corr()` menghitungnya untuk semua pasangan kolom sekaligus. Ujilah dugaan yang wajar bagi perencanaan: *makin padat sebuah kelurahan, makin banyak pula fasilitas pendidikannya?*

```
1 kolom_korelasi = ["kepadatan_pddk", "jml_tk_negeri", "jml_tk_swasta",
2                  "jml_sd_negeri", "jml_sd_swasta"]
3 df[kolom_korelasi].corr()
```

Baris 1

Menyusun daftar variabel yang akan dicari nilai korelasinya.

Baris 3

Memanggil metode `.corr()` untuk menghasilkan matriks korelasi.

	kepadatan_pddk	jml_tk_negeri	jml_tk_swasta	jml_sd_negeri	jml_sd_swasta
kepadatan_pddk	1.000000	-0.073817	-0.192438	0.155989	-0.041028
jml_tk_negeri	-0.073817	1.000000	0.003054	-0.061323	-0.102833
jml_tk_swasta	-0.192438	0.003054	1.000000	0.034594	0.536261
jml_sd_negeri	0.155989	-0.061323	0.034594	1.000000	-0.074743
jml_sd_swasta	-0.041028	-0.102833	0.536261	-0.074743	1.000000

Cara membaca **matriks korelasi** ini: tiap sel adalah r untuk pasangan kolom baris-kolomnya; diagonalnya selalu 1 (setiap variabel berkorelasi sempurna dengan dirinya sendiri); dan matriksnya simetris — cukup baca separuhnya.

Hasilnya menampar dugaan kita. Korelasi kepadatan penduduk dengan keempat kolom fasilitas semuanya lemah (dari $-0,192$ sampai $0,156$): pada data ini, **kelurahan yang lebih padat tidak serta-merta lebih kaya fasilitas pendidikan** — sinyal awal yang layak menjadi bahan diskusi pemerataan layanan. Justru pasangan terkuat muncul di tempat lain: `jmL_tk_swasta` dengan `jmL_sd_swasta` ($r = 0,536$, kekuatan sedang) — kelurahan yang banyak TK swastanya cenderung banyak pula SD swastanya, gelagat bahwa penyedia layanan swasta berkerumun di kantong pasar yang sama.

⚠ Penting!

Korelasi Bukan Sebab-Akibat

Sekuat apa pun r , korelasi hanya mencatat dua hal bergerak bersama — ia tidak pernah membuktikan yang satu *menyebabkan* yang lain. Analisis kita di sini juga sengaja disebut korelasi **kasar**: belum ada uji signifikansi atau kendali atas variabel lain; perkakas statistika yang lebih ketat untuk itu Anda pelajari di mata kuliah statistika. Dalam EDA, posisi korelasi adalah pemandu — penunjuk hubungan mana yang pantas diselidiki lebih dalam, bukan hakim yang memutus perkara.

4.3.e Sintesis EDA: Menarik Kesimpulan Karakteristik Sosio-Demografi Kota

Pemeriksaan selesai; saatnya menyusun lembar kompilasi versi perencanaan. Digabungkan, temuan-temuan EDA kita membentuk potret karakter sosio-demografi Kota Bandar Lampung menurut datanya sendiri:

- **Kepadatan yang timpang.** Separuh kelurahan berkepadatan di bawah 6.450 jiwa/km², tetapi ekor distribusinya panjang ke atas hingga 43.907 — enam kelurahan ekstrem (terpadat: Kota Karang) menyeret rata-rata kota jauh di atas mediannya.
- **Kota di atas dataran dan lembah.** Modus topografi adalah dataran (77 kelurahan), disusul lembah/DAS (49); dan dua kelurahan ternyata masih tercacah berstatus perdesaan.
- **Layanan pendidikan usia dini bertumpu pada swasta.** Lebih dari tiga perempat kelurahan tanpa TK negeri, dan RA/BA negeri tidak ada sama sekali (kolom seragam nol).
- **Kekosongan data yang bermakna.** Dua kolom SUTET kosong secara struktural — kosongnya adalah informasi (“tidak berlaku”), bukan kelalaian; satu kolom kita tambal nol, satu lagi sengaja dibiarkan.
- **Kepadatan tidak menjamin fasilitas.** Korelasi kepadatan-fasilitas pendidikan lemah; yang menonjol justru kerumunan sesama layanan swasta (TK–SD swasta, $r = 0,536$).

Potret inilah bekal kita melangkah: dugaan mana yang pantas diuji serius, kolom mana yang siap pakai, dan angka mana yang harus diperlakukan hati-hati. Alur perkakasnya pun kini utuh — basis data menertibkan penyimpanan (Modul 3), Pandas mengolah (Modul 2), EDA mengenali karakter dan kualitasnya (modul ini) — dan pada Modul 5, temuan-temuan yang masih berupa angka ini akan kita bunyikan menjadi **visualisasi**.

Tutup sesi kerja basis data Anda:

```
1 koneksi.close()
```

Baris 1

Menutup koneksi ke *database* setelah semua operasi selesai.

4.4 Latihan Mandiri

Kerjakan di notebook baru `modul/latihan-04.ipynb` dengan *kernel* `analitika-perkotaan`. Alurnya sama persis dengan instruksi di atas; yang berganti adalah fasilitasnya — dari TK ke **pendidikan dasar (SD dan SMP)** — plus satu kolom kategorikal yang lebih kaya.

1. **Buka lemari arsip.** Buka koneksi ke `bd.sqlite`, lalu tarik profil lengkap kelurahan ber-nama `kec` ke *DataFrame* memakai `read_sql` dengan kueri `LEFT JOIN` seperti pada instruksi.
2. **Kolom total.** Buat dua kolom baru (Modul 2): `jumlah_sd` dari `jumlah_sd_negeri + jumlah_sd_swasta`, dan `jumlah_smp` dari `jumlah_smp_negeri + jumlah_smp_swasta`.
3. **Karakter data.** Tampung `["jumlah_sd", "jumlah_smp"]` dalam variabel `list`, tampilkan `describe()`-nya, lalu tuliskan di sel teks: berapa median masing-masing, dan apakah sebarannya lebar?
4. **Frekuensi lapangan usaha.** Rekap `value_counts()` untuk kolom kategorikal `lap_usaha_sebagian_warga` — sumber penghasilan utama sebagian besar warga tiap kelurahan. Arti kode yang muncul di data Bandar Lampung (selengkapnya lihat kamus data PODES 2021 yang dibagikan dosen):

Kode	Lapangan usaha
1	Pertanian, kehutanan, dan perikanan
3	Industri pengolahan
5	Pengadaan air, pengelolaan sampah, limbah, dan daur ulang
6	Konstruksi
7	Perdagangan besar dan eceran; reparasi mobil dan sepeda motor
8	Transportasi dan pergudangan
9	Penyediaan akomodasi dan makan minum
13	Jasa perusahaan
14	Administrasi pemerintahan, pertahanan, dan jaminan sosial wajib
17	Jasa lainnya

Tuliskan satu kalimat: lapangan usaha apa yang menjadi tumpuan terbanyak kelurahan-kelurahan Bandar Lampung, dan masuk akalkah itu untuk sebuah kota?

5. **Perburuan outlier.** Dengan perulangan `for` atas `list` `["jumlah_sd", "jumlah_smp"]`, hitung pagar IQR tiap kolom, cacah *outlier*-nya, dan berikan keterangan otomatis memakai percabangan `if-elif-else` seperti pada instruksi. Kelurahan mana pemuncak *outlier* `jumlah_sd`?
6. **Kelengkapan data.** Periksa kekosongan kolom-kolom yang Anda pakai (`jumlah_sd`, `jumlah_smp`, dan kolom pecahannya) dengan `isna().sum()`. Adakah kolom yang kosong? Jika ada, diagnosis dulu penyebabnya seperti pada instruksi sebelum memutuskan penanganannya.
7. **Hubungan antarvariabel.** Hitung matriks korelasi `df[["jumlah_sd", "jumlah_smp"]].corr()`. Seberapa kuat hubungan jumlah SD dengan jumlah SMP antarkelurahan?

8. **Sintesis dan tutup.** Tutup notebook dengan satu paragraf sintesis ala laporan kompilasi: karakter sebaran fasilitas pendidikan dasar, temuan *outlier*-nya, kelengkapan datanya, dan makna korelasi SD-SMP bagi pemerataan layanan. Tutup koneksi basis data, lalu jalankan **Restart kernel & Run All** — seluruh sel harus lolos berurutan.

5 Visualisasi Data dengan Matplotlib dan Seaborn

! Sub-CPMK Pertemuan Ini

Mahasiswa mampu merumuskan visualisasi data perkotaan yang informatif menggunakan Matplotlib dan Seaborn, memilih jenis grafik yang tepat sesuai karakteristik data, menerapkan estetika grafik, dan menyusun interpretasi naratif atas visualisasi tersebut.

5.1 Pendahuluan

Coba ingat kembali pengalaman Anda menyusun laporan tugas besar, laporan kuliah lapangan, atau laporan studio berisi puluhan tabel data hasil survei lapangan. Ada momen ketika Anda sendiri, sebagai penyusun data, kesulitan menjelaskan temuannya secara lisan kepada teman satu kelompok — padahal Anda adalah orang yang paling paham isi datanya. Jika penyusun data saja kesulitan menceritakan ulang temuannya, bagaimana dengan pembaca yang baru pertama kali melihat data tersebut?

Fenomena ini bukan kebetulan, otak manusia tidak dirancang untuk memproses ratusan angka berjajar dalam baris dan kolom secara cepat, tetapi sangat terlatih mengenali bentuk — batang yang lebih tinggi langsung ditangkap sebagai “lebih banyak”, titik-titik yang membentuk pola diagonal langsung ditangkap sebagai “berhubungan”, kurva yang menggerombol di satu sisi langsung ditangkap sebagai “tidak merata”. Visualisasi data memanfaatkan kemampuan alami ini, menerjemahkan beban membaca angka menjadi pengenalan pola yang sudah kita miliki sejak lahir.

Bagi mahasiswa perencanaan wilayah dan kota, kemampuan ini bukan sekadar nilai tambah, melainkan kompetensi inti. Sepanjang studi dan karier nanti, Anda akan terus dihadapkan pada data sekunder dalam jumlah besar — profil kelurahan, data sensus, hasil survei kebutuhan fasilitas — yang harus diringkas menjadi bahan pengambilan keputusan dalam waktu yang terbatas, misalnya pemangku kepentingan pemerintah daerah, anggota legislatif, hingga warga dalam forum konsultasi publik. Keakuratan analisis Anda tidak banyak berarti jika ia tidak pernah benar-benar tersampaikan.

Di sinilah Python, melalui pustaka Matplotlib dan Seaborn, menjadi alat kerja yang relevan dipelajari. Matplotlib memberi kendali terperinci atas setiap elemen grafik — posisi sumbu, ukuran teks, warna batang, hingga penempatan legenda — sementara Seaborn, yang dibangun di atasnya, menawarkan pendekatan lebih ringkas dengan gaya visual yang lebih estetik secara bawaan, sangat berguna untuk eksplorasi pola secara cepat sebelum menentukan grafik mana yang layak ditampilkan dalam laporan akhir.

Namun perlu digarisbawahi, kemampuan teknis membuat grafik hanyalah satu sisi keterampilan yang kita latih; sisi lainnya adalah menuliskan interpretasi naratif yang bermakna bagi arah pembangunan kota, sebab grafik yang indah tanpa narasi yang tepat hanya akan berhenti menjadi hiasan, bukan alat pengambilan keputusan.

5.2 Gambaran Umum Kasus

Di *command center* pengelolaan Kota Bandar Lampung, layar-layar besar di depan operator tidak menampilkan tabel angka mentah. Yang tersaji di sana adalah grafik: batang yang menjulang menandai kecamatan mana yang timpang fasilitas pendidikannya, sebaran titik yang memperlihatkan

pola tersembunyi antara kepadatan penduduk dan ketersediaan sekolah, atau kurva distribusi yang membedakan kelurahan biasa dari yang benar-benar ekstrem. Tabel baris-kolom yang kaku sulit dibaca sekilas oleh operator yang harus mengambil keputusan cepat; grafik yang dirancang dengan baik mempercepat pemahaman itu dan meminimalkan risiko salah tafsir kebijakan pembangunan kota.

Namun grafik yang asal dibuat sama berbahayanya dengan tabel mentah yang tidak diolah. Grafik tanpa judul yang jelas, label sumbu yang terpotong, warna yang saling bertabrakan, atau legenda yang hilang, dapat membuat operator command center salah membaca kondisi kota, bahkan menyesatkan pengambil keputusan yang memantau layar itu dari kejauhan. Dalam modul ini kita, sebagai operator command center, akan mempelajari prinsip dasar visualisasi data yang baik dan mempraktikkannya untuk memvisualisasikan profil fasilitas pendidikan dan kepadatan penduduk Kota Bandar Lampung menggunakan pustaka Matplotlib dan Seaborn.

Pada modul ini, kedua pustaka tersebut akan digunakan untuk mengolah kembali dataset `bdl_kelurahan.csv`: kolom-kolom seperti `jml_tk_negeri`, `jml_tk_swasta`, `kepadatan_pddk`, dan `lap_usaha_sebagian_warga` akan diubah menjadi rangkaian grafik yang menjawab pertanyaan perencanaan secara langsung — kelurahan mana yang timpang fasilitas pendidikannya, apakah kepadatan penduduk berkaitan dengan ketersediaan fasilitas, dan bagaimana pola mata pencaharian warga tersebar di seluruh kota — sekaligus melatih kita menangani data kosong yang lazim dijumpai pada data sekunder pemerintah.

5.3 Instruksi

1. Buat notebook baru `modul/modul-05.ipynb` di direktori proyek Anda.
2. Sambungkan ke *kernel analitika-perkotaan*.
3. Kerjakan seluruh instruksi di dalamnya.

5.3.a Prinsip Visualisasi: Memilih Grafik Sesuai Pertanyaan Perencanaan

Sebelum mulai memprogram, pahami terlebih dahulu kerangka pikir visualisasi data yang dirancang untuk kebutuhan analisis dan laporan operasional yang disajikan kepada pengambil keputusan kota:

- **Tiga Prinsip Schwabish:**

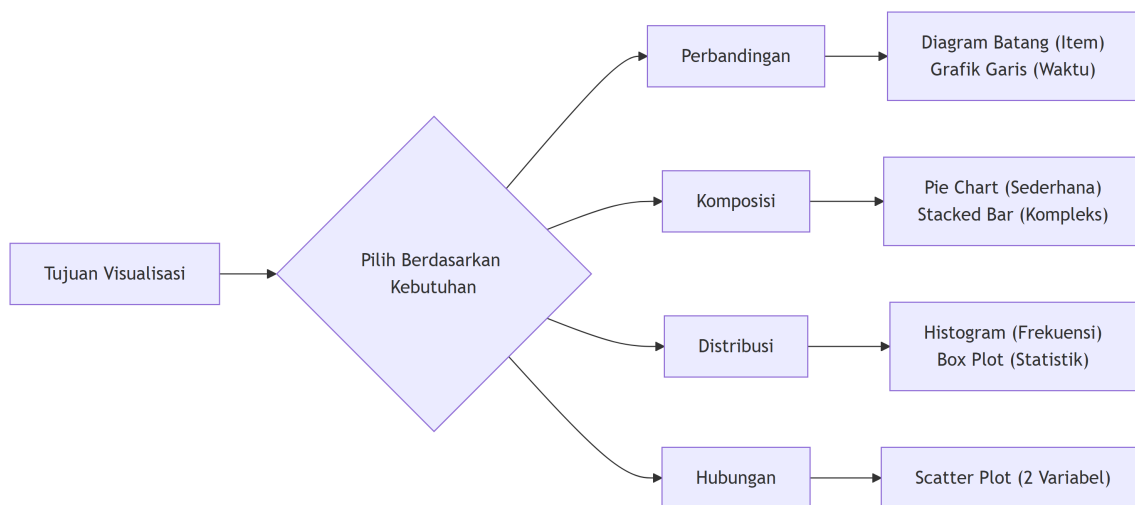
- ▶ *Tunjukkan Data*: Pastikan perhatian pembaca tertuju langsung pada informasi utama grafik.
- ▶ *Kurangi Kekacauan (Reduce Clutter)*: Bersihkan elemen dekoratif non-data yang mengganggu kognisi (seperti efek 3D, bayangan, atau garis kisi-kisi latar belakang yang terlalu tebal).
- ▶ *Integrasikan Teks*: Tempatkan label atau legenda sedekat mungkin dengan data untuk mengurangi beban kognitif pembaca.

- **Taksonomi Data:**

- ▶ *Kuantitatif (Numerik)*: Data yang dapat dihitung atau diukur (misalnya kepadatan penduduk atau jumlah sekolah). Data ini bisa berupa *diskrit* (bulat, seperti jumlah sekolah) atau *kontinu* (desimal, seperti luas wilayah).
- ▶ *Kategorikal (Kualitatif)*: Data pengelompokkan (seperti nama kecamatan atau jenis topografi). Data ini bisa berupa *nominal* (tanpa urutan, seperti jenis lapangan usaha) atau *ordinal* (memiliki tingkatan/urutan, seperti tingkat kepuasan layanan).
- ▶ **Analisis “So What?”**: Jangan pernah menggunakan grafik garis (*line chart*) bersambung untuk data yang bersifat diskrit antarkategori (misalnya jumlah sekolah per kelurahan). Garis yang terhubung

secara implisit menunjukkan adanya nilai antara di sepanjang garis tersebut, padahal di lapangan tidak ada nilai transisi antar-kelurahan. Ini adalah kesalahan visualisasi yang fatal.

- **Pohon Keputusan Pemilihan Grafik:** Berikut adalah panduan sederhana memilih grafik yang sesuai dengan tujuan analisis Anda:



Catatan: Diagram di atas membantu kita mencocokkan pertanyaan perencanaan dengan jenis visual yang tepat secara instan.

5.3.b Persiapan Data dan Impor Pustaka

Seperti pada modul-modul sebelumnya, kita akan menggunakan basis data relasional SQLite `bd1.sqlite` yang berada di folder dataset sebagai sumber data utama. Impor pustaka yang diperlukan (`sqlite3`, `pandas`, `matplotlib.pyplot`, dan `seaborn`), buka koneksi basis data, lalu muat data gabungan kelurahan dan kecamatan menggunakan `LEFT JOIN` ke dalam `DataFrame` Pandas:

```
1  import sqlite3
2  import pandas as pd
3  import matplotlib.pyplot as plt
4  import seaborn as sns
5
6  path_db = "../dataset-ganjil-2026-2027/bd1.sqlite"
7  koneksi = sqlite3.connect(path_db)
8
9  kueri = """
10 SELECT
11     kel.*,
12     kec.nama_kec
13 FROM
14     kelurahan kel
15 LEFT JOIN
16     kecamatan kec ON kel.kode_kec = kec.kode_kec
17 """
18 df = pd.read_sql(kueri, koneksi)
19 print(df.shape)
```

Baris 6-7

Alamat berkas basis data disimpan ke variabel `path_db`, lalu koneksinya dibuka dan ditampung di variabel `koneksi`.

Baris 17-18

Kueri SQL ditampung di variabel `kueri` agar rapi dibaca, lalu fungsi `read_sql()` mengirim kueri itu melalui variabel `koneksi` dan menampung hasilnya sebagai `DataFrame` di `df`.

```
(126, 34)
```

5.3.c Diagram Batang: Membandingkan Fasilitas Antar-Kecamatan

Pertanyaan pertama kita: *Kecamatan mana yang memiliki jumlah fasilitas taman kanak-kanak (TK) terbanyak?*

Untuk menjawab perbandingan kuantitas antar-item kategorikal (nama kecamatan), grafik batang adalah pilihan paling tepat. Kita perlu menghitung total TK (`jml_tk_negeri` + `jml_tk_swasta`) untuk setiap kelurahan terlebih dahulu, lalu mengelompokkannya (*group by*) berdasarkan nama kecamatan, dan mengurutkannya dari yang terbanyak.

```
1 df["total_tk"] = df["jml_tk_negeri"] + df["jml_tk_swasta"]
2
3 tk_per_kec = df.groupby("nama_kec")["total_tk"].sum().reset_index()
4 tk_per_kec = tk_per_kec.sort_values(by="total_tk", ascending=False)
5
6 plt.figure(figsize=(10, 6))
7 sns.barplot(
8     data=tk_per_kec,
9     x="total_tk",
10    y="nama_kec",
11    hue="nama_kec",
12    palette="viridis",
13    legend=False
14 )
15 plt.xlabel("Jumlah Taman Kanak-Kanak (TK)")
16 plt.ylabel("Kecamatan")
17 plt.show()
```

Baris 1

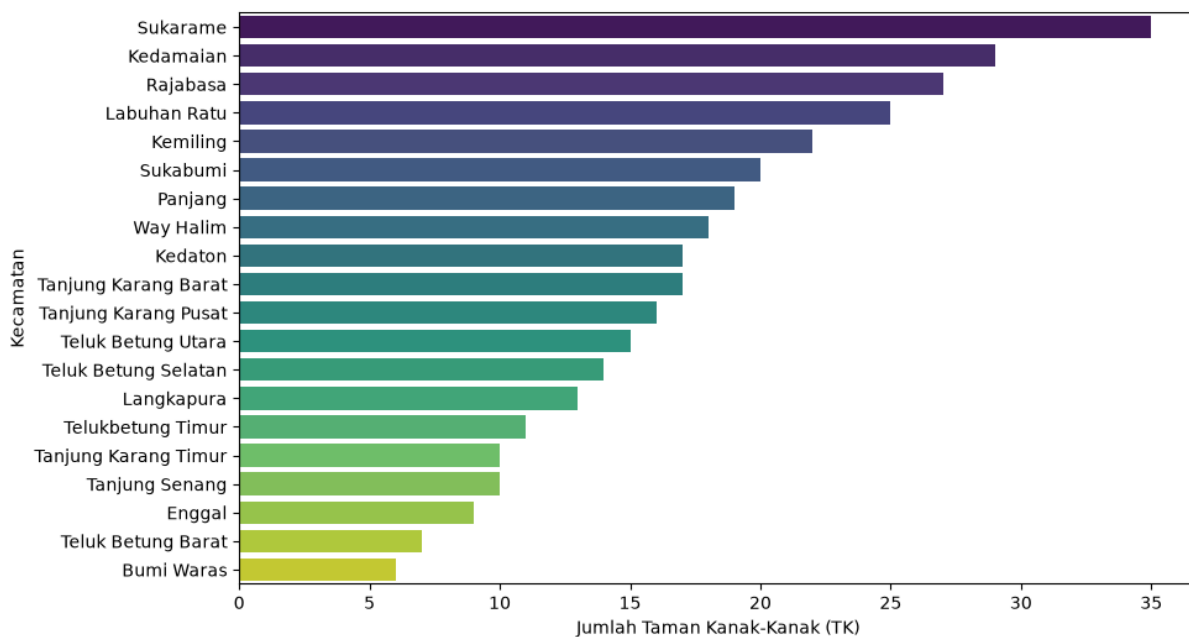
Kolom baru `total_tk` menjumlahkan TK negeri dan swasta di setiap kelurahan.

Baris 3-4

`groupby("nama_kec")` mengelompokkan baris berdasarkan nama kecamatan, `.sum()` menjumlahkan `total_tk` tiap kelompok, lalu `sort_values()` mengurutkan hasilnya dari kecamatan dengan total TK terbanyak.

Baris 6,14

`plt.figure()` membuka kanvas gambar baru dengan ukuran yang ditentukan, lalu `sns.barplot()` menggambar diagram batangnya.



⚠️ Penting!

Etika dan Batas Sumbu Y pada Grafik Batang

Grafik batang menggunakan panjang visual untuk membandingkan kuantitas. Karena itu, sumbu kuantitas (dalam diagram batang horizontal di atas adalah sumbu X, sedangkan pada diagram batang vertikal adalah sumbu Y) **wajib dimulai dari nol**. Jika Anda memotong sumbu kuantitas di atas nol, perbedaan tinggi batang akan terlihat jauh lebih dramatis daripada aslinya, sehingga membohongi pembaca secara visual. Hal ini dikategorikan sebagai distorsi data yang tidak etis.

Namun, untuk keperluan pembuktian teknis tertentu di mana Anda ingin mengatur batas sumbu secara manual (misalnya ingin menyoroti perbedaan yang sangat tipis pada grafik lain, atau memotong nilai dasar secara sengaja), Anda bisa menggunakan perintah limit pada sumbu: - Jika diagram batang vertikal atau sumbu Y: `plt.ylim(bottom=nilai_minimum)` atau `ax.set_ylim(bottom=nilai_minimum)`. - Jika diagram batang horizontal atau sumbu X: `plt.xlim(left=nilai_minimum)` atau `ax.set_xlim(left=nilai_minimum)`.

Terapkan teknik ini dengan sangat hati-hati, sertakan label nilai angka yang jelas di atas batang, dan beri catatan penjelas pada grafik Anda agar tidak menimbulkan salah tafsir.

5.3.d Scatter Plot: Menelusuri Hubungan Kepadatan dan Fasilitas

Pertanyaan kedua kita: *Apakah kelurahan yang semakin padat penduduknya memiliki jumlah fasilitas TK yang lebih banyak?*

Ini adalah pertanyaan hubungan (*relationship*) antara dua variabel numerik (*kepadatan_pddk* dan *total_tk*). Sesuai pohon keputusan, grafik sebar (*scatter plot*) adalah pilihan ideal untuk melihat sebaran titik amatan ini.

```

1 plt.figure(figsize=(10, 6))
2 sns.scatterplot(data=df, x="kepadatan_pddk", y="total_tk", alpha=0.7,
3 color="teal")
4 plt.xlabel("Kepadatan Penduduk (jiwa/km²)")
5 plt.ylabel("Jumlah Taman Kanak-Kanak (TK)")
6 plt.show()

```

Baris 1

`plt.figure()` membuka kanvas gambar baru dengan ukuran yang ditentukan.

Baris 2

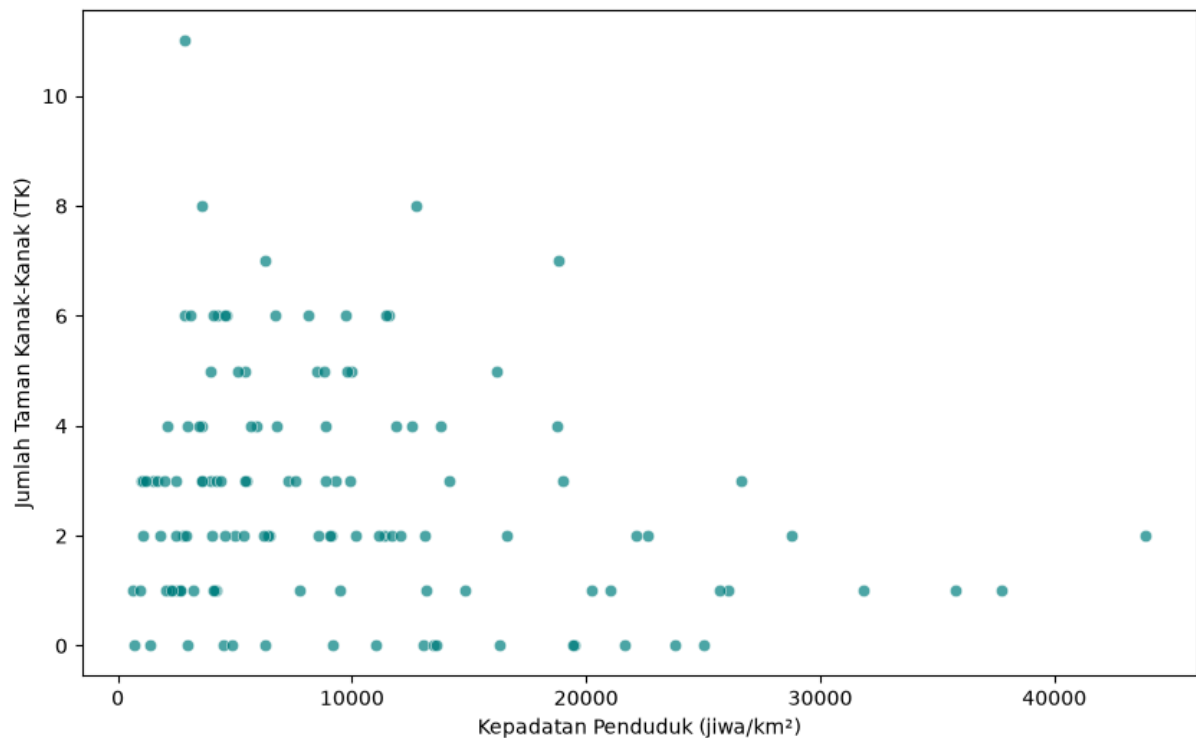
`sns.scatterplot()` menggambar titik-titik sebar dari dua kolom numerik.

Baris 3-4

`plt.xlabel()` dan `plt.ylabel()` memberi label pada sumbu X dan Y.

Baris 5

`plt.show()` menampilkan grafik di bawah sel.



✚ Pengetahuan VS Code

Fitur Interactive Plot Viewer

Ketika Anda menggunakan Visual Studio Code untuk menjalankan Jupyter Notebook, arahkan kursor mouse ke bagian grafik yang baru saja ter-render. Di sudut kanan atas gambar, VS Code secara otomatis menyediakan tombol ikon untuk mengaktifkan *Plot Viewer*. Dengan alat ini, Anda bisa memperbesar (zoom), menggeser area (pan), atau menyimpan file gambar secara langsung tanpa perlu mengetik kode tambahan di editor Anda.

5.3.e Histogram, Density Plot, dan Box Plot: Membaca Distribusi Kepadatan Penduduk Kota

Pertanyaan ketiga kita: *Bagaimana sebaran kepadatan penduduk antarkelurahan di Bandar Lampung? Apakah mayoritas kelurahan memiliki kepadatan penduduk yang seragam, atau ada ketimpangan sebaran?*

Ini adalah pertanyaan seputar distribusi (*distribution*). Kita akan memadukan histogram (mengelompokkan nilai ke dalam tabung/bin) dengan kurva estimasi kepadatan kernel (*Kernel Density Estimate* atau KDE) untuk memvisualisasikan bentuk sebarannya:

```
1 plt.figure(figsize=(10, 6))
2 sns.histplot(df["kepadatan_pddk"], kde=True, bins=20, color="purple")
3 plt.xlabel("Kepadatan Penduduk (jiwa/km2)")
4 plt.ylabel("Frekuensi Kelurahan")
5 plt.show()
```

Baris 1

`plt.figure()` membuka kanvas gambar baru.

Baris 2

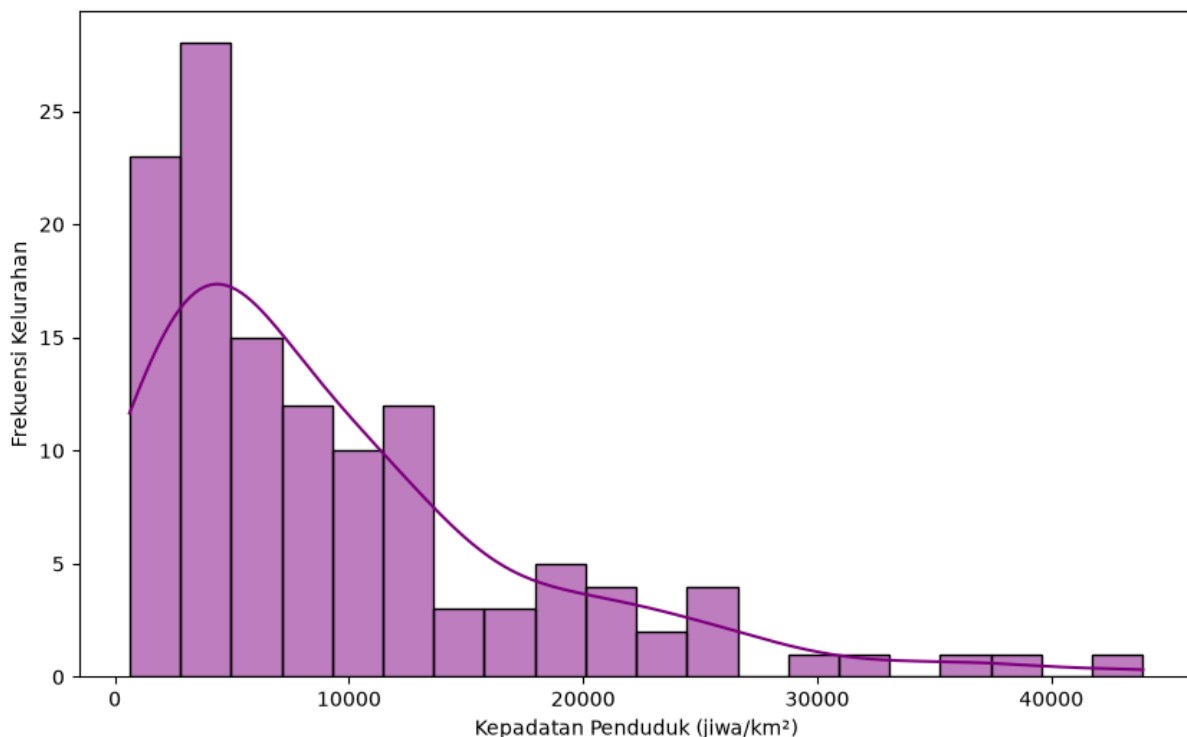
`sns.histplot()` menggambar histogram dengan opsi KDE (*Kernel Density Estimate*) untuk kurva estimasi kepadatan.

Baris 3-4

`plt.xlabel()` dan `plt.ylabel()` memberi label pada sumbu.

Baris 5

`plt.show()` menampilkan grafik.



Selain histogram, pohon keputusan kita juga menyebut *box plot* sebagai grafik distribusi. Jika histogram menunjukkan bentuk sebaran, *box plot* meringkas sebaran itu menjadi lima angka statistik: nilai

minimum, kuartil pertama, median, kuartil ketiga, dan nilai maksimum — sementara titik-titik di luar “pagar” adalah *outlier* yang sudah kita kenali di Modul 4. Keunggulan utamanya: *box plot* mudah dijabarkan antarkelompok kategorikal. Mari bandingkan sebaran kepadatan penduduk antarjenis topografi wilayah:

```
1 plt.figure(figsize=(10, 5))
2 sns.boxplot(data=df, x="topo", y="kepadatan_pddk", color="lightsteelblue")
3 plt.xlabel("Topografi Wilayah")
4 plt.ylabel("Kepadatan Penduduk (jiwa/km²)")
5 plt.show()
```

Baris 1

`plt.figure()` membuka kanvas gambar baru.

Baris 2

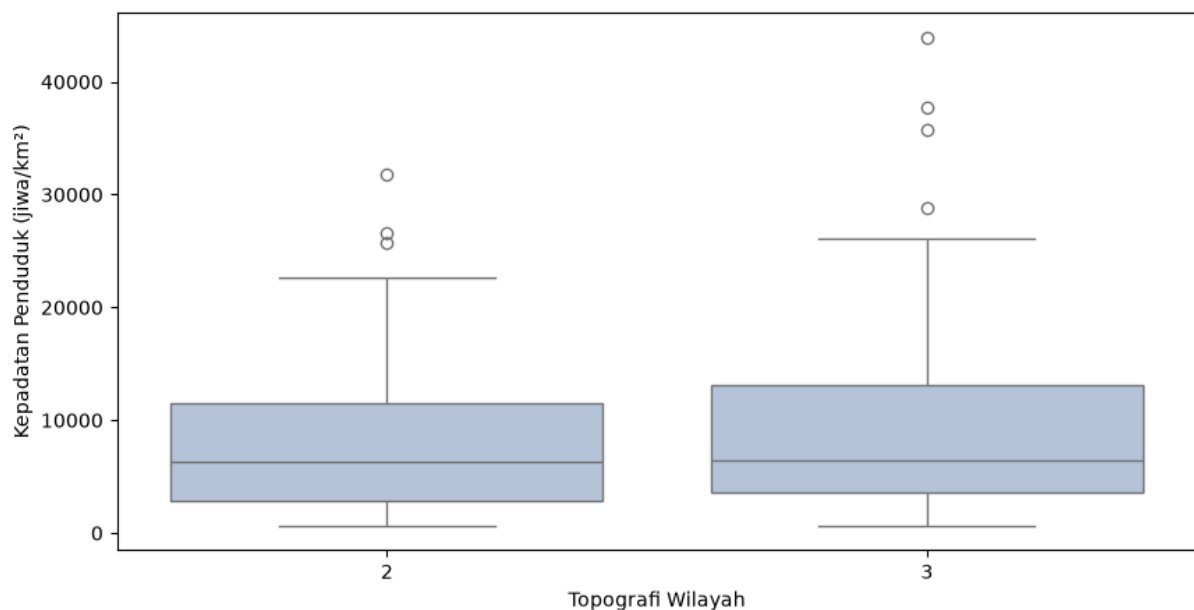
`sns.boxplot()` menggambar *box plot* yang memperlihatkan distribusi kepadatan penduduk (di sumbu Y) untuk setiap kategori topografi (di sumbu X).

Baris 3-4

`plt.xlabel()` dan `plt.ylabel()` memberi label pada sumbu.

Baris 5

`plt.show()` menampilkan grafik.



5.3.f Estetika Visual: Menambahkan Judul, Label, Legenda, dan Warna Harmonis

Grafik-grafik di atas secara teknis sudah benar, tetapi tampilannya masih sangat dasar. Untuk laporan command center yang bernilai profesional, kita wajib menerapkan estetika visual: menambahkan judul yang deskriptif, memperjelas label sumbu, merapikan legenda, dan memoles tampilannya.

Kita akan menyusun opsi pengaturan font dan warna di dalam variabel *dictionary*, lalu membongkarnya (*unpacking*) ke dalam perintah visualisasi untuk menjaga efisiensi penulisan kode:

```

1  sns.set_theme(style="whitegrid")
2
3  opsi_judul = {
4      "fontsize": 14,
5      "fontweight": "bold",
6      "color": "darkblue",
7      "pad": 15
8  }
9  opsi_label = {
10     "fontsize": 11,
11     "color": "dimgray"
12 }
13
14 plt.figure(figsize=(10, 6))
15 sns.scatterplot(
16     data=df,
17     x="kepadatan_pddk",
18     y="total_tk",
19     hue="topo",
20     palette="Set2",
21     alpha=0.8,
22     s=80
23 )
24
25 plt.title("Hubungan Kepadatan Penduduk dan Jumlah TK Kelurahan", **opsi_judul)
26 plt.xlabel("Kepadatan Penduduk (jiwa/km²)", **opsi_label)
27 plt.ylabel("Jumlah Taman Kanak-Kanak (TK)", **opsi_label)
28 plt.legend(title="Topografi Wilayah", bbox_to_anchor=(1.05, 1), loc='upper
left')
29 plt.tight_layout()
30 plt.show()

```

Baris 1

Fungsi `sns.set_theme()` mengatur gaya visual bawaan Seaborn untuk seluruh grafik berikutnya.

Baris 3,9

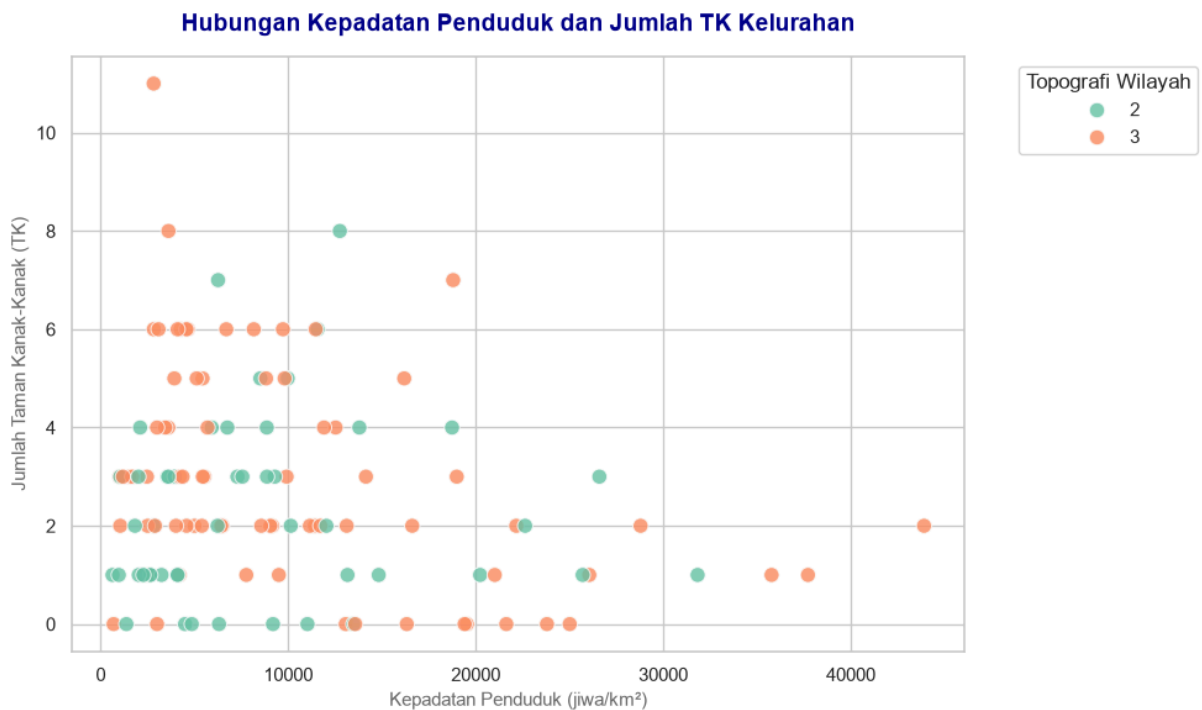
Variabel `opsi_judul` dan `opsi_label` adalah *dictionary* berisi opsi teks agar tampilan judul dan label seragam serta mudah diperbarui dari satu tempat.

Baris 14,23

`plt.figure()` membuka kanvas gambar baru, lalu `sns.scatterplot()` menggambar grafik sebar yang sudah dipoles.

Baris 25-27

Tanda `**opsi_judul` dan `**opsi_label` membongkar isi *dictionary* tersebut sebagai argumen kata kunci pada `plt.title()`, `plt.xlabel()`, dan `plt.ylabel()`.



Konsep Python

Dictionary dan Dictionary Unpacking (**)

Dictionary adalah tipe data koleksi pada Python yang menyimpan pasangan kunci (*key*) dan nilai (*value*). Berbeda dengan *list* yang diakses menggunakan indeks angka (0, 1, ...), kamus ini diakses menggunakan kunci bertipe string atau tipe data tak-berubah lainnya.

Contoh pendefinisian kamus:

```
1 opsi_judul = {"fontsize": 14, "color": "darkblue"}
```

Untuk menggunakan nilai di dalamnya, kita bisa memanggil kuncinya: `opsi_judul["color"]`.

Dictionary Unpacking ()** adalah operator pembongkaran. Ketika kita memanggil fungsi dan meletakkan tanda bintang ganda (**) sebelum nama variabel *dictionary*, Python akan secara otomatis membongkar seluruh pasangan kunci-nilai tersebut dan meneruskannya sebagai argumen kata kunci (*keyword arguments*) ke dalam fungsi tersebut.

Jadi, penulisan:

```
1 plt.title("Judul", **opsi_judul)
```

Secara instan diterjemahkan oleh interpreter Python menjadi:

```
1 plt.title("Judul", fontsize=14, color="darkblue")
```

Teknik ini sangat berguna untuk membuat templat gaya teks yang bersih, konsisten, dan mudah diperbarui di satu baris tunggal, sehingga Anda tidak perlu mengetik argumen dekoratif berulang-ulang di setiap grafik.

Konsep DPI (Dots Per Inch) untuk Ekspor Grafik

Saat Anda menyusun laporan operasional untuk command center kota, gambar visualisasi tidak cukup hanya tampil di layar komputer. Grafik tersebut harus diekspor sebagai file gambar statis untuk dimasukkan ke laporan akhir (Word/PDF) atau papan pemantauan cetak.

Gunakan fungsi `plt.savefig()` untuk menyimpan grafik. Secara default, resolusi ekspor berkisar pada 72-96 DPI (*Dots Per Inch*), yang akan terlihat buram atau pecah ketika dicetak di atas kertas fisik. Untuk publikasi cetak yang tajam, biasakan mengekspor dengan resolusi minimal **300 DPI**:

```
1 # Menyimpan file grafik dengan ketajaman tinggi untuk cetakan
2 plt.savefig("hubungan_kepadatan_tk.png", dpi=300, bbox_inches="tight")
```

Argumen `bbox_inches="tight"` memastikan agar tidak ada teks label atau legenda di pinggir grafik yang terpotong saat file gambar disimpan.

5.3.g Storytelling: Menuliskan Interpretasi Naratif di Bawah Grafik

Membuat grafik hanyalah separuh tugas dari seorang operator command center kota. Tugas terpenting berikutnya adalah menuliskan cerita di balik data (*storytelling*) dalam bentuk narasi pendek yang diletakkan tepat di bawah grafik. Narasi visual yang baik di command center minimal menjawab tiga pertanyaan dasar: 1. *Pola umum apa yang terlihat secara jelas pada grafik?* 2. *Apakah ada anomali atau titik ekstrem yang menarik perhatian?* 3. *Apa rekomendasi awal atau implikasi kebijakan tata ruang dari temuan visual tersebut?*

Mari kita buat interpretasi singkat dari grafik hubungan kepadatan dan jumlah TK di atas:

Interpretasi Grafik: Grafik sebar di atas menunjukkan bahwa hubungan antara kepadatan penduduk kelurahan dan ketersediaan fasilitas TK di Kota Bandar Lampung tergolong lemah. Kelurahan-kelurahan dengan kepadatan penduduk yang sangat tinggi (di atas 20.000 jiwa/km²) ternyata tidak otomatis memiliki fasilitas TK yang melimpah; sebagian besar menumpuk di bawah 5 unit TK. Hal ini mengindikasikan adanya isu kesenjangan layanan pendidikan anak usia dini. Selain itu, sebaran warna topografi memperlihatkan bahwa wilayah dataran memiliki variasi fasilitas yang lebih luas dibandingkan dengan wilayah lembah/aliran sungai yang cenderung terbatas. Pemerintah Kota disarankan memprioritaskan bantuan penyediaan ruang pendidikan di wilayah-wilayah padat penduduk yang saat ini minim layanan TK.

Tutup sesi kerja basis data Anda:

```
1 koneksi.close()
```

Baris 1

Menutup koneksi ke basis data agar berkas `bd1.sqlite` tidak terkunci.

5.4 Latihan Mandiri

Kerjakan latihan ini di berkas notebook baru bernama `modul/latihan-05.ipynb` menggunakan *kernel analitika-perkotaan*. Pastikan alur kode Anda rapi dan terstruktur, serta jalankan perintah **Restart kernel & Run All** untuk memverifikasi bahwa kode Anda bebas dari galat.

Latihan ini menggunakan dataset kelurahan yang sama, namun dengan fokus analisis fasilitas pendidikan dasar (**SD dan SMP**) serta struktur lapangan usaha warga:

1. **Persiapan Data:** Hubungkan notebook ke basis data `bdl.sqlite` dan tarik seluruh data kelurahan beserta kecamatan induknya ke *DataFrame* menggunakan Pandas.
2. **Variabel Baru:** Buat dua kolom baru untuk menghitung total fasilitas pendidikan dasar:
 - `total_sd = jml_sd_negeri + jml_sd_swasta`
 - `total_smp = jml_smp_negeri + jml_smp_swasta`
3. **Diagram Batang Data Kategorikal:** Hitung frekuensi tiap kategori lapangan usaha utama warga (`lap_usaha_sebagian_warga`) menggunakan `value_counts()` yang sudah Anda kenal di Modul 4, lalu sajikan hasilnya sebagai diagram batang. Urutkan batang dari kategori dengan frekuensi tertinggi.
4. **Scatter Plot:** Buat sebuah grafik sebar (*scatter plot*) untuk melihat hubungan antara jumlah SD (`total_sd`, sumbu X) dengan jumlah SMP (`total_smp`, sumbu Y) per kelurahan.
5. **Box Plot Distribusi:** Visualisasikan distribusi `total_sd` antar-kelurahan pada setiap kecamatan menggunakan *box plot* (`sns.boxplot` dengan sumbu Y berisi `nama_kec`). Amati kecamatan mana yang sebarannya paling lebar dan mana yang memiliki *outlier*.
6. **Estetika Diagram:** Poles diagram batang pada langkah 3 menggunakan opsi parameter kamus/*dictionary*: terapkan judul yang informatif, label sumbu yang bersih, warna palet yang harmonis, dan simpan grafik tersebut menggunakan `plt.savefig()` dengan resolusi 300 DPI.
7. **Interpretasi & Tutup:** Tuliskan satu paragraf interpretasi singkat (*storytelling*) mengenai kesimpulan visualisasi Anda terhadap ketersediaan fasilitas SD dan SMP serta struktur lapangan usaha warga, lalu tutup koneksi basis data Anda.

6 Pengenalan Data Spasial dengan GeoPandas

! Sub-CPMK Pertemuan Ini

Mahasiswa mampu mengoperasikan GeoPandas untuk membaca data spasial (GeoJSON), mengonversi data tabular koordinat menjadi objek spasial titik (GeoDataFrame), mengelola sistem koordinat proyeksi (CRS), dan memvisualisasikan peta tematik interaktif (*choropleth*) menggunakan Leafmap.

6.1 Pendahuluan

Bayangkan Anda kembali ke pengalaman survei lapangan pada mata kuliah studio sebelumnya. Anda berdiri di sudut sebuah kelurahan, memegang GPS, mencatat koordinat patok batas wilayah satu per satu hingga membentuk garis batas administrasi yang utuh ketika dihubungkan. Setiap titik itu hanyalah dua angka — lintang dan bujur — tetapi ketika seluruh titik itu diplot dan disambungkan, ia berubah menjadi sesuatu yang jauh lebih bermakna di mana terdapat batas wilayah yang bisa dianalisis, diukur luasnya, dan ditumpangsusunkan dengan peta tematik lain.

Persoalan yang sering luput disadari adalah, seluruh pekerjaan analisis data perkotaan yang telah kita pelajari sejauh ini — mengolah tabel dengan Pandas, mencari korelasi, membuat grafik dengan Matplotlib dan Seaborn — baru menjawab pertanyaan “berapa banyak” dan “seberapa besar hubungannya”, belum menjawab pertanyaan paling mendasar dalam perencanaan wilayah dan kota yaitu “di mana?” Sebuah kelurahan dengan kepadatan penduduk tertinggi tidak banyak berguna diketahui jika kita tidak bisa menunjukkan di sisi kota sebelah mana ia berada dan bagaimana pola sebarannya dibandingkan kelurahan-kelurahan di sekitarnya — data tanpa dimensi ruang adalah data yang kehilangan separuh maknanya bagi seorang perencana.

Ini alasan mengapa GeoPandas menjadi pustaka yang perlu kita kuasai setelah terampil mengolah data tabular biasa. GeoPandas memperluas kemampuan Pandas dengan menambahkan satu kolom istimewa — kolom geometri, yang menyimpan bentuk spasial suatu objek berupa titik, garis, maupun poligon — sehingga tabel yang tadinya hanya berisi angka dan teks kini bisa “digambar” di atas peta, persis seperti proses digitasi di perangkat lunak GIS.

6.2 Gambaran Umum Kasus

Setiap persoalan perkotaan selalu punya alamat. Kelurahan yang padat itu ada di sebelah mana, sekolah yang menumpuk itu berkumpul di sisi kota yang mana, dan kelurahan yang kosong dari fasilitas itu berbatasan dengan apa — semuanya pertanyaan yang tidak bisa dijawab oleh tabel. Sampai Modul 5, seluruh angka profil kelurahan Bandar Lampung kita perlakukan sebagai baris yang berdiri sendiri, seolah-olah letaknya tidak penting. Mulai modul ini kita kembalikan letaknya. Istilahnya pun sudah ada dan kita kenal akrab: GIS, *geographical information system*.

Sebagai mahasiswa tingkat akhir, Anda pasti sudah pernah mengolah data spasial menggunakan perangkat lunak khusus GIS tersebut. Anda memasukkan *layer* batas kelurahan dan sebaran fasilitas di bidang kerja, memastikan letak keduanya pas, dan mengolah keduanya sehingga menghasilkan informasi yang dibutuhkan. Modul ini mengerjakan hal yang sama persis, hanya saja mejanya berupa Python dan *layer*-nya berupa berkas.

Perhatikan salah satu langkah yang dideskripsikan di atas: memastikan letak keduanya **pas**. Dalam bekerja dengan data spasial, hal itu diurus lewat Sistem Referensi Koordinat — dan cara menyebut sistem koordinat secara tepat lewat nomor katalog EPSG adalah salah satu yang akan kita dalami di sini.

Data yang kita pakai ada dua rupa. Yang pertama batas administrasi kelurahan Kota Bandar Lampung dalam format GeoJSON — inilah poligonnya. Yang kedua data yang sudah akrab: profil kelurahan dari PODES BPS 2021 dan daftar satuan pendidikan dari Kemendikdasmen, keduanya berupa tabel biasa yang menyimpan koordinat sebagai angka di dalam kolom. Pekerjaan kita adalah menghidupkan angka-angka koordinat itu menjadi titik, menempelkan atribut PODES ke poligon yang benar, lalu menutupnya dengan peta tematik kepadatan penduduk yang bisa digeser dan diperbesar.

6.3 Instruksi

1. Buat *notebook* baru `modul/modul-06.ipynb` di direktori proyek Anda.
2. Sambungkan *notebook* ke *kernel* `analitika-perkotaan`.
3. Kerjakan seluruh instruksi di bawah ini secara berurutan.

6.3.a Mengenal Format Vektor GeoJSON

Data spasial vektor menyimpan dua hal sekaligus dalam satu berkas: **bentuk** (geometri) dan **keterangan** (atribut). Bentuknya ada tiga rupa dasar — titik untuk objek yang cukup diwakili satu lokasi (sekolah, tiang listrik), garis untuk objek memanjang (jalan, sungai), dan poligon untuk objek berwilayah (batas kelurahan, blok permukiman). Keterangannya adalah kolom-kolom biasa seperti yang sudah Anda kenal sejak Modul 2.

Yang berbeda adalah wadahnya. Tiga format berikut yang paling sering Anda temui:

Format	Wujud berkas	Ciri yang perlu diingat
Shapefile (<code>.shp</code>)	Sekumpulan berkas: <code>.shp</code> , <code>.shx</code> , <code>.dbf</code> , <code>.prj</code> , dan kerap beberapa lagi	Format lama yang paling luas didukung. Semua berkasnya wajib bepergian bersama dan sepakat namanya
GeoJSON (<code>.geojson</code>)	Satu berkas teks	Terbaca manusia, ramah web, mudah dikirim lewat surel atau diunggah. Ukurannya membengkak untuk data yang sangat besar
GeoPackage (<code>.gpkg</code>)	Satu berkas basis data	Format modern; satu berkas sanggup memuat banyak <i>layer</i> sekaligus. Makin sering dipakai lembaga resmi seperti BIG

Modul ini memakai GeoJSON. Karena isinya teks biasa, strukturnya bisa kita bedah langsung. Sebuah berkas GeoJSON selalu berupa satu `FeatureCollection` yang membungkus daftar `features`, dan setiap *feature* membawa pasangan `geometry` dengan `properties`:

```
1 {  
2   "type": "FeatureCollection",
```

```

3     "features": [
4     {
5         "type": "Feature",
6         "geometry": {
7             "type": "MultiPolygon",
8             "coordinates": [ [ [ [105.2683, -5.3971], [105.2691, -5.3968], ... ] ] ]
9         },
10        "properties": {
11            "kode_permendagri": "1871011003",
12            "nama_des": "Kedaton",
13            "nama_kec": "Kedaton",
14            "luas": 109.39784638771118
15        }
16    }
17 ]
18 }

```

Perhatikan dua hal. Pertama, `properties` isinya persis seperti satu baris tabel — nanti ia akan menjadi kolom-kolom biasa. Kedua, `coordinates` berisi pasangan angka, dan **angka pertama adalah bujur, angka kedua lintang**. Urutan ini akan kembali muncul beberapa kali dan sudah menjatuhkan banyak orang; kita akan membahasnya lagi nanti.

i Pengetahuan Komputer

Shapefile Itu Sekumpulan Berkas, Bukan Satu

Ini sumber kekecewaan yang berulang setiap tahun di studio: seseorang mengirimkan `batas_kelurahan.shp` lewat WhatsApp, penerimanya membukanya, dan perangkat lunaknya menolak atau membuka peta tanpa nama wilayah sama sekali.

Penyebabnya sederhana. Shapefile bukan satu berkas, melainkan satu keluarga berkas yang menanggung tugas berbeda-beda:

- `.shp` — bentuk geometrinya saja, tanpa nama dan tanpa angka
- `.shx` — indeks yang memberitahu di posisi mana tiap bentuk berada
- `.dbf` — seluruh atributnya (nama kelurahan, luas, kode wilayah)
- `.prj` — keterangan sistem koordinat yang dipakai

Tanpa `.dbf`, peta terbuka tetapi kosong keterangan. Tanpa `.prj`, perangkat lunak tidak tahu bagaimana menempatkan bentuknya di atas bumi. Karena itu, cara mengirim Shapefile yang benar adalah **memampatkan seluruh isi folder-nya menjadi satu .zip**, bukan memilih satu berkas.

Inilah salah satu alasan GeoJSON dan GeoPackage makin disukai: satu berkas, satu kali kirim, tidak ada yang bisa tertinggal.

6.3.b Membaca Berkas GeoJSON Batas Kelurahan dengan GeoPandas

Perkakas kita adalah **GeoPandas**. Namanya bukan kebetulan: GeoPandas dibangun tepat di atas Pandas yang sudah Anda pakai sejak Modul 2. Segala yang Anda kuasai di sana — `.head()`, `.info()`, memilih

kolom, menyaring baris dengan `&` dan `|` — tetap berlaku apa adanya. Yang ditambahkan hanya satu: sebuah kolom istimewa bernama `geometry` yang menyimpan bentuk.

Impor pustakanya, simpan alamat berkas ke dalam variabel, lalu bacalah berkas batas kelurahan:

```
1 import geopandas as gpd
2
3 path_batas = "../dataset-ganjil-2026-2027/batas_kelurahan_bdl.geojson"
4 gdf_kelurahan = gpd.read_file(path_batas)
5 gdf_kelurahan.head()
```

Baris 1

GeoPandas dipanggil dengan nama pendek `gpd`, mengikuti kebiasaan yang sama seperti `pd` untuk Pandas di Modul 2.

Baris 3

Alamat berkas disimpan ke dalam variabel supaya baris berikutnya tetap pendek dan alamatnya cukup diketik sekali. Tanda `..` di depan berarti “naik satu tingkat dari *folder modul*”, persis seperti yang sudah Anda pakai di Modul 2 sampai 5.

Baris 4

`read_file()` membaca berkas GeoJSON dan mengubahnya menjadi **GeoDataFrame**. Fungsi ini mengenali sendiri format berkasnya dari isinya, jadi perintah yang sama juga sanggup membaca Shapefile dan GeoPackage.

Baris 5

`.head()` menampilkan lima baris pertama, persis seperti pada DataFrame biasa.

```
kode_permendagri    nama_des nama_kec    luas \
0      1871011003      Kedaton  Kedaton  109.397846
1      1871011004    Surabaya  Kedaton   62.607745
2      1871011005  Sukamenanti  Kedaton   54.288118
3      1871011006    Sidodadi  Kedaton   74.003176
4      1871011009  Sukamenanti Baru  Kedaton   27.424105

                                geometry
0  MULTIPOLYGON Z (((105.26504 -5.38006 0, 105.26...
1  MULTIPOLYGON Z (((105.26516 -5.38534 0, 105.26...
2  MULTIPOLYGON Z (((105.25597 -5.3876 0, 105.256...
3  MULTIPOLYGON Z (((105.26128 -5.38641 0, 105.26...
4  MULTIPOLYGON Z (((105.25709 -5.39134 0, 105.25...
```

Kolom `kode_permendagri`, `nama_des`, `nama_kec`, dan `luas` adalah isi `properties` yang tadi kita lihat di dalam berkas mentahnya. Kolom `geometry` di ujung kanan adalah isi `coordinates`-nya, sudah dirangkum menjadi bentuk yang bisa digambar.

 **Penting!**

Kolom `geometry` Bukan Kolom Biasa

Sekilas `geometry` tampak seperti kolom teks yang isinya panjang dan aneh. Ia bukan teks. Isinya objek geometri, dan objek inilah yang membuat `GeoDataFrame` sanggup melakukan hal-hal yang tidak bisa dilakukan `DataFrame`: menggambar peta, menghitung luas, mengukur jarak, dan memeriksa apakah sebuah titik berada di dalam sebuah poligon.

Konsekuensi praktisnya: **jangan pernah menghapus kolom `geometry`** dengan `drop(columns=...)` kalau Anda masih membutuhkan bentuknya. Begitu kolom itu hilang, `GeoDataFrame` berubah menjadi `DataFrame` biasa dan seluruh kemampuan spasialnya ikut hilang tanpa pesan galat apa pun.

Sekarang periksa isinya secara ringkas, seperti kebiasaan Anda sejak Modul 2 dan Modul 4:

```
1 print(gdf_kelurahan.shape)
2 print(gdf_kelurahan.geometry.geom_type.value_counts())
3 print(gdf_kelurahan.crs)
```

Baris 1

Ada berapa baris dan berapa kolom. Angka barisnya harus 126 — sebanyak kelurahan di Kota Bandar Lampung, sama persis dengan jumlah baris data PODES yang Anda olah sejak Modul 2.

Baris 2

Rupa geometri tiap barisnya. Hasilnya `MultiPolygon`, artinya satu kelurahan boleh terdiri atas lebih dari satu bidang terpisah — hal yang lumrah untuk wilayah yang terpotong sungai atau punya kantong wilayah.

Baris 3

Sistem koordinat yang sedang dipakai berkas ini. Untuk sementara cukup catat jawabannya; kita bedah tuntas di bagian penyalarsan CRS nanti.

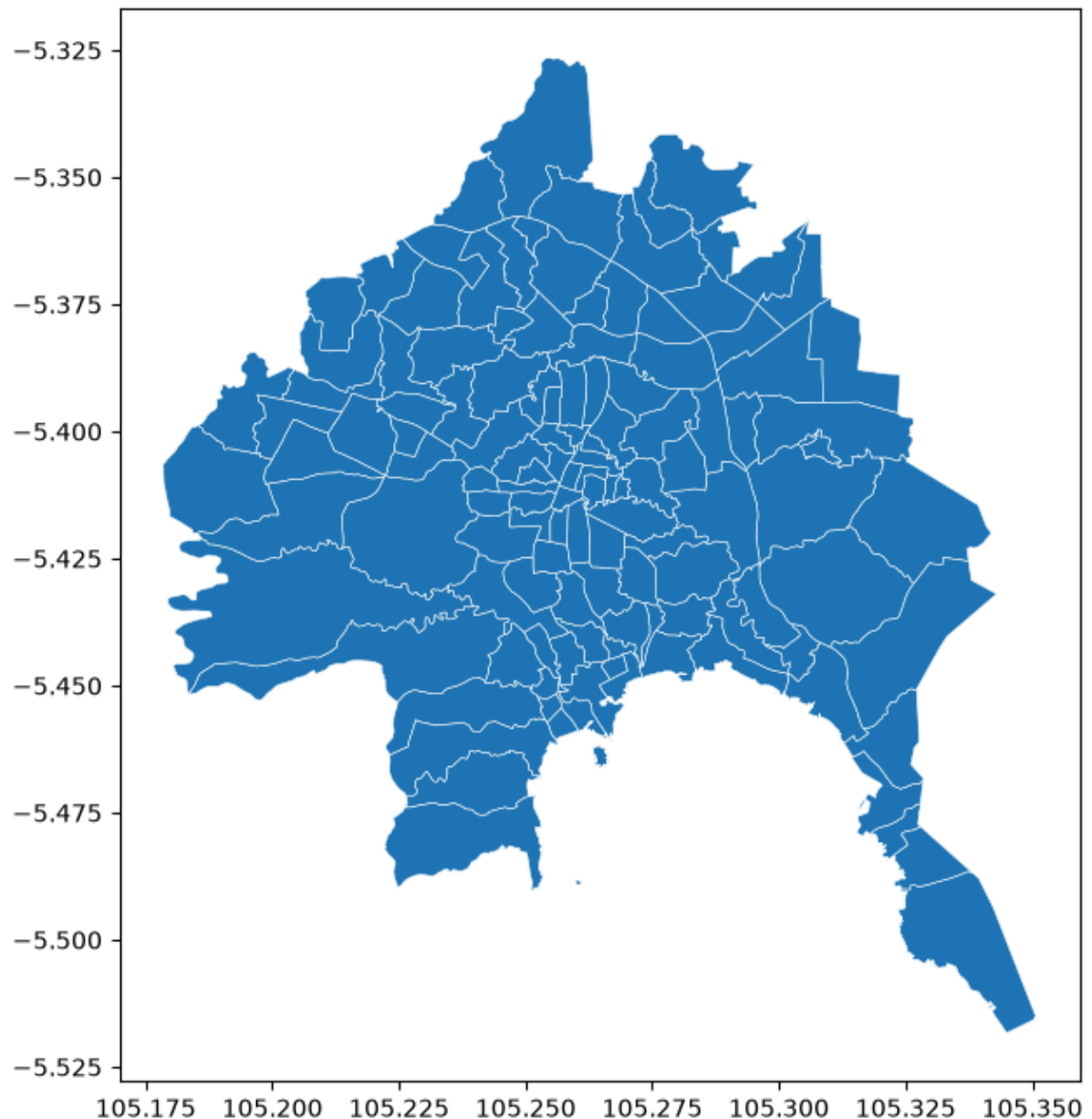
```
(126, 5)
MultiPolygon    126
Name: count, dtype: int64
EPSG:4326
```

Menggambar petanya cukup satu perintah. Inilah lembar kalkir pertama Anda:

```
1 ax = gdf_kelurahan.plot(figsize=(8, 8), edgecolor="white", linewidth=0.4)
```

Baris 1

`.plot()` adalah milik `GeoDataFrame` dan langsung menggambar seluruh geometrinya. Parameter `figsize` mengatur ukuran kanvas dalam inci, parameter `edgecolor` mewarnai garis batas antar-kelurahan, dan parameter `linewidth` menipiskannya agar 126 poligon tidak tenggelam oleh garis. Di balik layar, yang bekerja adalah Matplotlib yang sudah Anda kenal di Modul 5. Hasilnya ditampung ke variabel `ax` semata-mata agar Jupyter tidak ikut mencetak keterangan teknis objeknya di atas gambar; tanpa penampungan itu akan muncul baris `<Axes: >` yang tidak berguna.



Bentuk Kota Bandar Lampung sudah muncul, tetapi seluruhnya berwarna sama. Peta ini belum menceritakan apa pun karena atributnya belum kita lekatkan — pekerjaan yang akan kita tuntaskan pada dua bagian berikutnya.

6.3.c Memplot Titik Koordinat: Mengonversi Data Tabular Menjadi Geometri Spasial

Poligon batas kelurahan tadi datang sudah berbentuk. Yang lebih sering Anda hadapi di dunia nyata justru kebalikannya: tabel biasa yang menyimpan koordinat sebagai dua kolom angka. Data PODES adalah contohnya, begitu pula daftar satuan pendidikan. Bagian ini mengubah angka-angka itu menjadi titik yang bisa digambar.

Mulai dari data PODES. Bacalah seperti biasa dengan Pandas, lalu periksa lebih dulu kolom koordinatnya:

```

1 import pandas as pd
2
3 path_podes = "../dataset-ganjil-2026-2027/bdl_kelurahan.csv"
4 df = pd.read_csv(path_podes, dtype={"kode_permendagri": str})
5
6 print(df[["nama_des", "lat", "lon", "north_south"]].head())
7 print(df["north_south"].value_counts())

```

Baris 1

Pandas tetap dipakai untuk membaca tabel biasa. GeoPandas baru masuk setelah koordinatnya diubah menjadi bentuk.

Baris 3

Alamat berkas kembali disimpan ke variabel, mengikuti kebiasaan yang sama.

Baris 4

`dtype={"kode_permendagri": str}` memaksa kolom kode wilayah dibaca sebagai **teks**. Tanpa ini Pandas akan menebaknya sebagai angka, dan penggabungan data di bagian nanti akan gagal diam-diam. Alasan lengkapnya ada pada kotak di bagian penggabungan.

Baris 6

Empat kolom yang menyangkut lokasi ditampilkan bersama supaya mudah dibandingkan.

Baris 7

`value_counts()` dari Modul 4 dipakai untuk memeriksa isi kolom `north_south`. Seluruh 126 baris bernilai 2.

```

      nama_des      lat      lon  north_south
0      Bakung  5.454921  105.248314           2
1      Kuripan  5.448879  105.254387           2
2  Negeri Olok Gading  5.445735  105.246566           2
3      Sukarame Ii  5.440540  105.241166           2
4      Batu Putuk  5.436184  105.223633           2
north_south
2      126
Name: count, dtype: int64

```

Perhatikan baik-baik hasilnya. Seluruh nilai `lat` **positif**, padahal Kota Bandar Lampung jelas berada di belahan bumi selatan. Kuncinya ada di kolom `north_south` yang bernilai 2 untuk semua baris: BPS menyimpan besaran lintang dan arah belahan buminya secara **terpisah**, dengan 1 berarti Lintang Utara dan 2 berarti Lintang Selatan.

Sebelum koordinat ini dipakai, arah itu harus kita satukan kembali ke dalam angkanya:

```

1 df["lat_wgs84"] = -df["lat"]
2
3 print(df[["nama_des", "lat", "lat_wgs84", "lon"]].head(3))

```

Baris 1

Seluruh nilai lintang dibalik tandanya menjadi negatif, karena kita sudah memastikan pada langkah sebelumnya bahwa semua barisnya Lintang Selatan. Hasilnya disimpan sebagai kolom baru `lat_wgs84`, bukan menimpa kolom `lat` — supaya angka aslinya tetap ada dan bisa diperiksa ulang bila ada yang mencurigakan.

Baris 3

Ketiga kolom disandingkan untuk memastikan pembalikannya berjalan.

	nama_des	lat	lat_wgs84	lon
0	Bakung	5.454921	-5.454921	105.248314
1	Kuripan	5.448879	-5.448879	105.254387
2	Negeri Olok Gading	5.445735	-5.445735	105.246566

⚠ Penting!

Satu Tanda Minus yang Memindahkan Kota

Kalau langkah pembalikan tanda ini terlewat, tidak akan ada pesan galat sama sekali. Kode tetap berjalan, peta tetap terbit, titik tetap muncul — hanya saja seluruh 126 kelurahan Bandar Lampung mendarat di sekitar perbatasan Malaysia dan Thailand, sekitar 1.200 kilometer di sebelah utara tempat seharusnya.

Inilah jenis kesalahan yang paling berbahaya dalam pekerjaan spasial: yang tidak berteriak. Karena itu biasakan satu hal sebelum mempercayai koordinat dari sumber mana pun — **lihat dulu angkanya**. Untuk Indonesia, bujur selalu berada di rentang 95 sampai 141, dan lintang di rentang -11 sampai 6. Angka lintang positif besar untuk wilayah di selatan khatulistiwa adalah tanda bahaya, bukan kebetulan.

Sekarang koordinatnya siap diubah menjadi bentuk. Kita kerjakan bertahap supaya terlihat apa yang sebenarnya terjadi. Bentuk titik disediakan oleh **Shapely**, pustaka geometri yang bekerja di balik layar GeoPandas:

```
1  from shapely.geometry import Point
2
3  pusat_kota = (105.2683, -5.3971)
4  titik_pusat = Point(pusat_kota)
5  print(titik_pusat)
6
7  daftar_koordinat = [(df["lon"][i], df["lat_wgs84"][i]) for i in range(len(df))]
8  daftar_titik = [Point(koordinat) for koordinat in daftar_koordinat]
9
10 print("Bujur pertama :", daftar_koordinat[0][0])
11 print("Lintang pertama :", daftar_koordinat[0][1])
12 print("Titik pertama :", daftar_titik[0])
```

Baris 1

`Point` diambil dari Shapely. Cara impor `from ... import ...` ini mengambil satu nama saja dari sebuah pustaka, berbeda dengan `import pandas as pd` yang mengambil seluruh isinya.

Baris 3

Satu pasangan koordinat ditulis di dalam kurung biasa. Bentuk seperti ini disebut **tuple**, dan penjelasannya ada di kotak di bawah. Urutannya **bujur dulu, baru lintang**.

Baris 4

Tuple tadi diserahkan ke `Point` dan berubah menjadi satu titik geometri — sejenis dengan isi kolom `geometry` yang Anda lihat pada poligon kelurahan.

Baris 5

Dicetak untuk melihat wujudnya.

Baris 7

Bentuk ringkas yang disebut **list comprehension** membangun 126 pasangan koordinat sekaligus. `range(len(df))` menyusuri nomor baris satu per satu — perulangan yang sama seperti Modul 4, hanya ditulis padat.

Baris 8

Daftar pasangan koordinat tadi diubah menjadi daftar titik, sekali lagi dalam satu baris.

Baris 10-11

Kedua isi *tuple* pertama dicetak terpisah. Nomor urut `[0]` mengambil anggota pertama dan `[1]` anggota kedua, sama seperti cara Anda mengambil isi *list* di Modul 2.

Baris 12

Titik pertama dicetak untuk memastikan angkanya sejalan dengan *tuple* asalnya.

```
POINT (105.2683 -5.3971)
Bujur pertama : 105.24831377392
Lintang pertama : -5.4549208780046
Titik pertama : POINT (105.24831377392 -5.4549208780046)
```

Konsep Python

Tuple: Pasangan Nilai yang Tidak Berubah

Anda sudah mengenal **List** sejak Modul 2 sebagai wadah berisi banyak nilai. **Tuple** adalah wadah serupa dengan dua perbedaan penting:

	List	Tuple
Ditulis dengan	Kurung siku <code>["a", "b"]</code>	Kurung biasa <code>("a", "b")</code>
Isinya bisa diubah?	Bisa	Tidak bisa
Cocok untuk	Kumpulan yang bisa bertambah dan berkurang	Kesatuan yang jumlah dan urutan anggotanya tetap

```
1 kolom_pilihan = ["nama_des", "kepadatan_pddk"] # list - boleh ditambah
2 pusat_kota     = (105.2683, -5.3971)           # tuple - tidak boleh diubah
```

Mengapa koordinat pantas menjadi tuple? Karena sebuah koordinat selalu terdiri atas **tepat dua angka dengan urutan yang bermakna**. Menambah anggota ketiga tidak masuk akal, dan menukar urutannya mengubah artinya — pasangan `(105.2683, -5.3971)` berada di Bandar Lampung,

sedangkan `(-5.3971, 105.2683)` entah di mana. Sifat *tuple* yang tidak bisa diubah adalah pengaman: ia menolak dibongkar setelah dibentuk.

Mengambil isinya sama seperti pada *list*, memakai nomor urut yang dimulai dari nol:

```
1 print(pusat_kota[0]) # 105.2683 -> bujur
2 print(pusat_kota[1]) # -5.3971 -> lintang
```

Konsep Python

List Comprehension

Di Modul 4 Anda memakai perulangan `for` untuk mengerjakan sesuatu berulang kali. Sering kali yang kita inginkan dari perulangan itu cuma satu: **sebuah daftar baru**. Untuk keperluan itu Python menyediakan bentuk ringkasnya.

Keduanya di bawah ini mengerjakan hal yang persis sama:

```
1 # Bentuk panjang - perulangan for seperti Modul 4
2 daftar_titik = []
3 for koordinat in daftar_koordinat:
4     daftar_titik.append(Point(koordinat))
5
6 # Bentuk ringkas - list comprehension
7 daftar_titik = [Point(koordinat) for koordinat in daftar_koordinat]
```

Cara membacanya dari tengah ke luar: mulai dari `for koordinat in daftar_koordinat` — “untuk setiap koordinat di dalam daftar koordinat” — lalu ke bagian depan `Point(koordinat)` — “buatkan sebuah titik darinya” — dan kurung siku yang membungkusnya menegaskan bahwa hasil akhirnya sebuah *list*.

Polanya selalu tiga bagian:

```
[ apa yang dihasilkan for tiap anggota in dari mana ]
```

Bentuk ringkas ini dipakai ketika perulangannya sederhana dan tujuannya memang menyusun daftar. Bila di dalamnya perlu percabangan bertingkat atau beberapa langkah berbeda, tulislah tetap dengan `for` biasa — kejelasan lebih berharga daripada kependekan.

Daftar titik itu kini bisa dipasangkan ke tabel PODES sehingga menjadi `GeoDataFrame` utuh:

```
1 crs_geografis = "EPSG:4326"
2
3 gdf_podes = gpd.GeoDataFrame(df, geometry=daftar_titik, crs=crs_geografis)
4 gdf_podes[["nama_kec", "nama_des", "kepadatan_pddk", "geometry"]].head()
```

Baris 1

Kode sistem koordinat disimpan ke variabel supaya bisa dipakai berulang tanpa salah ketik. Arti kodenya dibahas tuntas di bagian berikutnya.

Baris 3

`gpd.GeoDataFrame()` menggabungkan tiga hal: tabel PODES apa adanya, daftar titik sebagai kolom `geometry`, dan keterangan sistem koordinatnya. Tanpa parameter `crs=`, GeoPandas tidak tahu angka-angka itu harus ditafsirkan sebagai apa.

Baris 4

Beberapa kolom penting ditampilkan bersama kolom `geometry` yang baru terbentuk.

```
      nama_kec      nama_des  kepadatan_pddk \
0  Teluk Betung Barat      Bakung          6479
1  Teluk Betung Barat      Kuripan         16321
2  Teluk Betung Barat  Negeri Olok Gading        7761
3  Teluk Betung Barat      Sukarame Ii         1514
4  Teluk Betung Barat      Batu Putuk         2214

      geometry
0  POINT (105.24831 -5.45492)
1  POINT (105.25439 -5.44888)
2  POINT (105.24657 -5.44574)
3  POINT (105.24117 -5.44054)
4  POINT (105.22363 -5.43618)
```

Penting!

`points_from_xy`: X Dulu, Baru Y

Tiga langkah tadi — menyusun *tuple*, mengubahnya menjadi `Point`, lalu memasangnya — sengaja ditulis panjang supaya terlihat apa yang terjadi. Dalam pekerjaan sehari-hari, GeoPandas menyediakan satu perintah yang mengerjakan ketiganya sekaligus:

```
1  gdf_podes = gpd.GeoDataFrame(
2      df,
3      geometry=gpd.points_from_xy(df["lon"], df["lat_wgs84"]),
4      crs=crs_geografis,
5  )
```

Perhatikan urutan argumennya: **bujur lebih dulu, lintang belakangan**. Ini bukan kesengajaan yang aneh-aneh. Dalam matematika, sumbu mendatar selalu disebut lebih dulu sebagai koordinat-x, dan bujur yang mendatar. Yang membuatnya terasa terbalik adalah kebiasaan lisan kita sendiri: kita terbiasa mengucap “lintang, bujur”, dan aplikasi peta di ponsel pun menampilkannya dengan urutan itu.

Akibat tertukarnya tidak selalu berupa galat. Untuk Bandar Lampung, menukar `(105.27, -5.40)` menjadi `(-5.40, 105.27)` menghasilkan titik yang tetap sah secara teknis — hanya saja ia mendarat di Samudra Arktik. Sekali lagi: kesalahan spasial jarang berteriak. Setelah membuat titik, **selalu gambar petanya lebih dulu** sebelum melanjutkan.

Kini giliran data satuan pendidikan. Berbeda dengan PODES yang lengkap, data ini punya lubang — sebagian sekolah terdaftar di Dapodik tanpa titik lokasi. Periksa dulu seberapa besar lubangnya dengan cara yang sudah Anda pakai di Modul 4:

```

1 path_sekolah = "../dataset-ganjil-2026-2027/sekolah_bdl.csv"
2 df_sekolah = pd.read_csv(path_sekolah)
3
4 print("Jumlah baris      :", len(df_sekolah))
5 print(df_sekolah[["lat", "lon"]].isna().sum())
6 print(df_sekolah[["Nama Satuan Pendidikan", "Bentuk", "lat", "lon"]].head(3))

```

Baris 1

Alamat berkas satuan pendidikan.

Baris 2

Dibaca biasa dengan Pandas. Kolom kode wilayah tidak ada di berkas ini, jadi parameter `dtype` tidak diperlukan.

Baris 4

Seluruh sekolah yang terdaftar di Kota Bandar Lampung.

Baris 5

`isna().sum()` dari Modul 4 menghitung berapa baris yang kolom koordinatnya kosong.

Baris 6

Beberapa baris pertama ditampilkan. Perhatikan nilai `lat`-nya — sudah bertanda **negatif** apa adanya.

```

Jumlah baris      : 1143
lat      304
lon      304
dtype: int64
   Nama Satuan Pendidikan Bentuk      lat      lon
0      TK IT DAARUL FIKRI      TK      NaN      NaN
1           SPS KENANGA      SPS -5.349419  105.227866
2      TK SMART KIDS      TK -5.400995  105.245682

```

Dua hal yang perlu Anda catat dari hasil di atas. Pertama, ada ratusan sekolah tanpa koordinat; sebagian besar TK dan Kelompok Bermain ber-NPSN baru yang titiknya belum diisikan ke Dapodik. Kedua, dan ini yang penting: kolom `lat` di berkas ini **sudah bertanda negatif**, tidak seperti data PODES yang harus kita balik lebih dulu.

⚠ Penting!

Dua Sumber, Dua Kebiasaan

PODES menyimpan lintang sebagai angka positif dengan kolom arah terpisah. Kemendikdasmen menyimpannya sebagai angka bertanda apa adanya. Keduanya sah, keduanya lazim, dan tidak ada yang salah — yang keliru hanyalah menganggap semua sumber data pasti berperilaku sama.

Ini bukan perkara tanda minus semata. Sumber data yang berbeda juga berbeda dalam hal penulisan kode wilayah, satuan luas, tahun acuan, dan cara menandai nilai kosong. Karena itu, langkah pertama setiap kali menerima berkas baru bukanlah mengolahnya, melainkan **memeriksa**nya — persis seperti yang barusan kita lakukan dengan `head()`, `isna().sum()`, dan `value_counts()`.

Baris tanpa koordinat tidak bisa dijadikan titik, jadi buang dulu, baru ubah menjadi GeoDataFrame:

```
1 df_sekolah = df_sekolah.dropna(subset=["lat", "lon"])
2 print("Sekolah berkoordinat :", len(df_sekolah))
3
4 gdf_sekolah = gpd.GeoDataFrame(
5     df_sekolah,
6     geometry=gpd.points_from_xy(df_sekolah["lon"], df_sekolah["lat"]),
7     crs=crs_geografis,
8 )
9 ax = gdf_sekolah.plot(figsize=(7, 7), markersize=4, color="tab:red")
```

Baris 1

`dropna(subset=[...])` dari Modul 4 membuang baris yang kolom `lat` atau `lon`-nya kosong. Parameter `subset=` membatasi pemeriksaan hanya pada kedua kolom itu — tanpanya, baris yang kolom lain mana pun kosong ikut terbuang.

Baris 2

Sisa barisnya dihitung ulang. Bandingkan dengan angka sebelum pembuangan.

Baris 4

GeoDataFrame dirakit, kali ini memakai jalan pintas yang tadi diperkenalkan di kotak catatan.

Baris 6

`points_from_xy` menerima kolom bujur lebih dulu, lalu kolom lintang. Perhatikan yang dipakai adalah `lat` apa adanya, **tanpa** pembalikan tanda — karena data ini memang sudah negatif.

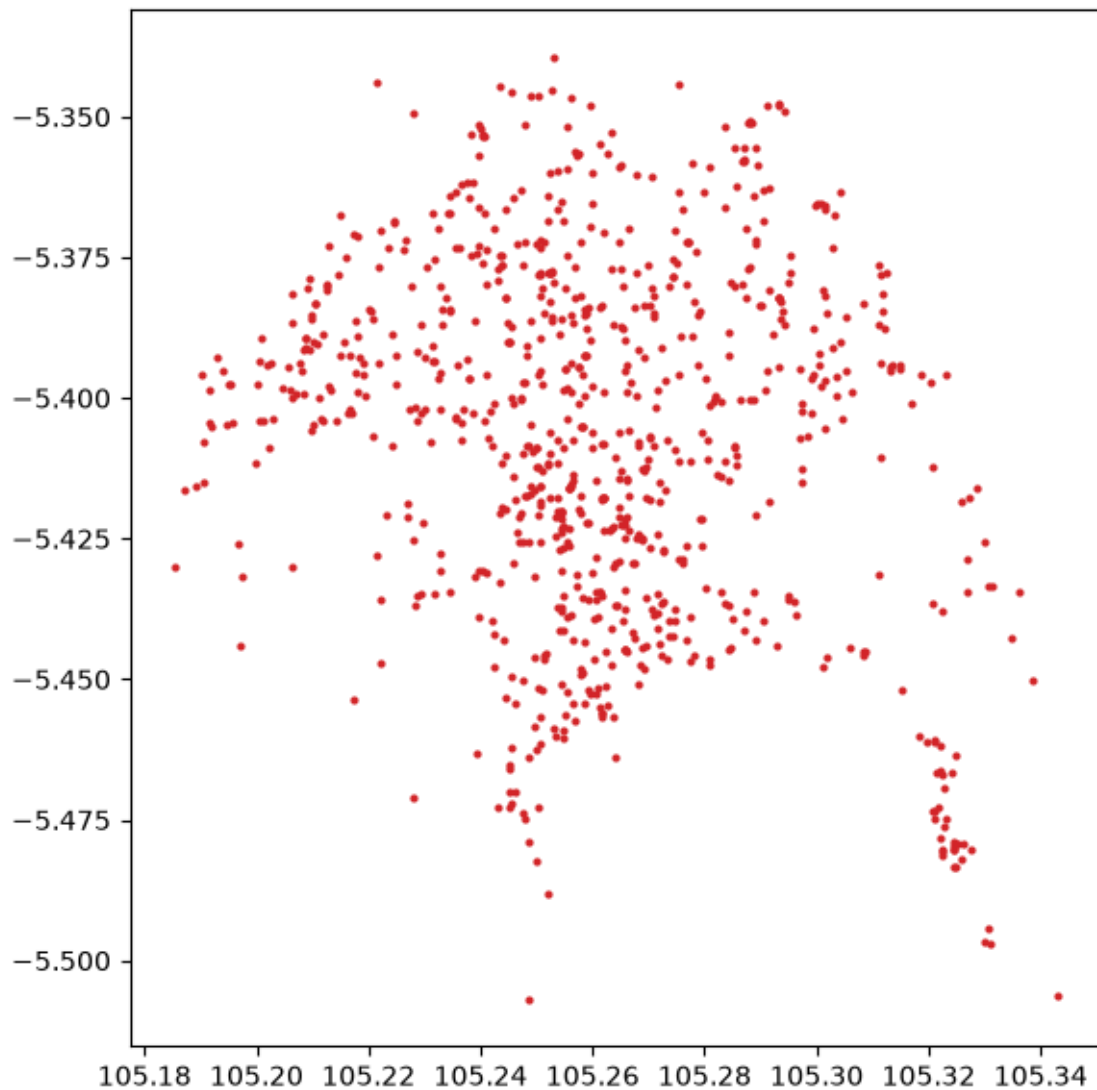
Baris 7

Sistem koordinatnya sama dengan data PODES, memakai variabel yang sama.

Baris 9

Sebarannya langsung digambar sebagai pemeriksaan. Parameter `markersize` mengecilkan titik agar ratusan sekolah tidak menyatu menjadi gumpalan.

```
Sekolah berkoordinat : 839
```



Bentuk sebaran yang muncul seharusnya sudah menyerupai siluet Kota Bandar Lampung yang Anda gambar di bagian sebelumnya. Kalau bentuknya melenceng jauh atau titiknya berhamburan, itulah tanda urutan bujur-lintangnya tertukar.

6.3.d Penyelarasan CRS: Memastikan Data Spasial Menempel pada Proyeksi yang Sama

Anda sudah mengenal persoalan ini sejak semester tiga, jadi cukup satu pengingat singkat. Permukaan bumi melengkung, sedangkan peta datar; setiap cara memindahkan yang satu ke yang lain punya konsekuensi. Dua keluarga besarnya:

	Sistem koordinat geografis	Sistem koordinat terproyeksi
Satuannya	Derajat lintang dan bujur	Meter atau kaki
Gunanya	Menyimpan dan menggambar lokasi	Mengukur luas dan jarak

Yang menjadi persoalan sehari-hari bukan teorinya, melainkan cara menyebut sistem koordinat secara tepat. Di sinilah **EPSG** masuk. Dua kode berikut yang akan kita pakai sepanjang modul:

```

1 crs_geografis = "EPSG:4326"
2 crs_proyeksi = "EPSG:32748"
3
4 print(gdf_kelurahan.crs)
5 print(gdf_podes.crs)
6 print(gdf_sekolah.crs)

```

Baris 1

Sistem yang dipakai berkas GeoJSON dan koordinat PODES kita, satuannya derajat. Inilah yang sudah kita pakai sejak tadi.

Baris 2

Sistem yang akan kita tuju ketika hendak mengukur, satuannya meter.

Baris 4-6

Sistem koordinat ketiga lapis data diperiksa satu per satu. Ketiganya menjawab `EPSG:4326` — artinya ketiga lembar kalkir kita memang **sudah selaras** dan boleh ditumpuk. Pemeriksaan sesederhana ini wajib dilakukan setiap kali Anda menggabungkan data dari sumber yang berbeda.

```

EPSG:4326
EPSG:4326
EPSG:4326

```

Sampai di sini, “EPSG:4326” dan “EPSG:32748” masih berupa mantra yang Anda salin tanpa tahu asalnya. Mari kita bongkar, karena kemampuan menemukan dan memastikan kode EPSG yang benar adalah salah satu keterampilan spasial yang paling sering dipakai dan paling jarang diajarkan.

Mulai dari yang paling dekat: keterangan sistem koordinat sudah melekat pada data yang Anda pegang, dan bisa ditanyakan langsung.

```

1 print(gdf_kelurahan.crs.to_epsg())
2 print(gdf_kelurahan.crs.name)
3 print(gdf_kelurahan.crs.is_projected)
4 print(gdf_kelurahan.crs.axis_info[0].unit_name)

```

Baris 1

Nomor EPSG-nya saja, tanpa awalan, sebagai angka.

Baris 2

Nama resmi sistem koordinatnya.

Baris 3

Apakah ia sistem terproyeksi. Jawaban `False` berarti geografis — dan itu peringatan bahwa data ini belum boleh dipakai untuk mengukur.

Baris 4

Satuan sumbunya. Terbaca `degree`, sejalan dengan jawaban sebelumnya.

```

4326
WGS 84
False
degree

```

Pertanyaan berikutnya lebih menentukan: kalau Anda perlu mengukur, kode EPSG mana yang tepat untuk wilayah kerja Anda? GeoPandas sanggup menebaknya sendiri dari letak datanya.

```
1 print(gdf_kelurahan.estimate_utm_crs())
```

Baris 1

`estimate_utm_crs()` melihat di mana data Anda berada di permukaan bumi, lalu memilihkan zona UTM yang tepat untuknya. Jawabannya `EPSG:32748` — itulah nilai yang tadi kita simpan ke variabel `crs_proyeksi`.

```
EPSG:32748
```

Jawaban dari mesin sebaiknya jangan langsung dipercaya. Untunglah setiap kode EPSG membawa keterangan wilayah berlakunya sendiri, dan keterangan itu bisa kita minta:

```
1 from pyproj import CRS
2
3 crs_periksa = CRS.from_epsg(32748)
4 print(crs_periksa.name)
5 print(crs_periksa.axis_info[0].unit_name)
6 print(crs_periksa.area_of_use.name)
```

Baris 1

`pyproj` adalah pustaka yang mengurus sistem koordinat di balik layar GeoPandas. Ia ikut terpasang bersama GeoPandas, jadi tidak perlu dipasang tersendiri.

Baris 3

Kode EPSG yang hendak diperiksa dibuka dari katalognya.

Baris 4

Nama resminya: `WGS 84 / UTM zone 48S`.

Baris 5

Satuannya `metre` — inilah yang membuatnya layak dipakai mengukur.

Baris 6

Wilayah berlakunya. Keteranganannya menyebut rentang **102° BT sampai 108° BT di belahan bumi selatan, Indonesia**. Kota Bandar Lampung berada di sekitar 105,3° BT — jelas di dalam rentang itu. Kode ini benar untuk wilayah kerja kita.

```
WGS 84 / UTM zone 48S
```

```
metre
```

```
Between 102°E and 108°E, southern hemisphere between 80°S and equator, onshore and offshore. Indonesia.
```

 Pengetahuan Komputer

EPSG: Nomor Katalog Sistem Koordinat

Sebuah sistem koordinat sesungguhnya adalah kumpulan parameter yang panjang: bentuk elipsoid acuannya, titik nolnya, cara proyeksinya, faktor skalanya, dan seterusnya. Menuliskan semuanya setiap kali jelas tidak praktis, dan sangat mudah salah.

Karena itu seluruh sistem koordinat yang pernah dipakai orang dikumpulkan ke dalam satu katalog bernomor. Katalog itu dikelola *International Association of Oil & Gas Producers* — dahulu bernama *European Petroleum Survey Group*, dan dari sanalah singkatan **EPSG** berasal. Menyebut “EPSG:32748” sama halnya menyebut nomor katalog buku di perpustakaan: pendek, tunggal, tidak mungkin salah tafsir, dan menggantikan satu paragraf penuh parameter.

Tiga cara mengetahui kode yang Anda butuhkan:

1. **Tanyakan pada datanya.** `gdf.crs.to_epsg()` memberi tahu kode yang sedang dipakai berkas yang Anda pegang.
2. **Minta ditebakkan.** `gdf.estimate_utm_crs()` memilihkan zona UTM yang sesuai dengan letak data Anda.
3. **Cari lewat peramban.** Situs epsg.io memuat seluruh katalognya. Ketik nama wilayah atau nomornya, dan halamannya menampilkan nama resmi, satuan, serta cakupan wilayah berlakunya.

Cara memastikan kode itu benar adalah dengan memeriksa `area_of_use`-nya seperti yang barusan kita lakukan, lalu mencocokkannya dengan letak wilayah kerja Anda.

Membaca nomornya. Khusus keluarga UTM, penomorannya punya pola yang bisa Anda hafal:

- 326xx — UTM belahan bumi **utara**, zona xx
- 327xx — UTM belahan bumi **selatan**, zona xx

Nomor zonanya dihitung dari bujur: bagi dua angka (`bujur + 180`) dengan 6, bulatkan ke bawah, lalu tambah satu. Untuk Bandar Lampung di 105,3° BT: $(105,3 + 180) \div 6 = 47,55$, dibulatkan ke bawah menjadi 47, ditambah satu menjadi **48**. Digabungkan dengan belahan selatan: $327 + 48 =$ **32748**. Persis angka yang ditebak GeoPandas tadi.

Untuk wilayah Indonesia lainnya, polanya sama: Sumatra bagian utara masuk zona 47 (32747), Jawa bagian timur dan Bali zona 49 (32749), dan Papua zona 53 sampai 54.

Sekarang kita buktikan mengapa semua ini penting. Poligon kelurahan kita membawa kolom `luas` bawaan dari penyedia datanya. Mari kita hitung sendiri luasnya dari geometrinya, sekali di sistem geografis dan sekali di sistem terproyeksi, lalu bandingkan ketiganya:

```
1 gdf_kelurahan_utm = gdf_kelurahan.to_crs(crs_proyeksi)
2 gdf_kelurahan_utm["luas_hitung_ha"] = gdf_kelurahan_utm.area / 10_000
3
4 print("Luas dari kolom bawaan :", round(gdf_kelurahan["luas"].sum() / 100, 2),
      "km2")
5 print("Luas          hitung          di          UTM          48S          :",
      round(gdf_kelurahan_utm["luas_hitung_ha"].sum() / 100, 2), "km2")
6 print("Luas hitung di WGS 84  :", round(gdf_kelurahan.area.sum(), 6), "derajat
      persegi")
```

Baris 1

`to_crs()` memindahkan seluruh geometri ke sistem koordinat lain. Bentuk dan hubungan antar-poligonnya terjaga; yang berubah cara angkanya dinyatakan.

Baris 2

`.area` menghitung luas tiap poligon. Karena datanya kini bersatuan meter, hasilnya meter persegi; dibagi 10.000 menjadi hektar. Tanda garis bawah pada `10_000` hanya pemisah ribuan agar angkanya mudah dibaca — Python mengabaikannya.

Baris 4

Total luas menurut kolom bawaan, dibagi 100 untuk mengubah hektar menjadi kilometer persegi.

Baris 5

Total luas hasil hitungan kita sendiri, dengan konversi yang sama.

Baris 6

Perhitungan yang sama dijalankan pada data yang **belum** diproyeksikan, sebagai pembanding. GeoPandas akan memunculkan peringatan berwarna merah muda di sini — dan itu memang tujuannya. Bacalah isinya: ia mengingatkan bahwa hasil `area` pada CRS geografis kemungkinan besar keliru, dan menyarankan `to_crs()` lebih dulu. Peringatan ini **tidak menghentikan** perhitungannya, sehingga sangat mudah terlewat kalau Anda terbiasa menggulir cepat.

```
Luas dari kolom bawaan : 183.75 km2
Luas hitung di UTM 48S : 183.6 km2
Luas hitung di WGS 84 : 0.014993 derajat persegi
```

```
1 C:\Users\LENOVO\AppData\Local\Temp\ipykernel_35360\56778976.py:6: UserWarning:
  Geometry is in a geographic CRS. Results from 'area' are likely incorrect. Use
  'GeoSeries.to_crs()' to re-project geometries to a projected CRS before this
  operation.
2
3 print("Luas hitung di WGS 84 :", round(gdf_kelurahan.area.sum(), 6), "derajat
  persegi")
```

Bandingkan ketiga angkanya. Baris ketiga dan keempat berdekatan — selisihnya kurang dari satu persen, yang wajar untuk perbedaan cara penyedia data menghitung luas. Artinya penyelarasan CRS kita benar, dan sekarang Anda punya kemampuan mengukur luas wilayah dari geometrinya sendiri tanpa bergantung pada kolom bawaan.

Baris terakhir adalah pelajarannya. Angka itu bukan luas dalam satuan apa pun yang bermakna; ia luas dalam “derajat persegi”, satuan yang besarnya berubah-ubah tergantung posisi di bumi. Python tidak menolak menghitungnya, dan tidak ada galat yang muncul.

 **Penting!**

Bersatuan Meter Belum Tentu Boleh Dipakai Mengukur

Ada satu sistem koordinat yang sangat sering Anda temui: **Web Mercator**, bernomor [EPSG:3857](#). Hampir seluruh peta daring memakainya — Google Maps, OpenStreetMap, dan peta latar yang akan muncul di bagian akhir modul ini. Satuannya meter, dan `is_projected`-nya `True`.

Godaannya jelas: ia bersatuan meter, jadi tampaknya boleh dipakai mengukur. **Jangan.** Web Mercator dirancang untuk menampilkan peta di layar, bukan untuk mengukur. Ia melebih-lebihkan luas makin jauh dari khatulistiwa, dan pada lintang tinggi kelebihanannya berlipat-lipat — inilah sebab Greenland tampak sebesar Afrika di banyak peta daring, padahal luasnya hanya seperempat belas. Untuk Bandar Lampung yang dekat khatulistiwa, kesalahannya memang kecil. Tetapi kebiasaan yang benar dibangun dari sekarang: **untuk menggambar pakai sistem yang cocok untuk menggambar, untuk mengukur pakai sistem yang dirancang untuk wilayah Anda.** Bagi Lampung, sistem itu adalah UTM Zona 48S, [EPSG:32748](#).

6.3.e Penggabungan Data: Menggabungkan Atribut Kelurahan ke Poligon Batas

Sampai di sini kita punya dua hal yang terpisah: poligon batas kelurahan yang keterangannya sangat sedikit, dan tabel PODES yang kaya keterangan tetapi tidak punya bentuk wilayah. Keduanya berbicara tentang 126 kelurahan yang sama. Yang perlu kita lakukan adalah menjodohkannya.

Pekerjaan ini sudah Anda kenal — di Modul 3 Anda menggabungkan tabel kelurahan dengan tabel kecamatan memakai `LEFT JOIN` di SQL. Pandas punya perintah yang mengerjakan hal yang sama, dan ia bekerja pada `GeoDataFrame` tanpa perbedaan apa pun:

```
1 kolom_gabung = ["kode_permendagri", "kepadatan_pddk",
2                 "jml_sd_negeri", "jml_sd_swasta",
3                 "jml_smp_negeri", "jml_smp_swasta"]
4
5 gdf_peta = gdf_kelurahan.merge(df[kolom_gabung], on="kode_permendagri",
6                                how="left")
7 gdf_peta.head()
```

Baris 1-3

Kolom yang hendak dibawa ke peta dikumpulkan lebih dulu di dalam sebuah *list*, persis seperti variabel `kolom_pilihan` yang Anda pakai di Modul 2. Cara ini membuat baris penggabungannya tetap pendek dan daftarnya mudah diubah kalau nanti Anda butuh kolom lain. Perhatikan `kode_permendagri` ikut disertakan — tanpa kuncinya, tidak ada yang bisa dijodohkan.

Baris 5

`merge` adalah padanan Pandas dari `LEFT JOIN` di Modul 3. Parameter `on=` menyebut kunci penghubungnya, dan argumen `how="left"` menjamin seluruh 126 poligon tetap utuh sekalipun ada baris atribut yang tidak menemukan pasangan.

Baris 6

Hasilnya diperiksa. Kolom-kolom PODES kini sudah menempel di sebelah kolom `geometry`.

	kode_permendagri	nama_des	nama_kec	luas	\
0	1871011003	Kedaton	Kedaton	109.397846	
1	1871011004	Surabaya	Kedaton	62.607745	
2	1871011005	Sukamenanti	Kedaton	54.288118	
3	1871011006	Sidodadi	Kedaton	74.003176	
4	1871011009	Sukamenanti Baru	Kedaton	27.424105	

```

                                geometry  kepadatan_pddk  \
0  MULTIPOLYGON Z (((105.26504 -5.38006 0, 105.26...      8818
1  MULTIPOLYGON Z (((105.26516 -5.38534 0, 105.26...      9042
2  MULTIPOLYGON Z (((105.25597 -5.3876 0, 105.256...     28790
3  MULTIPOLYGON Z (((105.26128 -5.38641 0, 105.26...      9717
4  MULTIPOLYGON Z (((105.25709 -5.39134 0, 105.25...     20233

    jml_sd_negeri  jml_sd_swasta  jml_smp_negeri  jml_smp_swasta
0                1              1              0              3
1                2              0              0              0
2                2              0              0              0
3                1              2              0              1
4                1              0              0              2

```

⚠ Penting!

Yang Kiri Menentukan: GeoDataFrame Harus di Depan

Perhatikan urutan penulisannya: `gdf_kelurahan.merge(df, ...)` — **GeoDataFrame di depan**, DataFrame biasa di dalam kurung.

Kalau urutannya dibalik menjadi `df.merge(gdf_kelurahan, ...)`, perintahnya tetap berjalan dan hasilnya tetap terbit. Tetapi yang keluar adalah **DataFrame biasa**, bukan GeoDataFrame. Kolom `geometry` masih ada, isinya masih objek geometri, hanya saja Pandas tidak lagi memperlakukannya sebagai kolom istimewa. Akibatnya `.plot()` menggambar grafik yang salah, `.to_crs()` menghilang, dan `.area` tidak bisa dipanggil.

Aturannya mudah diingat: **hasil penggabungan mengikuti jenis yang di kiri**. Kalau Anda ingin hasilnya tetap bisa dipetakan, taruh GeoDataFrame di sebelah kiri.

i Pengetahuan Komputer

Kode Wilayah Adalah Teks, Bukan Angka

Inilah alasan `dtype={"kode_permendagri": str}` yang kita pasang jauh di awal bagian sebelumnya.

Kode Permendagri sebuah kelurahan berbentuk `1871011003`. Ia tampak seperti angka, tetapi sesungguhnya bukan besaran — tidak ada gunanya menjumlahkan atau merata-ratakan kode wilayah. Ia sebuah **penanda**, dan penanda selalu berjenis teks.

Kalau dibiarkan terbaca sebagai angka, dua persoalan muncul:

- **Nol di depan menguap.** Kode wilayah yang diawali nol, seperti `0371...`, akan kehilangan nolnya begitu diperlakukan sebagai bilangan.
- **Penggabungan gagal tanpa peringatan.** Di berkas GeoJSON, kode itu tersimpan sebagai teks `"1871011003"`. Kalau di sisi Pandas ia terbaca sebagai bilangan `1871011003`, Python tidak menganggap keduanya sama. Hasil `merge` tetap terbit dengan 126 baris, hanya saja seluruh kolom PODES-nya kosong.

Karena itu, biasanya menyebutkan `dtype` untuk setiap kolom kode wilayah sejak perintah `read_csv` — bukan membetulkannya belakangan.

Penggabungan yang berhasil tidak pernah dianggap berhasil sebelum diperiksa. Dua pemeriksaan berikut wajib dilakukan setiap kali:

```
1 print("Jumlah poligon      :", len(gdf_peta))
2 print("Atribut yang kosong :", gdf_peta["kepadatan_pddk"].isna().sum())
3 print("Masih GeoDataFrame? :", isinstance(gdf_peta, gpd.GeoDataFrame))
```

Baris 1

Jumlah barisnya harus tetap 126. Bertambah berarti ada kunci yang berpasangan lebih dari satu kali; berkurang berarti parameter `how=` Anda bukan `"left"`.

Baris 2

Jumlah atribut yang kosong harus nol. Angka 126 di sini adalah gejala khas kunci yang tipenya tidak sama, seperti yang diterangkan pada kotak di atas.

Baris 3

Jenis hasilnya dipastikan masih `GeoDataFrame`, bukan `DataFrame` biasa.

```
Jumlah poligon      : 126
Atribut yang kosong  : 0
Masih GeoDataFrame? : True
```

6.3.f Peta Tematik Pertama: Choropleth Kepadatan Penduduk

Sekarang seluruh bahan sudah di tangan: poligon yang benar, atribut yang menempel, dan sistem koordinat yang selaras. Saatnya membuat **peta tematik** — peta yang mewarnai wilayah menurut nilai sebuah variabel, dan dalam bahasa perencanaan lazim disebut *choropleth map*.

Kita buat dua rupa. Yang pertama peta diam, dibangun dengan Matplotlib yang sudah Anda kuasai di Modul 5. Peta seperti inilah yang masuk ke dalam laporan cetak:

```
1 import matplotlib.pyplot as plt
2
3 opsi_judul = {"fontsize": 14, "fontweight": "bold"}
4
5 fig, ax = plt.subplots(figsize=(9, 9))
6 gdf_peta.plot(column="kepadatan_pddk", cmap="OrRd", scheme="quantiles", k=5,
7               legend=True, edgecolor="white", linewidth=0.3, ax=ax)
8 gdf_sekolah.plot(ax=ax, color="black", markersize=3)
9 ax.set_title("Kepadatan Penduduk dan Sebaran Satuan Pendidikan\nKota Bandar
Lampung",
10             **opsi_judul)
11 ax.set_axis_off()
12 plt.show()
```

Baris 1

Matplotlib diimpor seperti di Modul 5.

Baris 3

Kamus estetika dari Modul 5 dipakai lagi apa adanya.

Baris 5

Satu kanvas gambar disiapkan. Variabel `ax` adalah “kertas gambar” tempat semua lapisan akan ditumpuk — inilah kunci menumpuk peta di Matplotlib.

Baris 6-7

Lapisan pertama: poligon kelurahan. Parameter `column=` menentukan kolom mana yang mewarnai, parameter `cmap=` memilih gradasi warnanya, argumen `scheme="quantiles"` dengan argumen `k=5` membagi nilainya menjadi lima kelas, dan argumen `legend=True` memunculkan keterangan warnanya.

Baris 8

Lapisan kedua: titik satuan pendidikan. Karena memakai variabel `ax` yang sama, ia digambar **di atas** poligon, bukan di kanvas terpisah — persis seperti menumpuk kalkir.

Baris 10

Judul dipasang, dan tanda `**` membongkar kamus langkah 2 menjadi argumen — pola yang sama seperti Modul 5. Tanda `\n` di dalam teks judul memindahkan sisanya ke baris baru.

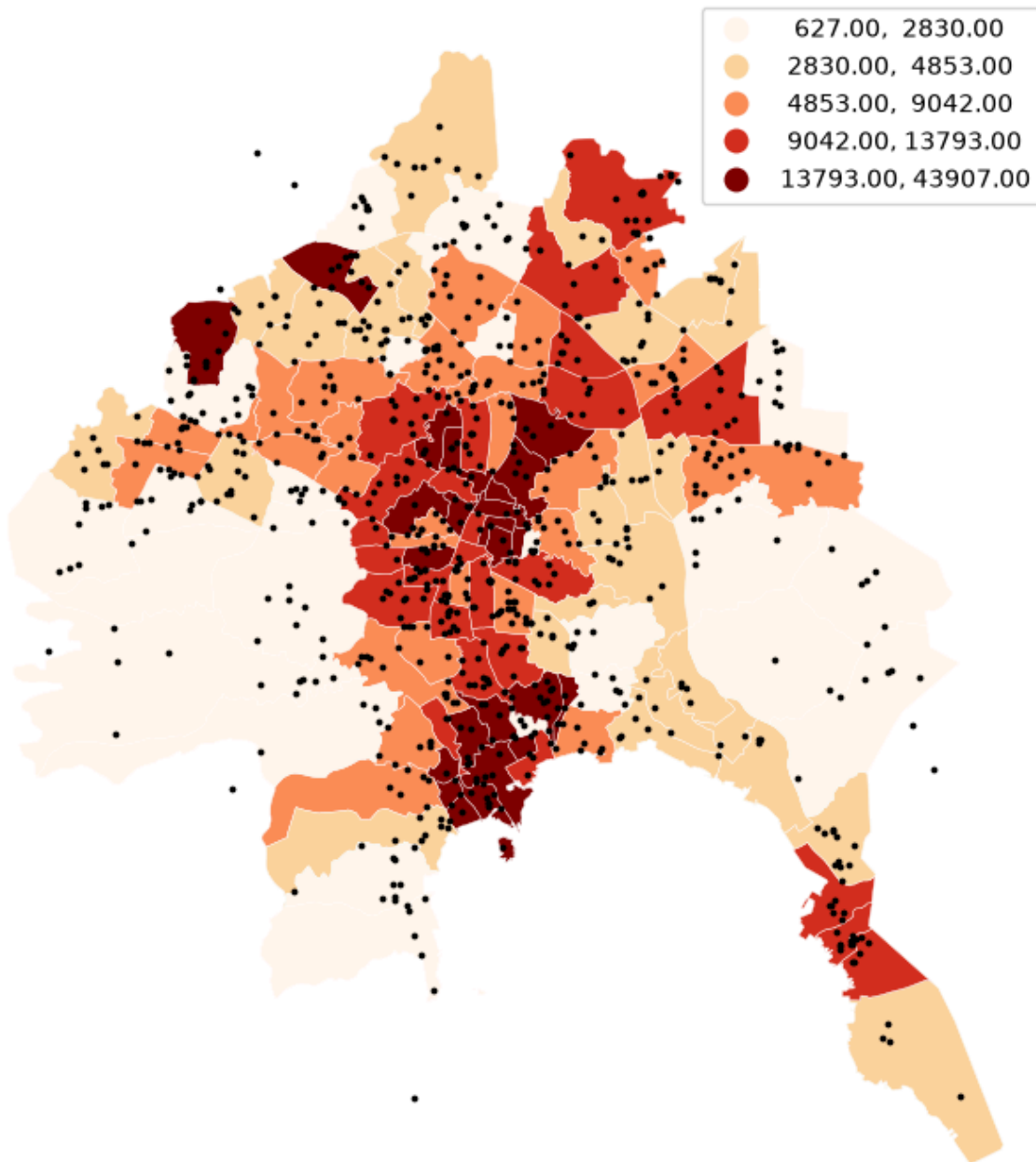
Baris 11

Sumbu X dan Y dimatikan. Peta tidak memerlukan angka koordinat di tepinya, dan tanpa itu gambarnya jauh lebih bersih.

Baris 12

Gambar ditampilkan.

Kepadatan Penduduk dan Sebaran Satuan Pendidikan Kota Bandar Lampung



⚠ Penting!

Skema Klasifikasi Adalah Keputusan, Bukan Kebetulan

Pilihan `scheme="quantiles"` di atas tampak sepele, padahal ia keputusan penyajian yang berdampak besar.

Kuantil membagi 126 kelurahan menjadi lima kelompok yang **jumlah anggotanya sama banyak** — sekitar 25 kelurahan per warna. Hasilnya peta yang selalu terlihat “berimbang”, karena setiap warna pasti terpakai. Kelemahannya: dua kelurahan yang kepadatannya nyaris sama bisa terjatuh ke warna berbeda hanya karena kebetulan berada di sisi berlainan dari batas kelompok.

Alternatifnya `scheme="EqualInterval"`, yang membagi **rentang nilainya** menjadi lima bagian sama lebar tanpa memedulikan berapa kelurahan yang masuk. Hasilnya lebih jujur terhadap besaran angkanya, tetapi bila ada satu-dua kelurahan yang kepadatannya jauh melampaui yang lain — dan di Modul 4 Anda sudah menemukan bahwa memang ada — hampir seluruh kota akan terwarnai satu warna pucat yang sama.

Cobalah mengganti `"quantiles"` menjadi `"EqualInterval"` pada kode di atas dan bandingkan hasilnya. **Data yang sama persis, cerita yang tampak berbeda.** Karena itu, dalam laporan perencanaan, skema klasifikasi yang dipakai wajib disebutkan — bukan disembunyikan.

Rupa yang kedua adalah peta interaktif: bisa digeser, diperbesar, dan disentuh untuk memunculkan keterangan. Peta seperti ini yang dipakai untuk penelusuran sendiri maupun paparan di layar. Perkakasnya **Leafmap**.

```
1  import leafmap.foliumap as leafmap
2
3  pusat_kota_peta = (-5.3971, 105.2683)
4  m = leafmap.Map(center=pusat_kota_peta, zoom=12)
5  m.add_data(gdf_peta,
6             column="kepadatan_pddk",
7             scheme="Quantiles", k=5,
8             cmap="OrRd",
9             legend_title="Kepadatan (jiwa/km2)",
10            layer_name="Kepadatan Penduduk")
11  m
```

Baris 1

Yang diimpor adalah `leafmap.foliumap`, **bukan** `leafmap` polos. Alasannya ada pada kotak di bawah, dan ini bukan perkara selera.

Baris 3

Titik tengah peta, sekali lagi berupa *tuple*. Perhatikan urutannya (`lintang`, `bujur`) — **kebalikan** dari urutan yang dipakai `Point` dan `points_from_xy` tadi.

Baris 4

Kanvas peta dibuka pada tingkat perbesaran 12, kira-kira selebar satu kota. Angka yang lebih besar berarti lebih dekat.

Baris 5-10

Poligon yang sudah bergabung dengan atribut PODES ditumpuk ke peta. Argumennya sejalan dengan peta diam tadi: kolom pewarna, skema klasifikasi, gradasi warna, judul legenda, dan nama lapisannya.

Baris 11

Nama variabel peta ditulis sendiri di baris terakhir agar Jupyter menampilkannya — kebiasaan yang sama seperti menulis `df.head()` tanpa `print()`.

```
Unable to display output for mime type(s): text/html
```

Arahkan kursor ke salah satu kelurahan. Keteranganannya akan muncul sendiri. Cobalah juga menggeser dan memperbesar petanya.

⚠ Penting!

Tuple yang Sama, Urutan yang Terbalik

Bandingkan dua baris ini, yang keduanya menunjuk tempat yang sama persis:

```
1 titik_pusat      = Point((105.2683, -5.3971))    # (bujur, lintang)
2 pusat_kota_peta = (-5.3971, 105.2683)          # (lintang, bujur)
```

Keduanya *tuple*. Keduanya berisi dua angka yang sama. Urutannya terbalik.

Sebabnya perbedaan silsilah. `Point` berasal dari dunia geometri, yang selalu menyebut koordinat-*x* lebih dulu — dan bujurlah yang mendatar. Leafmap berdiri di atas pustaka peta web Leaflet, yang mengikuti kebiasaan navigasi dan selalu menyebut lintang lebih dulu, sama seperti aplikasi peta di ponsel Anda.

Tidak ada yang bisa dilakukan selain mengingatnya. Cara paling aman: setiap kali menuliskan koordinat, **sebutkan urutannya dalam komentar atau nama variabelnya**, dan jangan pernah menyalin pasangan koordinat dari satu perkakas ke perkakas lain tanpa memeriksanya lebih dulu.

🔗 Pengetahuan VS Code

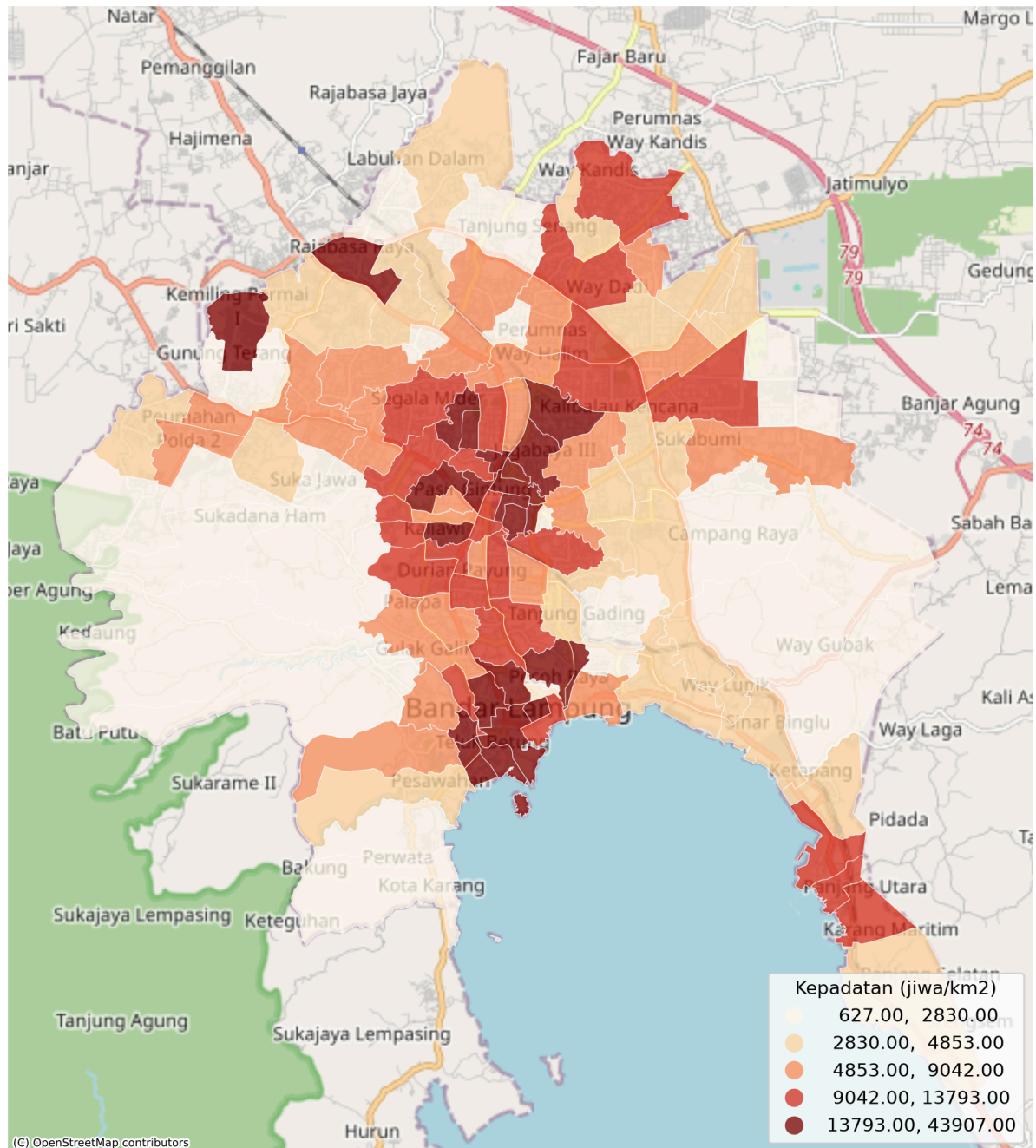
Mengapa Harus `leafmap.foliumap`

Leafmap sesungguhnya punya dua mesin penggerak yang bisa dipilih, dan keduanya menghasilkan peta yang tampak serupa di layar. Perbedaannya baru terasa ketika *notebook* Anda ditutup atau dibagikan.

- `import leafmap` — memakai mesin **ipyleaflet**. Petanya berupa *widget* yang hidup selama *kernel* masih berjalan. Begitu *notebook* ditutup, di-*restart*, atau diserahkan ke orang lain, tempat petanya berubah menjadi kotak kosong.
- `import leafmap.foliumap as leafmap` — memakai mesin **folium**. Petanya berupa halaman web utuh yang tertanam di dalam berkas *notebook*, dan tetap ada meskipun *kernel*-nya sudah mati.

Untuk keperluan praktikum ini — di mana *notebook* Anda dikumpulkan, dibuka orang lain, dan dinilai — mesin kedualah yang benar. Penulisan `as leafmap` di belakangnya membuat seluruh perintah tetap ditulis `leafmap.Map(...)` seperti biasa, sehingga contoh kode dari sumber mana pun tetap berlaku.

Kalau suatu saat peta Anda tampak baik di layar tetapi hilang setelah *Restart Kernel & Run All*, periksalah baris impor ini lebih dulu.



Gambar 1: Peta interaktif kepadatan penduduk per kelurahan Kota Bandar Lampung. Pada versi cetak modul ini, peta hanya dapat ditampilkan sebagai gambar diam; jalankan sel di atas pada komputer Anda untuk memperoleh peta yang bisa digeser dan diperbesar.

Terakhir, simpan peta interaktif itu menjadi berkas tersendiri. Berkas inilah yang bisa Anda buka di peramban mana pun tanpa Python, lampirkan ke laporan, atau kirimkan kepada orang lain:

```
1 m.to_html("peta-kepadatan-bdl.html")
2 print("Peta tersimpan sebagai peta-kepadatan-bdl.html")
```

Baris 1

`to_html()` menuliskan seluruh peta — beserta data, warna, dan legendanya — menjadi satu berkas halaman web. Karena tidak disebutkan alamat lengkapnya, berkas itu jatuh di *folder* yang sama dengan *notebook* Anda, yaitu `modul`.

Baris 2

Pemberitahuan singkat bahwa penyimpanannya selesai.

```
Peta tersimpan sebagai peta-kepadatan-bdl.html
```

Bukalah berkas `peta-kepadatan-bdl.html` lewat **File Explorer** dengan mengkliknya dua kali. Ia akan terbuka di peramban dan berperilaku persis seperti peta di dalam *notebook* — tanpa memerlukan Python sama sekali.

6.4 Latihan Mandiri

Kerjakan latihan ini di berkas *notebook* baru bernama `modul/latihan-06.ipynb` menggunakan *kernel* `analitika-perkotaan`. Pastikan alur kode Anda rapi dan terstruktur, serta jalankan perintah **Restart Kernel & Run All** untuk memverifikasi bahwa kode Anda bebas dari galat.

Latihan ini memakai *dataset* spasial yang sama, tetapi dengan fokus analisis yang berbeda: bukan kepadatan penduduk, melainkan **ketersediaan fasilitas pendidikan dasar (SD dan SMP)** di tiap kelurahan.

1. **Persiapan Data:** Baca berkas `batas_kelurahan_bdl.geojson` dengan GeoPandas dan berkas `bdl_kelurahan.csv` dengan Pandas. Ingat memaksa kolom `kode_permendagri` dibaca sebagai teks. Periksa jumlah baris keduanya dan pastikan sistem koordinatnya sama.
2. **Variabel Baru:** Buat satu kolom baru bernama `total_dikdas` yang berisi jumlah seluruh fasilitas pendidikan dasar per kelurahan:

```
total_dikdas = jml_sd_negeri + jml_sd_swasta + jml_smp_negeri + jml_smp_swasta
```
3. **Titik Satuan Pendidikan:** Baca `sekolah_bdl.csv`, buang baris yang koordinatnya kosong, lalu **saring hanya sekolah berjenjang SD dan SMP** menggunakan kolom `Bentuk` dengan penyaringan *boolean* yang Anda kuasai sejak Modul 2. Ubah hasilnya menjadi GeoDataFrame titik. Perhatikan sekali lagi tanda lintangnya.
4. **Pemeriksaan EPSG:** Cari tahu kode EPSG yang paling tepat untuk mengukur di wilayah Kota Bandar Lampung menggunakan `estimate_utm_crs()`, lalu buktikan kode itu benar dengan memeriksa `area_of_use`-nya. Proyeksikan poligon kelurahan ke sistem tersebut dan hitung luas tiap kelurahan dalam hektar.
5. **Penggabungan:** Gabungkan kolom `kode_permendagri`, `nama_des`, dan `total_dikdas` ke poligon batas kelurahan. Lakukan kedua pemeriksaan wajibnya: jumlah baris tetap 126, dan tidak ada atribut yang kosong.
6. **Peta Diam:** Buat *choropleth* `total_dikdas` per kelurahan memakai gradasi warna **hijau** (`cmap="YlGn"`), dengan lima kelas kuantil dan legenda. Tumpuk titik SD dan SMP dari langkah 3 di atasnya, beri judul yang informatif, matikan sumbunya, lalu simpan gambarnya dengan `plt.savefig()` pada resolusi 300 DPI seperti yang Anda pelajari di Modul 5.
7. **Peta Interaktif & Interpretasi:** Bangun peta Leafmap dari GeoDataFrame gabungan langkah 5 dengan skema warna yang sama, lalu tumpuk titik sekolahnya memakai `m.add_points_from_xy(df_sekolah, x="lon", y="lat")`. Simpan hasilnya sebagai `peta-dikdas-bdl.html`. Tutup dengan satu paragraf interpretasi: kelurahan mana yang tampak paling terlayani fasilitas pendidikan dasar, kelurahan mana yang paling timpang, dan apakah sebaran titik sekolahnya sejalan dengan warna kelurahannya.

7 Analisis Spasial Lanjutan: Tumpang Susun (Overlay) dan Agregasi Spasial

! Sub-CPMK Pertemuan Ini

Mahasiswa mampu menerapkan analisis spasial tumpang susun (*overlay*) berupa *spatial join* menggunakan GeoPandas untuk menggabungkan data batas administrasi kelurahan dengan data objek fisik perkotaan, serta melakukan agregasi spasial untuk menghitung statistik jumlah dan kepadatan objek per wilayah kelurahan.

7.1 Pendahuluan

Kembali ke pengalaman Anda mengoperasikan perangkat lunak GIS di studio. Sebuah *layer* poligon batas administrasi kelurahan diaktifkan di panel bawah, lalu di atasnya ditambahkan *layer* baru berisi titik-titik lokasi fasilitas hasil survei lapangan. Begitu kedua *layer* ditampilkan bersamaan, barulah terlihat jelas titik sekolah mana yang “jatuh” di kelurahan mana.

Namun sekadar menampilkan dua *layer* secara visual di atas satu sama lain sesungguhnya belum menjawab apa-apa — mata kita memang bisa menebak-nebak titik mana berada di poligon mana, tetapi untuk menghitungnya secara pasti dan sistematis ke seluruh ratusan kelurahan sekaligus, kita butuh sesuatu yang lebih dari sekadar tumpang tindih tampilan.

Pada pertemuan sebelumnya, kita sudah berhasil membuat titik-titik fasilitas pendidikan dan poligon batas kelurahan sama-sama “berbicara” dalam bahasa spasial yang sama — keduanya sudah menjadi objek geometri dengan sistem koordinat yang selaras. Namun kedua data itu masih berdiri sendiri-sendiri, belum benar-benar “bertemu” untuk menjawab pertanyaan yang sesungguhnya ingin dijawab seorang perencana, contohnya, kelurahan mana yang benar-benar kekurangan sekolah, dan kelurahan mana yang justru kelebihan?

Di sinilah GeoPandas menunjukkan kekuatannya melalui operasi *spatial join*, versi komputasional dari operasi tumpang susun (*overlay*) yang biasa Anda jalankan lewat menu di perangkat lunak GIS. Fungsi `gpd.sjoin()` menghitung secara otomatis dan presisi, untuk ribuan titik dan ratusan poligon sekaligus dalam hitungan detik, titik mana masuk ke poligon mana — memberi “label” kelurahan pada setiap titik satuan pendidikan, persis seperti hasil operasi *overlay* di GIS namun sepenuhnya bisa direproduksi dan diintegrasikan ke dalam alur analisis data yang lebih panjang.

7.2 Gambaran Umum Kasus

Berapa banyak bangunan yang berdiri di Kelurahan Kangkung? Berapa bagian dari tanahnya yang sudah tertutup atap? Dan bagaimana angka itu dibandingkan dengan Batu Putuk di kaki bukit sana? Pertanyaan semacam ini adalah pertanyaan pertama yang diajukan siapa pun yang hendak menghitung daya dukung permukiman, memperkirakan kebutuhan air bersih, atau menaksir limpasan air hujan di sebuah kota.

Data untuk menjawabnya sekarang tersedia, dan tersedia untuk seluruh dunia. Citra satelit resolusi tinggi kini rutin ditelaah oleh model kecerdasan buatan yang mengenali atap bangunan satu per satu, lalu menuangkannya menjadi poligon. Untuk Kota Bandar Lampung saja, hasilnya **369.672 bangunan** — jumlah yang mustahil dikumpulkan lewat survei lapangan mana pun dalam satu masa jabatan wali kota.

Ada satu hal yang tidak dimiliki data itu: **alamat**. Tidak ada kolom kelurahan, tidak ada kolom kecamatan, tidak ada kode wilayah. Yang ada hanya bentuk atap dan letaknya di permukaan bumi. Padahal seluruh pertanyaan perencanaan tadi diajukan per kelurahan, karena kelurahan-lah satuan tempat kebijakan dijalankan dan anggaran dialokasikan.

Di sinilah tumpang susun (*overlay*) bekerja. Modul ini menempelkan keterangan wilayah kepada ratusan ribu bangunan itu semata-mata dari letaknya, menghitung berapa banyak yang jatuh di tiap kelurahan, lalu mengubahnya menjadi dua angka yang bisa dibaca perencana: kepadatan bangunan per kilometer persegi dan seberapa besar bagian tanah kelurahan yang sudah tertutup bangunan. Sepanjang jalan, Anda akan menemukan bahwa cara paling wajar mengerjakannya justru menghasilkan hitungan yang salah — dan Anda akan membetulkannya sendiri.

7.3 Instruksi

Buat notebook baru `modul/modul-07.ipynb` di direktori proyek Anda, sambungkan ke *kernel analitika-perkotaan*, dan kerjakan seluruh instruksi di dalamnya.

7.3.a Tumpang Susun (Overlay): Ketika Data Tidak Punya Alamat

Cara paling tua untuk menjawab “ada berapa objek di dalam wilayah itu” sudah lama dikerjakan di studio, jauh sebelum ada komputer. Peta batas wilayah digambar di selembar kalkir, sebaran objeknya di lembar yang lain, keduanya ditumpuk di atas meja lampu, lalu dihitung satu per satu berapa objek yang jatuh di dalam tiap batas. Persis itulah arti “tumpang susun” — dan dalam bahasa Inggris istilah yang akan Anda temui di mana-mana adalah **overlay**.

Perangkat lunak GIS yang biasa Anda pakai sudah menyediakan tombolnya. Di ArcGIS namanya *Spatial Join*, di QGIS namanya *Join Attributes by Location*. Keduanya meminta tiga hal yang sama: satu *layer* berisi objek yang hendak dihitung, satu *layer* poligon sebagai wadahnya, dan satu aturan yang menentukan kapan sebuah objek dianggap “berada di dalam”. Di Python, ketiganya menjadi argumen dari satu fungsi bernama `gpd.sjoin`. Pekerjaannya sama persis; yang berbeda hanya mejanya.

Mari kita mulai dengan berkenalan dengan data bangunannya.

1. Buat notebook baru bernama `modul-07.ipynb` di dalam folder `modul`, lalu sambungkan ke *kernel analitika-perkotaan*.
2. Ketik dan jalankan sel berikut.

```
1 import geopandas as gpd
2
3 path_bangunan = "../dataset-ganjil-2026-2027/bangunan_bdl.gpkg"
4 gdf_bangunan = gpd.read_file(path_bangunan)
5
6 print("Jumlah bangunan :", len(gdf_bangunan))
7 print("Kolom yang ada :", list(gdf_bangunan.columns))
```

Baris 1

GeoPandas, pustaka yang sama persis dengan yang Anda pakai di Modul 6.

Baris 3

Alamat berkasnya disimpan ke dalam variabel supaya baris berikutnya tetap pendek — kebiasaan yang sudah Anda pegang sejak Modul 2.

Baris 4

`read_file()` tidak hanya sanggup membaca GeoJSON. Ia mengenali format dari akhiran berkasnya, dan `.gpkg` di sini adalah **GeoPackage**, format yang sudah disebut di tabel Modul 6 tetapi baru sekarang kita pakai.

Baris 6

Jumlah barisnya. Satu baris berarti satu bangunan.

Baris 7

Daftar kolomnya. Bacalah baik-baik.

```
Jumlah bangunan : 369672
Kolom yang ada  : ['height', 'occupancy', 'source', 'luas_m2', 'geometry']
```

Perhatikan daftar kolom yang tercetak: `height`, `occupancy`, `source`, `luas_m2`, dan `geometry`. Tidak ada `nama_des`. Tidak ada `nama_kec`. Tidak ada `kode_permendagri`. Data ini tahu persis di titik mana setiap bangunan berdiri, tetapi sama sekali tidak tahu bangunan itu masuk kelurahan yang mana.

3. Periksa isi ketiga kolom keterangannya untuk memastikan tidak ada yang bisa dipakai sebagai pengganti keterangan wilayah.

```
1 print(gdf_bangunan["source"].value_counts())
2 print()
3 print(gdf_bangunan["occupancy"].str[:3].value_counts().head())
```

Baris 1

Sumber pengenalan tiap bangunan. Datanya adalah gabungan dari tiga pihak yang memetakan atap secara terbuka: Google, OpenStreetMap, dan Microsoft.

Baris 3

Peruntukan bangunan. Kodenya bertingkat — `RES` dan `RES1` sama-sama hunian, `COM4` dan `COM5` sama-sama niaga — sehingga tiga huruf pertamanya diambil lebih dulu supaya keluarganya terkumpul. `.str[:3]` inilah yang mengerjakan pemotongan itu.

```
source
Google      216406
OSM          145593
Microsoft     7673
Name: count, dtype: int64

occupancy
UNK      361750
RES       4393
COM       1338
IND        961
EDU        811
Name: count, dtype: int64
```

Hasilnya menjelaskan sendiri mengapa kolom `occupancy` tidak bisa diandalkan: hampir seluruhnya `UNK`, singkatan dari *unknown*. Model yang mengenali atap dari citra satelit memang pandai melihat bentuk, tetapi tidak bisa menebak apa yang terjadi di dalamnya.

Jadi kesimpulannya tegas. Tidak satu pun dari 369.672 bangunan ini membawa keterangan wilayah, dan tidak ada kolom yang bisa dipinjam sebagai penggantinya. Satu-satunya keterangan yang bisa kita peroleh harus datang dari **letaknya**.

Konsep Python

Metode Teks `.str`

Sebuah kolom berisi angka bisa langsung dijumlahkan atau dibagi. Sebuah kolom berisi teks tidak. Untuk mengolah kolom teks, Pandas menyediakan satu pintu masuk bernama `.str`, dan di baliknya tersedia perlakuan teks yang sudah Anda kenal dari kehidupan sehari-hari:

```
1 kolom.str[:3]          # ambil tiga huruf pertama
2 kolom.str.upper()      # ubah seluruhnya menjadi HURUF BESAR
3 kolom.str.lower()      # ubah seluruhnya menjadi huruf kecil
4 kolom.str.strip()      # buang spasi berlebih di awal dan akhir
5 kolom.str.contains("SD") # bernilai True bila memuat potongan teks itu
```

Kuncinya ada pada satu kata: **`.str` mengerjakan perlakuan itu pada tiap baris sekaligus**. Anda tidak perlu menuliskan perulangan `for` seperti di Modul 4; satu baris perintah sudah berlaku untuk seluruh 369.672 baris.

Yang paling sering Anda perlukan kelak adalah `.str.upper()`. Data dari dua instansi yang berbeda hampir selalu berselisih dalam hal huruf besar-kecil — satu menulis “Kedaton”, yang lain menulis “KEDATON” — dan bagi komputer keduanya adalah dua teks yang berbeda. Menyamakan keduanya menjadi huruf besar lebih dulu adalah cara paling cepat mendamaikannya.

7.3.b Menyiapkan Kedua Lapis: Batas Kelurahan dan Penyelarasan CRS

Sekarang kita ambil lapis keduanya, yaitu batas administrasi kelurahan — berkas yang sudah akrab sejak Modul 6.

1. Baca berkas batas kelurahan, lalu periksa sistem koordinat kedua lapis data Anda.

```
1 path_batas = "../dataset-ganjil-2026-2027/batas_kelurahan_bdl.geojson"
2 gdf_kelurahan = gpd.read_file(path_batas)
3
4 print("Jumlah kelurahan :", len(gdf_kelurahan))
5 print("CRS kelurahan    :", gdf_kelurahan.crs)
6 print("CRS bangunan     :", gdf_bangunan.crs)
```

Baris 1

Berkas yang sama persis dengan Modul 6.

Baris 2

Dibaca menjadi GeoDataFrame berisi 126 poligon kelurahan.

Baris 4

Jumlahnya diperiksa lebih dulu — kebiasaan yang perlu Anda pelihara setiap kali membuka data.

Baris 5-6

Sistem koordinat kedua lapis dicetak berdampingan supaya ketahuan apakah keduanya sudah sepaham.

```
Jumlah kelurahan : 126
CRS kelurahan    : EPSG:4326
CRS bangunan     : EPSG:4326
```

Keduanya EPSG:4326, artinya sudah sepaham. Tetapi sepaham saja belum cukup untuk pekerjaan kita: nanti kita perlu **mengukur** luas kelurahan dalam kilometer persegi, dan Modul 6 sudah menunjukkan bahwa mengukur di atas CRS geografis menghasilkan “derajat persegi” yang tidak bermakna. Karena itu keduanya kita pindahkan ke UTM 48S.

2. Proyeksikan kedua lapis ke EPSG:32748.

```
1 crs_proyeksi = "EPSG:32748"
2
3 gdf_bangunan = gdf_bangunan.to_crs(crs_proyeksi)
4 gdf_kelurahan = gdf_kelurahan.to_crs(crs_proyeksi)
5
6 print("CRS bangunan :", gdf_bangunan.crs.to_epsg())
7 print("CRS kelurahan :", gdf_kelurahan.crs.to_epsg())
```

Baris 1

UTM zona 48 Selatan, satu-satunya sistem yang sudah Anda buktikan sendiri di Modul 6 sebagai yang tepat untuk wilayah Lampung dan bersatuan meter.

Baris 3-4

Kedua lapis dipindahkan, dan hasilnya ditimpakan kembali ke variabel semula agar tidak ada dua salinan data sebesar ini menganggur di memori.

Baris 6-7

Nomor EPSG keduanya dicetak sebagai bukti perpindahan berhasil.

```
CRS bangunan    : 32748
CRS kelurahan    : 32748
```

Penting!

Tumpang Susun Menuntut Satu Sistem Koordinat

Menumpuk dua lembar kalkir yang skalanya berbeda tidak akan menghasilkan apa-apa selain kebingungan. Begitu pula tumpang susun spasial: kedua lapis **wajib** berada pada CRS yang sama.

Bila keduanya berbeda, GeoPandas tidak menolak mengerjakannya. Ia hanya memunculkan peringatan berwarna merah muda — jenis peringatan yang sama seperti yang Anda temui di Modul

6 saat menghitung luas di CRS geografis — lalu tetap melanjutkan. Hasilnya bisa berupa tabel kosong sama sekali, atau yang lebih berbahaya, tabel berisi sebagian data yang tampak masuk akal padahal keliru.

Karena itu, jadikan ini gerakan wajib: **sebelum `sjoin`, cetak CRS kedua lapis dan pastikan keduanya sama**. Satu baris `print` sekarang jauh lebih murah daripada satu peta yang salah nanti.

3. Sebelum melangkah lebih jauh, tengok dulu wujud datanya. Menggambar 369.672 bangunan sekaligus tidak akan menghasilkan gambar yang bisa dibaca — yang tampak hanya bercak hitam sekota. Jadi kita ambil satu kecamatan saja.

```
1  kecamatan_contoh = gdf_kelurahan[gdf_kelurahan["nama_kec"] == "Kedaton"]
2  gdf_kedaton = gpd.read_file(path_bangunan, mask=kecamatan_contoh)
3
4  print("Kelurahan di Kecamatan Kedaton :", len(kecamatan_contoh))
5  print("Bangunan di Kecamatan Kedaton :", len(gdf_kedaton))
```

Baris 1

Penyaringan baris biasa memakai operator perbandingan `==` yang Anda kuasai sejak Modul 2. Hasilnya tujuh poligon kelurahan yang membentuk Kecamatan Kedaton.

Baris 2

Inilah yang baru: argumen `mask=` membuat `read_file()` hanya membaca bangunan yang bersinggungan dengan bentuk yang diberikan. Sisanya tidak pernah masuk ke memori sama sekali.

Baris 4-5

Kedua jumlah dicetak sebagai pemeriksaan.

```
Kelurahan di Kecamatan Kedaton : 7
Bangunan di Kecamatan Kedaton : 14301
```

Empat belas ribu bangunan di satu kecamatan saja. Angka sebesar itu belum berarti apa-apa sebelum Anda melihat wujudnya.

4. Gambarkan potongan itu supaya Anda punya bayangan seperti apa rupa datanya.

```
1  import matplotlib.pyplot as plt
2
3  fig, ax = plt.subplots(figsize=(8, 8))
4  gdf_kedaton.to_crs(crs_proyeksi).plot(ax=ax, color="dimgray")
5  kecamatan_contoh.plot(ax=ax, facecolor="none",
6                          edgecolor="crimson", linewidth=1.2)
7  ax.set_title("Bangunan di Kecamatan Kedaton, Kota Bandar Lampung",
8              fontsize=13, fontweight="bold")
9  ax.set_axis_off()
10 plt.show()
```

Baris 1

Matplotlib, pustaka penggambar yang sama seperti Modul 5.

Baris 3

Satu kanvas berukuran 8×8 inci disiapkan.

Baris 4

Bangunannya digambar lebih dulu sebagai lapisan bawah. Perhatikan `to_crs` yang disisipkan di tengah baris: berkasnya baru saja dibaca ulang dari cakram sehingga CRS-nya kembali `EPSG:4326`, dan ia harus disamakan dengan poligon kelurahan yang sudah berpindah ke UTM.

Baris 5-6

Batas kelurahannya ditumpuk di atas, tanpa isi (`facecolor="none"`) agar bangunan di bawahnya tetap terlihat.

Baris 7-8

Judul gambar, memakai pola argumen yang sama seperti Modul 5.

Baris 9

Sumbu koordinat dimatikan — pada peta, angka meter UTM di tepi kanvas tidak memberi keterangan apa pun kepada pembaca.

Baris 10

Gambar ditampilkan.

Bangunan di Kecamatan Kedaton, Kota Bandar Lampung



Sekarang Anda melihat persoalannya secara harfiah. Ribuan bangunan berdiri di dalam tujuh poligon kelurahan itu, dan tidak satu pun dari mereka tahu poligon yang mana. Perhatikan juga bangunan-bangunan yang tergambar melewati garis merah: parameter `mask=` memang menarik semua yang **bersinggungan** dengan kecamatan, termasuk yang hanya menyerempet tepinya. Ingat pemandangan itu; sebentar lagi ia menjadi pokok persoalan.

Pengetahuan Komputer

GeoJSON, GeoPackage, dan Data yang Terlalu Besar

Modul 6 memperkenalkan GeoJSON sebagai satu berkas teks yang ramah dikirim dan bisa dibaca manusia. Semua itu tetap benar — sampai datanya membesar.

Data bangunan ini semula memang berupa GeoJSON, dan ukurannya **234,5 MB**. Sebabnya melekat pada sifatnya sebagai berkas teks: setiap angka koordinat ditulis sebagai deretan karakter satu per satu, dan setiap nama kolom diulang kembali pada tiap baris. Data yang sama, setelah diubah

menjadi GeoPackage, tinggal **88,9 MB** — susut 62% tanpa kehilangan satu bangunan pun. Waktu membacanya turun dari **15,3 detik menjadi 1,5 detik**, sepuluh kali lebih cepat.

Sebabnya GeoPackage bukan berkas teks, melainkan **basis data** — kerabat dekat SQLite yang Anda pakai di Modul 3, hanya saja sanggup menyimpan geometri. Angka disimpan sebagai angka, nama kolom ditulis sekali di daftar tabelnya, dan tersedia indeks yang membuat perangkat lunak bisa melompat ke bagian yang dicari tanpa membaca seluruh isinya.

Indeks itulah yang membuat parameter `mask=` bekerja secepat tadi. Tanpa indeks, mengambil satu kecamatan tetap menuntut seluruh berkas dibaca dulu. Karena itu pegang pedoman ini: **GeoJSON untuk data kecil yang perlu dikirim atau dibaca mata; GeoPackage untuk data besar yang perlu diolah.**

7.3.c Operasi Tumpang Susun: `gpd.sjoin` dan Pemilihan Predicate

Sekarang kedua lapis siap dan berada pada sistem koordinat yang sama. Waktunya menumpuk keduanya.

Fungsi yang mengerjakannya bernama `gpd.sjoin`, singkatan dari *spatial join*. Ia menerima lapis yang isinya dihitung sebagai argumen pertama, lapis poligon wadahnya sebagai argumen kedua, lalu dua keterangan lagi: parameter `how` menentukan baris mana yang dipertahankan, dan parameter `predicate` menentukan aturan geometrisnya.

1. Jalankan tumpang susun dengan aturan yang paling wajar, yaitu “bangunan berada **di dalam** kelurahan”.

```
1  gdf_within = gpd.sjoin(gdf_bangunan, gdf_kelurahan,
2                          how="inner", predicate="within")
3
4  print("Bangunan semula      :", len(gdf_bangunan))
5  print("Bangunan hasil join :", len(gdf_within))
6  print("Selisihnya          :", len(gdf_bangunan) - len(gdf_within))
```

Baris 1

Lapis bangunan di depan, lapis kelurahan di belakang. Urutannya menentukan: hasilnya berupa baris-baris bangunan yang **ditempeli** kolom-kolom kelurahan tempatnya berada, bukan sebaliknya.

Baris 2

Argumen `how="inner"` mempertahankan hanya bangunan yang menemukan pasangan; argumen `predicate="within"` menetapkan aturannya, yaitu bangunan harus seluruhnya berada di dalam poligon kelurahan.

Baris 4-5

Jumlah sebelum dan sesudah dicetak berdampingan.

Baris 6

Selisih keduanya. Inilah angka yang perlu Anda perhatikan.

```
Bangunan semula      : 369672
Bangunan hasil join   : 365959
Selisihnya            : 3713
```

Ada **3.713 bangunan yang lenyap** — satu persen dari seluruh data. Dan perhatikan baik-baik: tidak ada pesan galat, tidak ada peringatan berwarna, tidak ada tanda apa pun bahwa sesuatu telah hilang. Kalau Anda tidak sengaja mencetak selisihnya, Anda tidak akan pernah tahu.

Ke mana perginya? Kembalilah pada gambar Kecamatan Kedaton tadi, pada bangunan-bangunan yang tergambar menembus garis merah. Bangunan yang tubuhnya terpotong batas kelurahan tidak berada seluruhnya di dalam kelurahan mana pun. Bagi `within`, ia bukan milik siapa-siapa, jadi ia dibuang.

Konsep Python

Keyword Argument: Argumen yang Menyebut Namanya

Perhatikan cara `sjoin` dipanggil. Dua argumen pertama diberikan begitu saja, sedangkan dua berikutnya menyebut namanya lebih dulu: `how="inner"` dan `predicate="within"`. Yang kedua inilah yang disebut **keyword argument** — argumen bernama.

Perbedaannya pada apa yang menentukan maknanya:

- **Argumen posisi** dikenali dari urutannya. `gpd.sjoin(gdf_bangunan, gdf_kelurahan)` berbeda hasilnya dari `gpd.sjoin(gdf_kelurahan, gdf_bangunan)`, karena yang di depan adalah lapis yang dihitung.
- **Keyword argument** dikenali dari namanya, sehingga urutannya bebas. Menulis `predicate="within", how="inner"` sama saja hasilnya.

Pola ini sudah pernah Anda temui, hanya belum bernama. Di Modul 5 Anda menyusun `opsi_label = {"fontsize": 12, ...}` lalu membongkarkannya dengan `plt.title(judul, **opsi_label)`. Yang dikerjakan tanda `**` itu persis ini: mengubah tiap pasangan di dalam *dictionary* menjadi *keyword argument*.

Kebiasaan yang perlu Anda pelihara: **sebutkan nama argumennya begitu jumlahnya lebih dari dua**. `gpd.sjoin(a, b, "inner", "within")` masih diterima Python, tetapi enam bulan lagi tidak ada yang bisa menebak arti dua teks di ujung itu tanpa membuka dokumentasinya.

2. Coba aturan yang lebih longgar. Kalau “seluruhnya di dalam” terlalu ketat, bagaimana kalau cukup “bersinggungan”?

```
1  gdf_intersects = gpd.sjoin(gdf_bangunan, gdf_kelurahan,
2                             how="inner", predicate="intersects")
3
4  print("Baris hasil join      :", len(gdf_intersects))
5  print("Bangunan yang berbeda :", gdf_intersects.index.nunique())
6  print("Tercatat lebih sekali :",
7        len(gdf_intersects) - gdf_intersects.index.nunique())
```

Baris 1-2

Tumpang susun yang sama persis, hanya aturannya diganti menjadi `intersects` — bersinggungan, sesedikit apa pun.

Baris 4

Jumlah baris pada tabel hasilnya.

Baris 5

Jumlah bangunan yang **berbeda** di dalam tabel itu. `nunique()` menghitung nilai unik, dan `index` adalah nomor baris asal tiap bangunan.

Baris 6-7

Selisih keduanya, yaitu berapa bangunan yang muncul lebih dari satu kali.

```
Baris hasil join      : 372127
Bangunan yang berbeda : 369672
Tercatat lebih sekali : 2455
```

Sekarang tidak ada satu pun bangunan yang hilang — angka pada baris kedua sama persis dengan 369.672. Tetapi muncul persoalan yang berlawanan: **2.455 bangunan tercatat dua kali**, masing-masing di dua kelurahan sekaligus. Bangunan yang tadi dibuang karena terpotong batas kini justru dihitung ganda.

Bagi keperluan kita, keduanya sama-sama salah. `within` membuat sebuah kelurahan tampak lebih sedikit bangunannya daripada kenyataan; `intersects` membuat jumlah bangunan sekota lebih besar daripada bangunan yang benar-benar ada.

⚠ Penting!

Daftar Predicate dan Dua yang Menjebak

`gpd.sjoin` menerima sepuluh aturan geometris: `within`, `contains`, `contains_properly`, `covers`, `covered_by`, `intersects`, `overlaps`, `crosses`, `touches`, dan `dwithin`. Dua di antaranya sering dipilih orang karena namanya terdengar menjanjikan, padahal tidak menolong sama sekali di sini.

`touches` bukan berarti “bersentuhan” dalam pengertian sehari-hari. Ia hanya bernilai benar bila kedua bentuk bersinggungan **di tepinya saja sementara bagian dalamnya sama sekali tidak tumpang tindih**. Akibatnya terbalik dari dugaan: bangunan yang benar-benar berada di tengah kelurahan justru bernilai salah untuk `touches`. Yang Anda maksud ketika berkata “menyentuh” hampir selalu `intersects`.

`covered_by` tidak menyelamatkan bangunan perbatasan. Ia memang sedikit lebih longgar daripada `within` — bentuk yang seluruhnya tergeletak persis di atas garis batas masih dihitung — tetapi bangunan yang sebagian tubuhnya keluar tetap gugur. Pada data ini hasilnya **sama persis dengan `within`**, 365.959 bangunan, dan berjalan lima kali lebih lambat.

⚠ Penting!

Mengapa Modul 6 Tidak Pernah Memusingkan Ini

Pilihan *predicate* baru berpengaruh ketika **kedua lapis sama-sama punya luas**. Sebuah titik tidak punya tubuh yang bisa terpotong: ia jatuh di dalam sebuah poligon, atau tidak sama sekali.

Buktinya bisa Anda uji sendiri kelak pada data titik satuan pendidikan: `within`, `covered_by`, dan `intersects` memberikan hasil yang sama persis, 827 titik, tanpa selisih satu baris pun.

Iniilah sebabnya seluruh pekerjaan spasial Anda di Modul 6 berjalan mulus tanpa pernah menyinggung soal ini. Begitu yang ditumpangsusunkan adalah poligon di atas poligon — bangunan, bidang tanah, tutupan lahan, area terbangun — persoalannya langsung muncul.

7.3.d Titik Wakil: Jalan Keluar dari Poligon yang Terpotong Batas

Kalau “di dalam” terlalu ketat dan “bersinggungan” terlalu longgar, jalan keluarnya bukan mencari aturan ketiga. Jalan keluarnya adalah **mengubah pertanyaannya**.

Bangunan yang berdiri di perbatasan tetap punya satu jawaban yang masuk akal bagi perencana: ia terdaftar di kelurahan tempat sebagian besar tubuhnya berada, sama seperti sebuah rumah yang halamannya menjorok ke kelurahan sebelah tetap dicatat di satu kelurahan saja. Cara paling sederhana dan paling lazim untuk menegakkan aturan itu adalah menciutkan tiap bangunan menjadi **satu titik wakil** di dalam tubuhnya, lalu menanyakan titik itu jatuh di poligon yang mana.

Dengan begitu persoalannya kembali menjadi persoalan Modul 6 — titik di dalam poligon — yang sudah Anda kuasai.

1. Ciutkan seluruh bangunan menjadi titik wakil, lalu ulangi tumpang susunnya.

```
1  gdf_titik = gdf_bangunan.copy()
2  gdf_titik["geometry"] = gdf_bangunan.representative_point()
3
4  gdf_join = gpd.sjoin(gdf_titik, gdf_kelurahan,
5                      how="inner", predicate="within")
6
7  print("Bangunan semula      :", len(gdf_bangunan))
8  print("Bangunan hasil join  :", len(gdf_join))
9  print("Jatuh di luar kota   :", len(gdf_bangunan) - len(gdf_join))
```

Baris 1

Seluruh tabel bangunan disalin lebih dulu dengan `.copy()`, supaya poligon aslinya tetap utuh dan masih bisa kita pakai nanti untuk menghitung luas.

Baris 2

Kolom `geometry` pada salinan itu diganti isinya: bukan lagi poligon, melainkan satu titik wakil untuk tiap bangunan. Seluruh kolom keterangannya — `luas_m2`, `source`, `occupancy` — ikut terbawa apa adanya.

Baris 4-5

Tumpang susun yang sama persis dengan yang pertama tadi, dengan aturan `within` yang sama. Yang berubah hanya wujud lapis pertamanya.

Baris 7-8

Jumlah sebelum dan sesudah.

Baris 9

Selisihnya.

```
Bangunan semula      : 369672
Bangunan hasil join   : 369095
Jatuh di luar kota    : 577
```

Sekarang **369.095 bangunan** memperoleh kelurahannya, masing-masing tepat satu. Tidak ada yang hilang karena terpotong batas, dan tidak ada yang dihitung dua kali.

Yang tersisa 577 bangunan memang benar-benar berada di luar batas Kota Bandar Lampung, dan itu bukan kesalahan hitung. Penyaringan data ini di sisi penyedia memakai batas kota yang sengaja dilebarkan sekitar 200 meter, supaya tidak ada bangunan tepi yang terlewat. Akibat wajarnya, sedikit bangunan dari wilayah tetangga ikut terbawa — dan tumpang susun tadilah yang menyaringnya keluar.

⚠ Penting!

`representative_point()`, Bukan `centroid`

GeoPandas menyediakan dua cara meringkas poligon menjadi satu titik, dan keduanya tidak setara.

- `centroid` menghitung titik berat bentuknya, seperti titik tempat sekeping karton bisa ditumpu dengan seimbang di ujung jari.
- `representative_point()` memilih satu titik yang **dijamin berada di dalam** poligonnya.

Untuk persegi panjang keduanya menghasilkan titik yang hampir sama. Perbedaannya baru muncul pada bentuk yang cekung, dan bentuk cekung ada di mana-mana pada data bangunan: rumah berdenah L, ruko yang mengitari halaman tengah, bangunan berbentuk U. Pada bentuk semacam itu **titik beratnya jatuh di halaman, di luar bangunannya sendiri**.

Bila titik itu kebetulan jatuh melewati batas kelurahan, bangunannya tercatat di kelurahan yang salah — dan sekali lagi, tanpa satu pun pesan galat.

Karena itu pedomannya sederhana: gunakan `centroid` bila Anda memang membutuhkan titik berat untuk keperluan geometris, dan gunakan `representative_point()` setiap kali Anda hanya butuh “satu titik yang mewakili bentuk ini”. Untuk tumpang susun, hampir selalu yang kedua.

7.3.e Agregasi Spasial: Menghitung Jumlah Bangunan per Kelurahan

Tabel `gdf_join` sekarang berisi 369.095 baris bangunan, masing-masing sudah membawa nama dan kode kelurahannya. Yang kita butuhkan bukan tabel sepanjang itu, melainkan satu baris per kelurahan beserta jumlah bangunannya. Pekerjaan meringkas ini bernama **agregasi**, dan perkakasnya adalah `groupby` yang Anda pakai di Modul 5 — kerabat langsung `GROUP BY` pada SQL Modul 3.

1. Kelompokkan bangunan menurut kode kelurahannya, lalu hitung banyaknya.

```
1  jumlah_bangunan = gdf_join.groupby("kode_permendagri").size()
2  jumlah_bangunan = jumlah_bangunan.reset_index(name="jml_bangunan")
3
4  print("Kelurahan yang muncul :", len(jumlah_bangunan))
5  jumlah_bangunan.head()
```

Baris 1

Baris-baris dikelompokkan berdasarkan kode kelurahan, lalu `size()` menghitung berapa baris di tiap kelompok. Kode wilayah dipakai sebagai kunci pengelompokan, bukan namanya — alasannya sama dengan yang sudah ditegaskan Modul 6: nama bisa berselisih ejaan, kode tidak.

Baris 2

Hasil metode `groupby` bukan tabel biasa melainkan deret berindeks; parameter `reset_index` mengembalikannya menjadi tabel dua kolom, dan parameter `name=` memberi nama pada kolom hitungannya.

Baris 4

Berapa kelurahan yang muncul di hasil.

Baris 5

Lima baris pertamanya ditampilkan.

```
Kelurahan yang muncul : 126
```

	kode_permendagri	jml_bangunan
0	1871011003	4156
1	1871011004	2513
2	1871011005	1750
3	1871011006	2827
4	1871011009	1102

Hasilnya rapi, tetapi belum bisa dipetakan: tabel dua kolom itu hanya berisi kode dan angka, tanpa sepotong pun geometri. Bentuknya masih tertinggal di lapis kelurahan.

2. Tempelkan hasil hitungan itu kembali ke poligon kelurahan, supaya angkanya punya bentuk dan bisa dipetakan.

```
1 gdf_peta = gdf_kelurahan.merge(jumlah_bangunan,
2                                 on="kode_permendagri", how="left")
3 gdf_peta["jml_bangunan"] = gdf_peta["jml_bangunan"].fillna(0)
4
5 print("Jumlah poligon      :", len(gdf_peta))
6 print("Total bangunan      :", int(gdf_peta["jml_bangunan"].sum()))
7 print("Masih GeoDataFrame? :", isinstance(gdf_peta, gpd.GeoDataFrame))
```

Baris 1

GeoDataFrame ditaruh di sebelah kiri `merge`, persis seperti Modul 6 — yang kiri menentukan, dan hanya dengan begitu hasilnya tetap membawa geometri.

Baris 2

Argumen `how="left"` mempertahankan **seluruh** 126 poligon kelurahan, termasuk yang tidak menemukan pasangan di tabel hitungan.

Baris 3

Kelurahan yang tidak menemukan pasangan akan berisi nilai kosong; `fillna(0)` menggantinya dengan nol.

Baris 5-6

Jumlah poligon dan total bangunan sebagai pemeriksaan.

Baris 7

Pemeriksaan bahwa hasilnya masih GeoDataFrame, bukan tabel biasa.

```
Jumlah poligon      : 126
Total bangunan      : 369095
Masih GeoDataFrame? : True
```

⚠ Penting!

Yang Tidak Punya Apa-apa Tidak Muncul Sama Sekali

Perhatikan baik-baik apa yang dikerjakan `how="left"` dan `fillna(0)` di atas. Pada data ini keduanya tampak tidak berguna: seluruh 126 kelurahan Bandar Lampung punya bangunan, jadi tidak ada satu pun baris yang perlu diselamatkan. Keduanya tetap ditulis, dan itu disengaja.

Sebabnya begini. `groupby` hanya mengenal apa yang ada di dalam tabelnya. Kelurahan yang **tidak punya satu pun** objek yang dihitung tidak pernah muncul di hasil pengelompokan — bukan sebagai nol, melainkan tidak ada sama sekali. Bila Anda lalu menggabungkannya dengan `how="inner"`, kelurahan itu ikut lenyap dari peta Anda, meninggalkan lubang putih yang mudah disangka kesalahan penggambaran.

Dan justru **nol itulah yang paling penting bagi perencana**. Kelurahan tanpa satu pun sekolah, tanpa satu pun fasilitas kesehatan, tanpa satu pun ruang terbuka — itu temuan, bukan data kosong. Menghilangkannya dari peta sama saja dengan menghapus persoalan yang seharusnya dilaporkan.

Anda akan segera membuktikan sendiri bahwa dua baris ini bukan hiasan. Pada Latihan Mandiri nanti, ada dua kelurahan yang tidak memiliki satu pun satuan pendidikan.

7.3.f Kepadatan dan Tutupan: Menormalkan Jumlah terhadap Luas

Jumlah bangunan saja belum bisa dibandingkan antar-kelurahan. Kelurahan yang luas tentu saja memuat lebih banyak bangunan daripada kelurahan sempit, dan itu bukan temuan. Yang bermakna adalah jumlah bangunan **per satuan luas** — dan itulah yang disebut kepadatan.

Kita akan menghitung dua angka sekaligus:

- **Kepadatan bangunan**, yaitu banyaknya bangunan tiap satu kilometer persegi.
- **Rasio tutupan bangunan**, yaitu bagian dari tanah kelurahan yang sudah tertutup atap. Inilah padanan kasar dari Koefisien Dasar Bangunan yang Anda kenal dari peraturan zonasi, hanya saja dihitung untuk seluruh kelurahan, bukan per persil.

1. Hitung luas tiap kelurahan dari geometrinya, lalu bagi jumlah bangunannya.

```
1  gdf_peta["luas_km2"] = gdf_peta.area / 1_000_000
2  gdf_peta["kepadatan_bangunan"] = (gdf_peta["jml_bangunan"]
3                                   / gdf_peta["luas_km2"])
4
5  luas_tertutup = gdf_join.groupby("kode_permendagri")["luas_m2"].sum()
6  luas_tertutup = luas_tertutup.reset_index(name="luas_bangunan_m2")
7
```

```

8   gdf_peta = gdf_peta.merge(luas_tertutup,
9                               on="kode_permendagri", how="left")
10  gdf_peta["luas_bangunan_m2"] = gdf_peta["luas_bangunan_m2"].fillna(0)
11  gdf_peta["rasio_tutupan"] = (gdf_peta["luas_bangunan_m2"]
12                               / (gdf_peta["luas_km2"] * 1_000_000))
13
14  kolom_lihat = ["nama_des", "jml_bangunan", "luas_km2",
15                 "kepadatan_bangunan", "rasio_tutupan"]
16  gdf_peta[kolom_lihat].sort_values("kepadatan_bangunan",
17                                     ascending=False).head(8)

```

Baris 1

Luas tiap poligon dihitung dari geometrinya, persis seperti Modul 6. Karena datanya sudah bersatuan meter, hasilnya meter persegi; dibagi sejuta menjadi kilometer persegi.

Baris 2-3

Kepadatannya: jumlah bangunan dibagi luas kelurahan.

Baris 5-6

Luas atap seluruh bangunan di tiap kelurahan dijumlahkan. Pola `groupby` yang sama seperti tadi, hanya saja yang dikerjakan bukan `size()` melainkan `sum()` atas kolom `luas_m2` — kolom yang memang sudah dibawa data bangunannya sejak awal.

Baris 8-10

Hasilnya ditempelkan ke tabel peta dengan pola yang sama persis: `how="left"`, lalu `fillna(0)`.

Baris 11-12

Rasio tutupannya: total luas atap dibagi luas kelurahan, keduanya dalam meter persegi supaya satuannya saling meniadakan dan hasilnya berupa pecahan antara 0 dan 1.

Baris 14-15

Kolom yang ingin dilihat disimpan ke dalam variabel `list` lebih dulu, kebiasaan sejak Modul 2.

Baris 16-17

Diurutkan dari yang paling padat, lalu diambil delapan teratas.

	nama_des	jml_bangunan	luas_km2	kepadatan_bangunan	\
20	Panjang Selatan	3373	0.544925	6189.842990	
125	Kangkung	2489	0.403343	6170.931105	
29	Tanjung Agung	826	0.142811	5783.873106	
30	Kebon Jeruk	1557	0.273220	5698.707868	
36	Kaliawi	2006	0.384495	5217.233607	
18	Sukajawa Baru	1407	0.273514	5144.163279	
110	Tanjungagung Raya	643	0.126836	5069.533147	
31	Sawah Lama	936	0.188599	4962.905150	
	rasio_tutupan				
20	0.509877				
125	0.470879				
29	0.494463				
30	0.445407				

36	0.384926
18	0.434599
110	0.437403
31	0.531349

Kelurahan Panjang Selatan dan Kangkung berdiri di puncak daftar dengan lebih dari 6.000 bangunan tiap kilometer persegi, dan rasio tutupan yang mendekati setengah — artinya hampir separuh permukaan tanah kedua kelurahan itu sudah tertutup atap. Cobalah membalik urutannya dengan mengganti argumen `ascending=False` menjadi `ascending=True`: di dasar daftar Anda akan menemukan Way Tataan dan Batu Putuk, yang rasio tutupannya hanya sekitar dua sampai tiga persen.

2. Sekarang perhatikan apa yang baru saja Anda kerjakan. Untuk menjawab satu pertanyaan — berapa banyak objek di tiap kelurahan — Anda menjalankan empat langkah berturut-turut: menciutkan menjadi titik wakil, menumpangsusunkan, mengelompokkan, lalu menempelkan kembali ke poligon.

Sebentar lagi kita ingin menanyakan hal yang sama untuk bangunan kecil, lalu untuk bangunan besar. Menyalin-tempel keempat langkah itu dua kali adalah cara tercepat untuk menciptakan kesalahan yang tidak akan Anda sadari — satu nama kolom lupa diganti, dan hasilnya tetap keluar tanpa galat sedikit pun.

Karena itu keempat langkah tadi kita bungkus menjadi satu perintah baru buatan sendiri.

```

1  def hitung_per_kelurahan(gdf_objek, gdf_wilayah, nama_kolom):
2      gdf_titik = gdf_objek.copy()
3      gdf_titik["geometry"] = gdf_objek.representative_point()
4
5      hasil_join = gpd.sjoin(gdf_titik, gdf_wilayah,
6                             how="inner", predicate="within")
7
8      jumlah = hasil_join.groupby("kode_permendagri").size()
9      jumlah = jumlah.reset_index(name=nama_kolom)
10
11     gdf_hasil = gdf_wilayah.merge(jumlah,
12                                   on="kode_permendagri", how="left")
13     gdf_hasil[nama_kolom] = gdf_hasil[nama_kolom].fillna(0)
14
15     return gdf_hasil

```

Baris 1

Baris pembuka sebuah fungsi. `def` menyatakan “mulai sekarang saya definisikan perintah baru”, `hitung_per_kelurahan` adalah namanya, dan tiga kata di dalam kurung adalah **parameter** — tempat kosong yang akan diisi ketika fungsinya dipanggil nanti.

Baris 2-3

Langkah titik wakil dari Bagian 4, tidak berubah sedikit pun. Bedanya, yang dikerjakan bukan lagi variabel `gdf_bangunan` melainkan parameter `gdf_objek`, apa pun yang kelak diisi ke sana.

Baris 5-6

Tumpang susun dari Bagian 3-4.

Baris 8-9

Agregasi dari Bagian 5. Nama kolom hasilnya tidak dipatok, melainkan diambil dari parameter `nama_kolom`.

Baris 11-13

Penempelan kembali ke poligon, lengkap dengan `how="left"` dan `fillna(0)` yang sudah ditegaskan tadi.

Baris 15

`return` menyerahkan hasilnya keluar. Tanpa baris ini, fungsi tetap berjalan tetapi tidak memberikan apa-apa kepada pemanggilnya.

Konsep Python

Fungsi: Perintah Buatan Sendiri

Sepanjang enam modul terakhir Anda selalu **memakai** fungsi buatan orang lain: `print()`, `read_csv()`, `groupby()`, `sjoin()`. Mulai sekarang Anda membuatnya sendiri.

Bentuk dasarnya tiga bagian:

```
1 def nama_fungsi(parameter_satu, parameter_dua): # kepala
2     hasil = parameter_satu + parameter_dua      # tubuh, menyorok ke dalam
3     return hasil                                # penyerahan hasil
```

- **Kepala** dibuka `def`, diikuti nama fungsinya dan daftar **parameter** di dalam kurung. Parameter adalah tempat kosong yang baru terisi saat fungsinya dipanggil.
- **Tubuh** adalah baris-baris yang menyorok ke dalam. Menyoroknya bukan hiasan — itulah yang memberi tahu Python di mana tubuh fungsi dimulai dan berakhir, sama seperti menyoroknya tubuh perulangan `for` di Modul 4.
- `return` menyerahkan hasil keluar. Begitu baris ini dijalankan, fungsi berhenti seketika.

Perbedaan antara **parameter** dan **argumen** kerap tertukar, dan mudah diingat: parameter adalah nama di kepala fungsi, argumen adalah nilai sesungguhnya yang Anda serahkan saat memanggilmnya.

Satu hal yang perlu Anda sadari: **variabel di dalam tubuh fungsi hidup di ruangnya sendiri**. Variabel `gdf_titik`, `hasil_join`, dan `jumlah` di atas lahir setiap kali fungsi dipanggil dan lenyap begitu fungsinya selesai. Mereka tidak akan muncul di daftar variabel notebook Anda, dan tidak akan menempa variabel bernama sama di luar sana. Inilah yang membuat fungsi aman dipanggil berulang kali.

Kapan sebuah langkah pantas dijadikan fungsi? Pedoman yang cukup untuk sekarang: **begitu Anda tergoda menyalin-tempel serangkaian baris untuk ketiga kalinya**. Bekal ini akan sangat terasa gunanya pada Ujian Tengah Semester, ketika seluruh alur kerja Bandar Lampung harus Anda pindahkan ke Kota Metro.

3. Panggil fungsi itu dua kali, untuk dua kelompok ukuran bangunan yang berbeda.

```
1 bangunan_kecil = gdf_bangunan[gdf_bangunan["luas_m2"] < 50]
2 bangunan_besar = gdf_bangunan[gdf_bangunan["luas_m2"] >= 200]
3
4 gdf_kecil = hitung_per_kelurahan(bangunan_kecil, gdf_kelurahan,
```

```

5             "jml_kecil")
6   gdf_besar = hitung_per_kelurahan(bangunan_besar, gdf_kelurahan,
7                                   "jml_besar")
8
9   print("Bangunan < 50 m2 :", len(bangunan_kecil))
10  print("Bangunan >= 200 m2 :", len(bangunan_besar))

```

Baris 1-2

Penyaringan baris memakai operator perbandingan dari Modul 2. Ambangnya dipilih dari sebaran luas bangunan di kota ini: di bawah 50 m² kira-kira sepertiga bangunan terkecil, di atas 200 m² kira-kira delapan persen bangunan terbesar.

Baris 4-5

Pemanggilan pertama. Ketiga argumennya mengisi ketiga parameter secara berurutan: data yang dihitung, poligon wadahnya, dan nama kolom yang diinginkan.

Baris 6-7

Pemanggilan kedua, dengan data dan nama kolom yang berbeda. Empat langkah tadi dikerjakan ulang seluruhnya tanpa satu baris pun ditulis lagi.

Baris 9-10

Jumlah masing-masing kelompok.

```

Bangunan < 50 m2 : 125163
Bangunan >= 200 m2 : 29015

```

Perhatikan bahwa Anda baru saja menjalankan empat langkah panjang itu dua kali dengan menuliskan dua baris pemanggilan saja. Sekarang keduanya berupa GeoDataFrame yang berdiri sendiri-sendiri, dan perlu disatukan supaya bisa dibandingkan.

4. Gabungkan kedua hasil itu ke tabel peta, lalu lihat kelurahan mana yang paling didominasi bangunan kecil.

```

1   gdf_peta = gdf_peta.merge(gdf_kecil[["kode_permendagri", "jml_kecil"]],
2                             on="kode_permendagri", how="left")
3   gdf_peta = gdf_peta.merge(gdf_besar[["kode_permendagri", "jml_besar"]],
4                             on="kode_permendagri", how="left")
5
6   gdf_peta["bagian_kecil"] = (gdf_peta["jml_kecil"]
7                               / gdf_peta["jml_bangunan"])
8
9   kolom_ukuran = ["nama_des", "jml_bangunan", "jml_kecil",
10                  "jml_besar", "bagian_kecil"]
11  gdf_peta[kolom_ukuran].sort_values("bagian_kecil",
12                                     ascending=False).head(8)

```

Baris 1-4

Dua penggabungan berturut-turut. Dari masing-masing hasil hanya dua kolom yang diambil — kunci wilayahnya dan kolom hitungannya — supaya kolom `nama_des` dan `geometry` tidak ikut terbawa dan bertabrakan.

Baris 6-7

Bagian bangunan kecil terhadap seluruh bangunan di kelurahan itu.

Baris 9-10

Kolom yang ingin ditampilkan.

Baris 11-12

Diurutkan dari yang bagian bangunan kecilnya paling besar.

	nama_des	jml_bangunan	jml_kecil	jml_besar	bagian_kecil
80	Kedaung	2280	996	94	0.436842
26	Ketapang	1587	678	172	0.427221
16	Sukadana Ham	3265	1380	159	0.422665
125	Kangkung	2489	1038	101	0.417035
71	Campang Raya	6366	2648	353	0.415960
61	Gedong Meneng Baru	1528	635	178	0.415576
23	Pidada	4074	1665	173	0.408689
105	Gunungsari	479	195	41	0.407098

⚠ Penting!

Ukuran Atap Adalah Petunjuk, Bukan Kesimpulan

Godaan terbesar dari tabel di atas adalah menariknya terlalu jauh: menyimpulkan bahwa kelurahan dengan bangunan kecil terbanyak adalah kawasan permukiman padat berpenghasilan rendah, lalu menuliskannya ke dalam laporan.

Jangan. Yang benar-benar diketahui data ini hanyalah **luas atap yang terlihat dari satelit**. Sebuah atap 30 m² bisa berarti rumah petak, tetapi bisa juga berarti garasi, warung, gudang belakang, musala kecil, atau satu bangunan besar yang atapnya terbaca terpisah-pisah oleh model pengenalan citra. Sebaliknya, satu atap 400 m² bisa berarti gedung sekolah, dan bisa juga berarti satu rumah besar.

Angka semacam ini berharga sebagai **penunjuk arah**: ia memberi tahu kelurahan mana yang pantas ditengok lebih dekat, disurvei, atau diperiksa silang dengan data lain. Ia bukan pengganti survei lapangan, dan tidak boleh berdiri sendiri sebagai dasar kebijakan yang menyangkut hajat orang banyak.

7.3.g Visualisasi: Peta Kepadatan Bangunan per Kelurahan

Seluruh angka tadi baru berbunyi setelah ditempatkan kembali di atas peta. Kita membuat dua rupa peta, persis seperti Modul 6: satu peta diam untuk dilampirkan ke laporan, dan satu peta interaktif untuk ditelusuri di layar.

1. Buat peta tematik *choropleth* kepadatan bangunan.

```
1 opsi_judul = {"fontsize": 14, "fontweight": "bold"}
2
3 fig, ax = plt.subplots(figsize=(9, 9))
4 gdf_peta.plot(column="kepadatan_bangunan", cmap="YlOrBr",
5               scheme="quantiles", k=5, legend=True,
```

```

6         edgecolor="white", linewidth=0.3, ax=ax)
7     ax.set_title("Kepadatan Bangunan per Kelurahan\nKota Bandar Lampung",
8                 **opsi_judul)
9     ax.set_axis_off()
10    plt.show()

```

Baris 1

Opsi estetika judul disimpan dalam *dictionary*, kebiasaan dari Modul 5.

Baris 3

Kanvas 9×9 inci.

Baris 4-6

Poligon kelurahan diwarnai menurut kepadatan bangunannya, dibagi lima kelas kuantil, dengan gradasi kuning ke coklat.

Baris 7-8

Judul peta, dengan `**opsi_judul` yang membongkar *dictionary* tadi menjadi *keyword argument* — sekarang Anda tahu namanya.

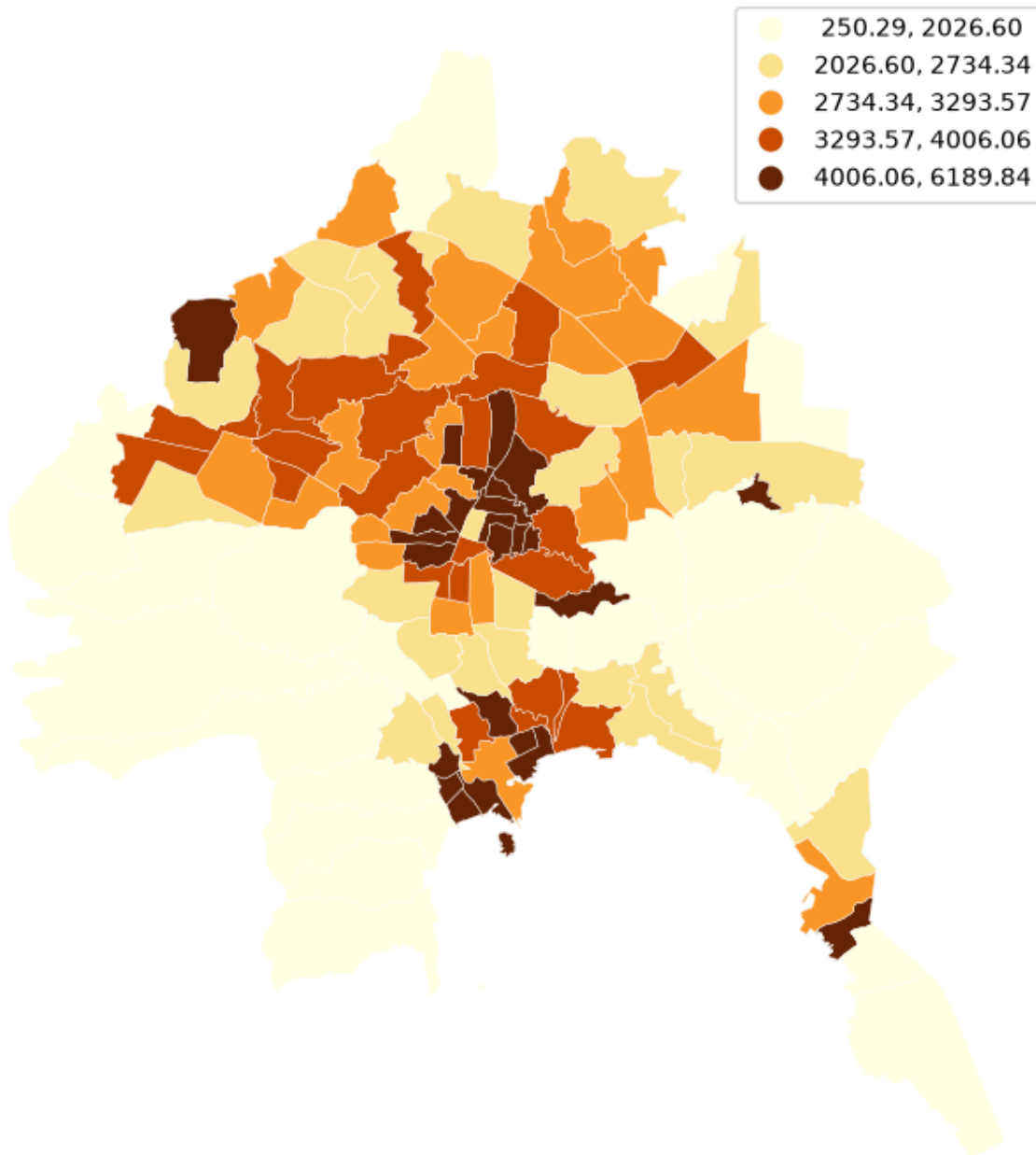
Baris 9

Sumbu dimatikan.

Baris 10

Peta ditampilkan.

Kepadatan Bangunan per Kelurahan Kota Bandar Lampung



Peta itu sudah siap dilampirkan ke laporan. Tetapi ia diam: pembaca tidak bisa memperbesar satu kelurahan yang menarik perhatiannya, dan tidak bisa membandingkan pola warnanya dengan permukiman yang sesungguhnya ada di lapangan.

2. Bangun peta interaktifnya dengan Leafmap.

```
1 import leafmap.foliummap as leafmap
2
3 gdf_peta_wgs = gdf_peta.to_crs("EPSG:4326")
4
5 pusat_kota_peta = (-5.3971, 105.2683)
6 m = leafmap.Map(center=pusat_kota_peta, zoom=12)
```

```

7     m.add_data(gdf_peta_wgs,
8                 column="kepadatan_bangunan",
9                 scheme="Quantiles", k=5,
10                cmap="YlOrBr",
11                legend_title="Bangunan per km2",
12                layer_name="Kepadatan Bangunan")
13     m

```

Baris 1

Mesin folium, bukan `leafmap` polos — alasannya sudah dibahas panjang di Modul 6, dan tetap berlaku di sini.

Baris 3

Peta web selalu bekerja pada CRS geografis, jadi data yang sudah kita pindahkan ke UTM harus dikembalikan ke `EPSG:4326` sebelum ditumpangkan. Kalau langkah ini terlewat, peta Anda akan tampak kosong padahal datanya baik-baik saja.

Baris 5

Titik tengah peta, dalam urutan (`lintang`, `bujur`) — kebalikan dari urutan geometri.

Baris 6

Kanvas peta dibuka pada perbesaran selebar satu kota.

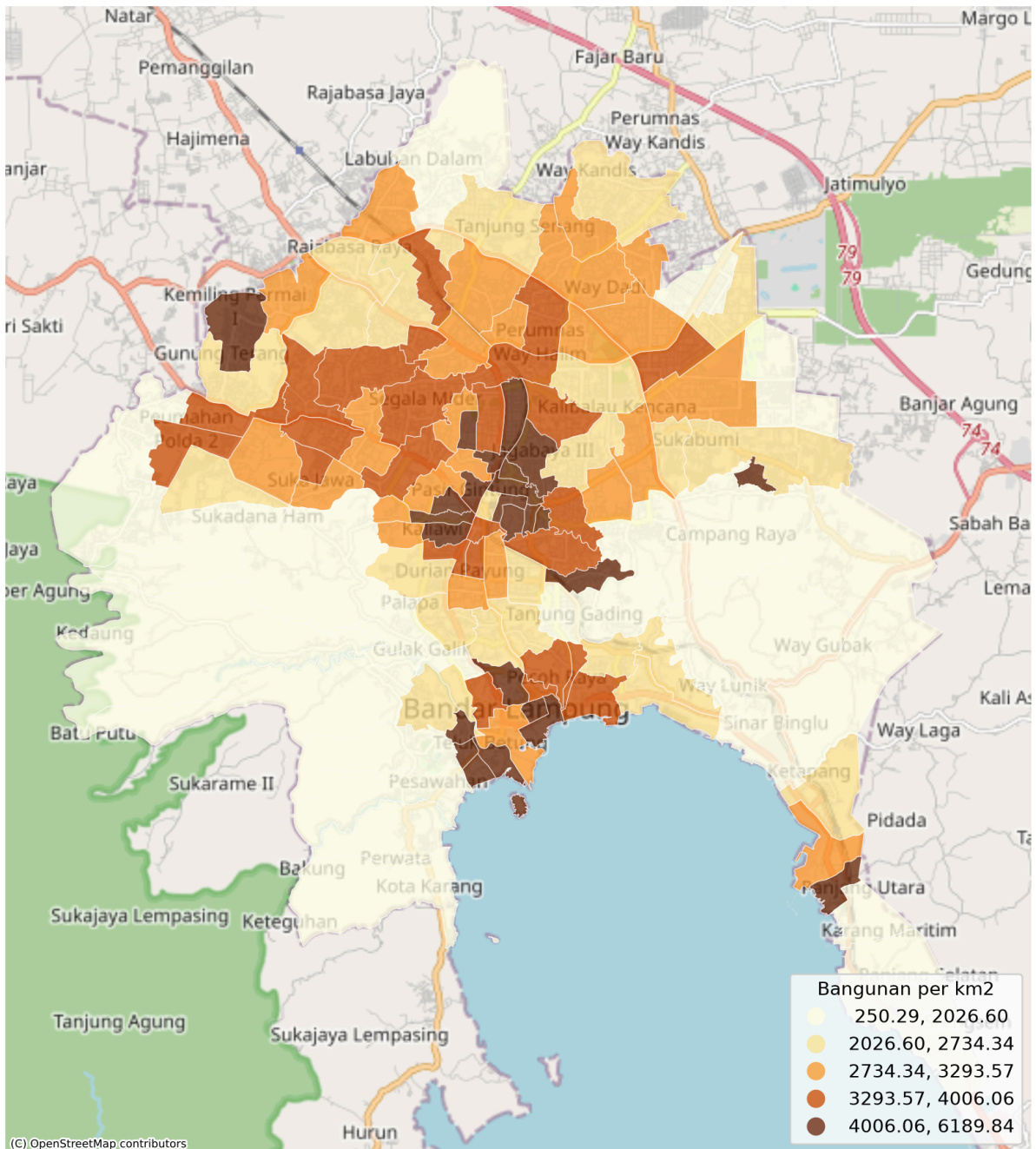
Baris 7-12

Poligon ditumpuk dengan pengaturan warna dan klasifikasi yang sama persis dengan peta diam tadi, supaya keduanya bisa dibandingkan tanpa syak.

Baris 13

Nama variabel peta ditulis sendirian agar Jupyter menampilkannya.

```
Unable to display output for mime type(s): text/html
```



Gambar 1: Peta interaktif kepadatan bangunan per kelurahan Kota Bandar Lampung. Pada versi cetak modul ini, peta hanya dapat ditampilkan sebagai gambar diam; jalankan sel di atas pada komputer Anda untuk memperoleh peta yang bisa digeser dan diperbesar.

3. Simpan peta interaktifnya sebagai berkas tersendiri agar bisa dibuka tanpa Python.

```
1 m.to_html("peta-kepadatan-bangunan-bdl.html")
2 print("Peta tersimpan sebagai peta-kepadatan-bangunan-bdl.html")
```

Baris 1

Seluruh peta beserta data, warna, dan legendanya dituliskan menjadi satu berkas halaman web di folder yang sama dengan notebook Anda.

Baris 2

Pemberitahuan singkat bahwa penyimpanannya selesai.

Lihatlah petanya baik-baik. Warna tergelap berkumpul di sekitar pusat kota lama Tanjungkarang dan di sepanjang pesisir Telukbetung sampai Panjang — kawasan yang tumbuh lebih dulu, berpetak kecil, dan hampir tidak menyisakan tanah kosong. Warna paling terang justru melingkari kota di kedua sisinya: perbukitan barat dengan Batu Putuk, Sukadana Ham, dan Sukarame II, pesisir selatan dengan Way Tataan, serta sisi timur menjelang Way Gubak. Di sana kepadatan bangunannya kurang dari sepersepuluh kawasan pesisir tadi.

Bagi seorang perencana, peta ini bukan jawaban melainkan daftar pertanyaan. Kelurahan bertutupan atap hampir separuh luas wilayahnya berarti sedikit sekali tanah yang tersisa untuk meresapkan air hujan — dan pertanyaan berikutnya adalah ke mana limpasannya pergi. Sementara kelurahan yang masih terang di lereng bukit mengundang pertanyaan sebaliknya: apakah ia memang belum terbangun, ataukah ia tidak boleh terbangun.

7.4 Latihan Mandiri

Kerjakan latihan ini di berkas notebook baru bernama `modul/latihan-07.ipynb` menggunakan *kernel* `analitika-perkotaan`. Pastikan alur kode Anda rapi dan terstruktur, serta jalankan perintah **Restart Kernel & Run All** untuk memverifikasi bahwa kode Anda bebas dari galat.

Kali ini objek yang dihitung bukan bangunan, melainkan **titik satuan pendidikan** Kota Bandar Lampung dari berkas `sekolah_bdl.geojson` — data yang sudah Anda kenal di Modul 6, berisi 839 sekolah yang seluruhnya sudah berkoordinat. Alurnya sama persis dengan Instruksi tadi, dengan satu langkah penutup yang baru.

1. **Persiapan Dua Lapis:** Baca `sekolah_bdl.geojson` dan `batas_kelurahan_bdl.geojson` dengan GeoPandas. Berbeda dengan data PODES di Modul 6, berkas ini sudah bergeometri titik sejak awal sehingga Anda tidak perlu membangunnya dari kolom `lat/lon`. Periksa jumlah baris keduanya, cetak CRS keduanya, lalu proyeksikan keduanya ke `EPSG:32748`.
2. **Tumpang Susun:** Lakukan *spatial join* titik sekolah ke dalam poligon kelurahan dengan `predicate="within"`. Periksa berapa titik yang tidak menemukan kelurahan mana pun. Kemudian **jawab dalam satu sel markdown**: mengapa pada latihan ini Anda tidak perlu menciutkan objek menjadi titik wakil seperti Bagian 4? Dan apakah hasilnya akan berbeda seandainya Anda memakai `predicate="intersects"`? Buktikan jawaban kedua dengan menjalankan keduanya dan membandingkan jumlah barisnya.
3. **Agregasi dan Kepadatan:** Gunakan fungsi `hitung_per_kelurahan` yang Anda tulis sendiri di Bagian 6 untuk menghitung jumlah sekolah per kelurahan. Salin definisi fungsinya ke notebook latihan Anda. Kemudian hitung kepadatan sekolah per kilometer persegi. Periksa dengan saksama: berapa kelurahan yang jumlah sekolahnya nol, dan apa jadinya kalau `fillna(0)` Anda hilangkan?
4. **Panggil Ulang untuk Dua Jenjang:** Saring data sekolah berdasarkan kolom `Bentuk` untuk memperoleh jenjang **SD** dan **SMP** secara terpisah, lalu panggil fungsi yang sama sekali lagi untuk masing-masing jenjang. Bandingkan berapa kelurahan yang tidak memiliki SD dan berapa yang tidak memiliki SMP.
5. **Peta:** Buat *choropleth* kepadatan seluruh satuan pendidikan per kelurahan memakai gradasi warna **hijau** (`cmap="YlGn"`) dengan lima kelas kuantil dan legenda, lalu tumpuk titik sekolahnya di atasnya. Beri judul yang informatif, matikan sumbunya, dan simpan gambarnya dengan `plt.savefig()` pada resolusi 300 DPI seperti yang Anda pelajari di Modul 5.

6. **Nama Melawan Letak:** Berkas sekolah membawa kolom `KeLurahan` berisi nama kelurahan menurut catatan Dapodik, sementara tumpang susun Anda menghasilkan kolom `nama_des` berisi kelurahan menurut **letak sesungguhnya**. Bandingkan keduanya. Karena ejaan huruf besar-kecil kedua sumber berbeda, samakan dulu dengan `.str.upper()` seperti pada callout Konsep Python di Bagian 1, lalu hitung berapa persen yang cocok.
7. **Interpretasi:** Tutup dengan dua paragraf. Paragraf pertama membaca petanya: kelurahan mana yang paling terlayani satuan pendidikan, mana yang paling timpang, dan apakah kelurahan berkepadatan bangunan tinggi pada modul tadi juga berkepadatan sekolah tinggi. Paragraf kedua membaca angka langkah 6: apa yang seharusnya dilakukan seorang perencana yang selama ini terbiasa menggabungkan data antar-instansi semata-mata dengan mencocokkan nama wilayah.

8 Pemanenan Data Mandiri Berbasis Agen (Agentic Data Harvesting) dan OpenStreetMap

! Sub-CPMK Pertemuan Ini

Mahasiswa mampu melakukan penarikan data perkotaan secara mandiri menggunakan teknik otomasi peramban berbasis agen (*Agentic Data Harvesting* via skill `browser-act`) atau mengekstraksi data jaringan jalan/fasilitas dari OpenStreetMap menggunakan *library osmnx*.

8.1 Pendahuluan

8.1.a Memahami Agen AI

Dalam era kecerdasan artifisial, peran seorang analis perkotaan mengalami lompatan mendasar. Sebagian besar dari kita mungkin sudah terbiasa berinteraksi dengan AI generatif dalam wujud **chatbot** (seperti ChatGPT atau Claude di peramban): pengguna mengetik pertanyaan atau perintah, lalu model memberikan jawaban teks dalam ruang obrolan satu arah. Dalam pola *chatbot*, AI bersifat pasif—ia hanya menunggu giliran bicara dan tidak memiliki akses langsung untuk mengutak-atik peramban, membuka aplikasi, atau mengumpulkan berkas di komputer Anda.

Paradigma baru yang kita pelajari pada modul ini adalah **Agen AI (AI Agent)**. Berbeda dengan *chatbot* biasa, agen AI adalah sistem otonom yang dibekali **perkakas kerja (tools)**, **kemampuan merencana (planning)**, dan **daya tindak (action capability)**. Ketika diberi instruksi terstruktur, agen dapat membuka peramban web sendiri, mengeklik tautan, melewati halaman demi halaman (*pagination*), mengatasi antarmuka berbasis JavaScript, mengekstrak data pasar properti yang tersebar, lalu menyusunnya langsung menjadi tabel data geospasial yang bersih di direktori kerja Anda.

Dengan kehadiran agen, perencana tidak lagi menghabiskan waktu sehari-hari menyalin data secara manual dari internet atau menulis kode perayap web (*web scraper*) rumit yang mudah rusak ketika tampilan web berubah. Kini perencana bertindak sebagai **Pengarah Data (Data Director)**: tugas utama Anda adalah merumuskan **Kerangka Acuan Kerja Data (Data Requirement Specification)** yang presisi—menentukan informasi apa yang dibutuhkan kota, batas wilayah studinya, derajat ketelitian spasial yang dituntut, dan format geospasial akhirnya—sementara proses penjelajahan dan pemanenan datanya diselesaikan secara mandiri oleh agen.

8.1.b Perangkat Lunak Agen AI

Kalau kita sudah mengenal ChatGPT, Gemini, Claude, atau DeepSeek, nama-nama tersebut sebenarnya mengacu pada **model bahasa** yang berinteraksi dengan kita. Perusahaan-perusahaan yang mengembangkan model tersebut, secara berturut-turut yakni OpenAI, Google, Anthropic, dan DeepSeek (kebetulan nama modelnya adalah nama perusahaannya), juga mengembangkan perangkat lunak agen masing-masing. OpenAI punya Codex, Google punya Antigravity, Anthropic punya Claude Code, dan DeepSeek sebentar lagi akan merilis perangkat lunak agennya sendiri bernama DeepSeek Harness.

8.2 Gambaran Umum Kasus

Ketika menyusun rencana tata ruang kawasan perdagangan dan jasa atau mengerjakan Tugas Besar studio, data resmi dari pemerintah daerah sering kali belum memuat dinamika pasar komersial terkini. Berapa harga sewa ruko per tahun di koridor Jalan Raden Intan? Berapa harga jual rata-rata ruko baru di koridor Jalan ZA Pagar Alam Kedaton? Di koridor mana saja ruko-ruko tersebut mengelompok?

Informasi pasar tersebut tersebar melimpah di portal properti publik seperti rumah123.com. Di saat yang sama, geometri koridor jalan dan sebaran fasilitas kota tersedia secara terbuka di OpenStreetMap. Pada modul ini, kita akan mempelajari dua gerbang pemanenan data mandiri:

1. **Gerbang Pertama: Pemanenan Data Berbasis Agen (*Agentic Data Harvesting*)**, di mana kita mengarahkan agen AI menggunakan **Prompt Satu Langkah (*One-Shot Prompt*)** dengan skill `browser-act` untuk memanen seluruh data listing ruko di Kota Bandar Lampung (baik ruko dijual maupun disewakan) lengkap dengan koordinat spasial (*Latitude & Longitude* 4 desimal), harga, dan luasnya.
2. **Gerbang Kedua: Locket Data Terbuka OpenStreetMap (*osmnx*)**, tempat jaringan jalan dan titik fasilitas kota dapat dipanen langsung berdasarkan koordinat dan radius wilayah untuk menjadi lapisan dasar (*base layer*) analisis spasial.

Di akhir modul, kedua lapisan data hasil panen mandiri ini akan dipadukan menjadi satu kesatuan peta sebaran komersial kota yang siap diintegrasikan ke dalam analisis studio perencanaan.

8.3 Instruksi

Buka proyek kerja Anda di **Antigravity**, aktifkan *kernel analitika-perkotaan*, dan kerjakan seluruh instruksi di bawah ini.

8.3.a Menyiapkan Mesin Agen dan Memeriksa Lingkungan

Sebelum memanggil agen, kita perlu memastikan lingkungan kerja Conda `analitika-perkotaan` telah memuat seluruh pustaka geospasial dan mesin pendukung peramban. Dalam ekosistem terpadu, seluruh dependensi (Python 3.12, Node.js runtime, GeoPandas, OSMnx, dan `browser-act-cli`) telah dikemas di dalam berkas konfigurasi `environment.yml`.

Periksa ketersediaan modul Python utama yang akan digunakan untuk memproses dan menganalisis dataset hasil panen agen.

```
1  import pandas as pd
2  import geopandas as gpd
3  import osmnx as ox
4  import folium
5
6  print("Versi Pandas      :", pd.__version__)
7  print("Versi GeoPandas:", gpd.__version__)
8  print("Versi OSMnx       :", ox.__version__)
9  print("Lingkungan siap digunakan untuk analisis geospasial!")
```

Baris 1

Pustaka analisis data tabular untuk memeriksa dan mengolah dataset properti.

Baris 2

Pustaka analisis geospasial untuk mengubah tabel berkoordinat menjadi objek spasial (*GeoDataFrame*).

Baris 3

Pustaka pemanen data jaringan jalan dan fasilitas wilayah dari OpenStreetMap.

Baris 4

Pustaka pembuat peta interaktif berbasis web (Leaflet.js).

Baris 6

Memeriksa versi Pandas yang terpasang di kernel aktif.

Baris 7

Memeriksa versi GeoPandas.

Baris 8

Memeriksa versi OSMnx.

Baris 9

Konfirmasi kesiapan lingkungan komputasi.

```
Versi Pandas      : 2.2.1
Versi GeoPandas: 1.1.2
Versi OSMnx       : 2.0.7
Lingkungan siap digunakan untuk analisis geospasial!
```

Pengetahuan Komputer

Otomasi Peramban Cerdas (Agentic Browser Automation)

Situs web modern umumnya menggunakan kerangka kerja JavaScript dinamis (*Client-Side Rendering* atau *Server-Side Hydration*). Jika diakses menggunakan kode penarik statis biasa (`requests`), halaman sering kali hanya mengembalikan kerangka kosong atau memicu blokir keamanan.

Dengan **skill `browser-act`**, agen AI berinteraksi langsung dengan mesin peramban web (*headless browser*). Agen dapat mensimulasikan gulir halaman (*scrolling*), berpindah halaman paginasi, membuka tab detail secara paralel, serta mengekstrak objek data terstruktur (termasuk tag koordinat geografis) persis seperti yang dilihat oleh manusia di layar monitor.

8.3.b Merumuskan Prompt Satu Langkah (One-Shot Prompt) untuk Pemanenan Data Pasar

Di studio PWK, jika Anda menugaskan surveyor dengan instruksi yang tidak jelas (misal: *'tolong cari ruko di Bandar Lampung'*), surveyor akan kebingungan: ruko di jalan mana, apakah yang dijual atau disewa, data apa saja yang dicatat, dan bagaimana format tabel laporannya. Sebaliknya, instruksi yang baik memuat **5 Unsur Kerangka Penugasan Data**:

1. **Peran & Tindakan:** Memanggil skill `/browser-act` untuk melakukan penarikan data menyeluruh.
2. **Sasaran URL & Paginasi:** Alamat portal dan instruksi menyisir seluruh nomor halaman.
3. **Ruang Lingkup Spasial:** Batas wilayah studi (Kota Bandar Lampung).

4. **Spesifikasi Variabel Wajib:** Identitas listing, lokasi kecamatan, koordinat *Latitude & Longitude* (4 desimal), harga numerik (IDR), tipe transaksi, luas tanah (LT), dan luas bangunan (LB).
5. **Format Berkas Keluaran:** Standar penyimpanan (CSV berpenyandian UTF-8 dan Excel XLSX).

💡 Formula Prompt Satu Langkah

Contoh One-Shot Prompt Mahasiswa di AntigraVity

Ketikkan perintah berikut pada kolom percakapan AntigraVity Anda:

```
1 /browser-act scrape seluruh ruko di Kota Bandar Lampung dari website https://www.rumah123.com/jual/bandar-lampung/ruko/ dan https://www.rumah123.com/sewa/bandar-lampung/ruko/. Ekstrak seluruh halaman paginasi untuk mendapatkan dataset terstruktur yang berisi id_listing, judul, kecamatan, latitude, longitude (4 desimal), harga_rp, tipe_transaksi (Jual/Sewa), luas_tanah_m2, luas_bangunan_m2, nama_agen, agensi, dan url. Simpan hasilnya ke ruko_bandar_lampung.csv dan ruko_bandar_lampung.xlsx.
```

8.3.c Bagian 3: Memuat dan Memeriksa Integritas Dataset Hasil Tangkapan Agen

Setelah agen AI menjalankan instruksi pemanenan dan menyimpan hasilnya, langkah pertama seorang perencana adalah memeriksa keutuhan data (*integrity check*).

1. Baca berkas dataset hasil panen agen `ruko_bandar_lampung.csv` dari folder dataset perkuliahan, lalu tampilkan ringkasan dimensi tabel dan tipe data kolomnya.

```
1 path_ruko = "../dataset-ganjil-2026-2027/ruko_bandar_lampung.csv"
2
3 df_ruko = pd.read_csv(path_ruko)
4
5 print("Jumlah Total Listing :", len(df_ruko))
6 print("Jumlah Kolom Data      :", df_ruko.shape[1])
7 print("\nDaftar Kolom Dataset :\n", list(df_ruko.columns))
```

Baris 1

Jalur berkas dataset ruko berkoordinat yang telah dipanen oleh agen.

Baris 3

Fungsi `pd.read_csv()` membaca berkas tabular ke dalam DataFrame Pandas.

Baris 5

Menampilkan jumlah total baris properti yang berhasil dipanen (525 unit ruko unik di Kota Bandar Lampung).

Baris 6

Menampilkan jumlah atribut variabel data.

Baris 7

Menampilkan daftar nama kolom dataset.

Jumlah Total Listing : 525
Jumlah Kolom Data : 23

Daftar Kolom Dataset :

```
['id_listing', 'judul', 'tipe_transaksi', 'kecamatan', 'lokasi_lengkap',  
'kota', 'latitude', 'longitude', 'sumber_koordinat', 'latitude_pinpoint_asli',  
'longitude_pinpoint_asli', 'harga_teks', 'harga_rp', 'periode_sewa',  
'luas_tanah_m2', 'luas_bangunan_m2', 'kamar_tidur', 'kamar_mandi', 'nama_agen',  
'agensi', 'deskripsi', 'url', 'waktu_scraping']
```

Tampilkan 5 baris pertama data untuk menginspeksi wujud nilai pada kolom identitas, koordinat spasial, harga, dan luas.

```
1 kolom_ringkas = [  
2     "id_listing", "judul", "tipe_transaksi", "kecamatan",  
3     "latitude", "longitude", "harga_rp", "luas_tanah_m2", "luas_bangunan_m2"  
4 ]  
5  
6 df_ruko[kolom_ringkas].head(5)
```

Baris 1

Variabel *list* berisi nama-nama kolom utama yang ingin diinspeksi (pola efisiensi kode sejak Modul 2).

Baris 6

Metode `.head(5)` menampilkan 5 baris teratas data.

	id_listing	judul	tipe_transaksi	kecamatan	latitude	longitude	harga_rp	luas_tanah_m2	luas_bangunan_m2
0	shs10137396	Ruko Lantai + Gudang SHM Kedaton	Jual	Kedaton	-5.3824	105.2586	8.000000e+02	625.0	900.0
1	shs10085689	Cepat Ruko Cocok Buat Usaha Kuliner De...	Jual	Sukarame	-5.4281	105.2430	1.500000e+02	149.0	160.0
2	shs9872679	Dijual Cepat Ruko Antasari	Jual	Sukarame	-5.4281	105.2430	6.800000e+01	85.0	160.0
3	shs10228348	Ruko Dijual	Jual	Sukarame	-5.3850	105.2950	2.500000e+02	228.0	384.0

	id_listing	judul	tipe_transaksi	kecamatan	latitude	longitude	harga	luas_bangunan_m2
4	shs456246	Ruko 3 Pintu @ Jalan Pattimura, Teluk Betung U...	Jual	Teluk Betung Utara	-5.4371	105.2578	3.750000e+09	192.0 576.0

Periksa distribusi jumlah ruko berdasarkan tipe transaksinya (Jual vs Sewa).

```
1 rekap_transaksi = df_ruko["tipe_transaksi"].value_counts()
2 print("Distribusi Listing Ruko Berdasarkan Transaksi:\n", rekap_transaksi)
```

Baris 1

Metode `.value_counts()` (Modul 4) menghitung frekuensi kategori transaksi.

Baris 2

Menampilkan jumlah unit ruko yang ditawarkan untuk dijual (289 unit) dan disewakan (236 unit).

```
Distribusi Listing Ruko Berdasarkan Transaksi:
tipe_transaksi
Jual      289
Sewa      236
Name: count, dtype: int64
```

8.3.d Bagian 4: Validasi Bounding Box dan Presisi Koordinat Spasial

Dalam perencanaan wilayah dan kota, kita tidak boleh menelan mentah-mentah angka koordinat dari internet. Koordinat harus diuji terhadap **Batas Wilayah Geografis (Bounding Box)** kota untuk memastikan tidak ada titik yang salah koordinat (misalnya koordinat nol atau koordinat kota lain).

i Pengetahuan Geospasial

Derajat Desimal dan Presisi 4 Angka di Belakang Koma

Pada sistem koordinat geografis WGS84 (derajat desimal):

- **1 desimal** ($\approx 0,1^\circ$) mewakili jarak sekitar **11,1 km**.
- **2 desimal** ($\approx 0,01^\circ$) mewakili jarak sekitar **1,1 km** (skala desa/kelurahan).
- **3 desimal** ($\approx 0,001^\circ$) mewakili jarak sekitar **110 meter** (skala blok permukiman).
- **4 desimal** ($\approx 0,0001^\circ$) mewakili jarak sekitar **11 meter** (skala kapling/tapak bangunan ruko).
- **5 desimal** ($\approx 0,00001^\circ$) mewakili jarak sekitar **1,1 meter**.

Format 4 desimal adalah standar emas untuk analisis peruntukan tata ruang komersial karena cukup presisi untuk membedakan persimpangan jalan dan blok ruko tanpa menimbulkan angka desimal semu (*false precision*).

Periksa batas minimum dan maksimum koordinat *Latitude* dan *Longitude* pada *dataset* ruko untuk memastikan seluruh titik berada dalam rentang Kota Bandar Lampung (Lintang: $-5,55^{\circ}$ s/d $-5,30^{\circ}$, Bujur: $105,15^{\circ}$ s/d $105,40^{\circ}$):

```
1  lat_min = df_ruko["latitude"].min()
2  lat_max = df_ruko["latitude"].max()
3  lng_min = df_ruko["longitude"].min()
4  lng_max = df_ruko["longitude"].max()
5
6  print(f"Rentang Latitude   : {lat_min:.4f} s/d {lat_max:.4f}")
7  print(f"Rentang Longitude  : {lng_min:.4f} s/d {lng_max:.4f}")
8
9  # Validasi Bounding Box Kota Bandar Lampung
10 di_dalam_bdl = (
11     (df_ruko["latitude"].between(-5.55, -5.30)) &
12     (df_ruko["longitude"].between(105.15, 105.40))
13 )
14
15 print("Persentase Titik Valid di Bandar Lampung :", (di_dalam_bdl.sum() /
len(df_ruko)) * 100, "%")
```

Baris 1

Menemukan nilai lintang paling selatan pada dataset.

Baris 2

Menemukan nilai lintang paling utara pada dataset.

Baris 3

Menemukan nilai bujur paling barat pada dataset.

Baris 4

Menemukan nilai bujur paling timur pada dataset.

Baris 6

Menampilkan rentang lintang berformat 4 desimal.

Baris 7

Menampilkan rentang bujur berformat 4 desimal.

Baris 15

Memverifikasi bahwa 100% data listing berada persis di dalam batas teritorial Kota Bandar Lampung.

```
Rentang Latitude   : -5.4750 s/d -5.3600
Rentang Longitude  : 105.1891 s/d 105.3200
Persentase Titik Valid di Bandar Lampung : 100.0 %
```

Konversikan tabel DataFrame Pandas menjadi objek geospasial **GeoDataFrame** (*geopandas*) dengan sistem koordinat WGS84 (EPSG:4326).

```
1  # Mengubah koordinat numerik menjadi geometri titik Point (Modul 6)
2  gdf_ruko = gpd.GeoDataFrame(
```

```

3     df_ruko,
4     geometry=gpd.points_from_xy(df_ruko["longitude"], df_ruko["latitude"]),
5     crs="EPSG:4326"
6 )
7
8 print("Tipe Objek Spasial :", type(gdf_ruko))
9 print("Sistem Koordinat (CRS) :", gdf_ruko.crs)

```

Baris 2

Membangun objek GeoDataFrame geospasial.

Baris 3

Memasukkan atribut tabel data ruko sebagai atribut tabular.

Baris 4

Fungsi `points_from_xy()` membentuk geometri titik dari kolom bujur (X) dan lintang (Y).

Baris 5

Menetapkan Sistem Referensi Koordinat WGS84 (EPSG:4326).

Baris 8

Mengonfirmasi bahwa tabel telah berubah menjadi lapisan spasial vektor titik.

Baris 9

Menampilkan identitas CRS aktif.

```

Tipe Objek Spasial : <class 'geopandas.geodataframe.GeoDataFrame'>
Sistem Koordinat (CRS) : EPSG:4326

```

8.3.e Bagian 5: Memanen Konteks Wilayah dari OpenStreetMap (`osmnx`)

Setelah memiliki titik-titik ruko, seorang perencana membutuhkan lapisan konteks jaringan kota: **jalan apa saja yang menghubungkan ruko-ruko tersebut?**

Pustaka **OSMnx** memungkinkan kita mengunduh seluruh koridor jaringan jalan dari server OpenStreetMap secara langsung hanya dengan menyebutkan nama kota atau titik koordinat pusat aktivitas.

1. Unduh jaringan jalan kendaraan bermotor di pusat aktivitas komersial Kota Bandar Lampung (Tanjung Karang) dalam radius 2.000 meter.

```

1 titik_pusat_bdl = (-5.4150, 105.2550)
2 radius_meter = 2000
3
4 # Mengunduh graf topologi jaringan jalan dari OpenStreetMap
5 graf_jalan = ox.graph_from_point(
6     titik_pusat_bdl,
7     dist=radius_meter,
8     network_type="drive"
9 )

```

```

10
11 # Mengonversi graf menjadi GeoDataFrame ruas jalan (edges)
12 gdf_simpang, gdf_ruas = ox.graph_to_gdfs(graf_jalan)
13
14 print("Jumlah Simpang Jalan Terdeteksi :", len(gdf_simpang))
15 print("Jumlah Ruas Jalan Terdeteksi    :", len(gdf_ruas))

```

Baris 1

Koordinat pusat komersial Tanjung Karang Kota Bandar Lampung berformat 4 desimal.

Baris 2

Radius pemanenan jaringan jalan (2 kilometer).

Baris 5

Fungsi `ox.graph_from_point()` mengirimkan kueri ekstraksi jaringan ke Overpass API OpenStreetMap.

Baris 6

Parameter titik koordinat pusat lingkaran studi.

Baris 7

Parameter jarak radius pencarian dalam satuan meter.

Baris 8

Filter tipe jaringan jalan untuk kendaraan bermotor (`drive`).

Baris 12

Fungsi `ox.graph_to_gdfs()` membongkar topologi jaringan menjadi dua GeoDataFrame terpisah: titik simpang (`nodes`) dan garis ruas jalan (`edges`).

Baris 14

Menampilkan jumlah persimpangan jalan.

Baris 15

Menampilkan jumlah segmen ruas jalan yang berhasil dipanen.

```

Jumlah Simpang Jalan Terdeteksi : 2661
Jumlah Ruas Jalan Terdeteksi    : 6090

```

```

C:\Users\LENOVO\AppData\Roaming\Python\Python314\site-
packages\geopandas\geodataframe.py:1969: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!

```

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
super().__setitem__(key, value)
```

C:\Users\LENOVO\AppData\Roaming\Python\Python314\site-packages\geopandas\geodataframe.py:1969: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
super().__setitem__(key, value)
```

C:\Users\LENOVO\AppData\Roaming\Python\Python314\site-packages\geopandas\geodataframe.py:1969: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
super().__setitem__(key, value)
```

8.3.f Bagian 6: Sintesis Spasial: Memadukan Sebaran Ruko dengan Jaringan Kota

Langkah pamungkas dalam pemanenan data adalah melakukan sintesis spasial: memadukan titik-titik ruko hasil panen agen AI dengan jaringan koridor jalan hasil ekstraksi OpenStreetMap ke dalam sebuah peta interaktif.

1. Buat peta interaktif Folium yang menampilkan koridor jalan dan titik sebaran ruko (dibedakan warnanya antara ruko dijual dan ruko disewakan).

```
1  # Membuat kanvas peta interaktif terpusat di Bandar Lampung
2  peta_interaktif = folium.Map(
3      location=[-5.4150, 105.2550],
4      zoom_start=13,
5      tiles="CartoDB positron"
6  )
7
8  # Menambahkan garis jaringan jalan OSM sebagai latar belakang abu-abu
9  folium.GeoJson(
10     gdf_ruas.to_json(),
11     style_function=lambda x: {"color": "#94a3b8", "weight": 1.5, "opacity": 0.7}
12 ).add_to(peta_interaktif)
13
14 # Menambahkan titik ruko dengan warna pembeda (Biru = Jual, Oranye = Sewa)
15 for idx, baris in gdf_ruko.head(100).iterrows():
16     warna = "#2563eb" if baris["tipe_transaksi"] == "Jual" else "#f97316"
17     folium.CircleMarker(
18         location=[baris["latitude"], baris["longitude"]],
19         radius=5,
20         color=warna,
21         fill=True,
22         fill_color=warna,
23         fill_opacity=0.8,
24         popup=f"<b>{baris['judul']}</b><br>Kecamatan: {baris['kecamatan']}<br>
25         Harga: {baris['harga_teks']}<br>LT/LB: {baris['luas_tanah_m2']} m² /
26         {baris['luas_bangunan_m2']} m²"
27     ).add_to(peta_interaktif)
28
29 # Menampilkan peta
30 peta_interaktif
```

Baris 2

Menginisialisasi objek peta interaktif Folium.

Baris 3

Titik koordinat jangkar pusat peta.

Baris 4

Tingkat perbesaran awal (*zoom level*).

Baris 5

Menggunakan tema peta terang minimalis (*CartoDB positron*) agar data tematik terlihat kontras.

Baris 9

Menambahkan lapisan vektor jalan OpenStreetMap.

Baris 10

Mengekspor GeoDataFrame jalan ke format GeoJSON.

Baris 15

Perulangan `for` untuk memetakan titik-titik ruko.

Baris 16

Logika penentuan warna simbol: Biru untuk Ruko Dijual, Oranye untuk Ruko Disewakan.

Baris 28

Menampilkan peta interaktif langsung di dalam sel notebook.

```
<folium.folium.Map at 0x18c5fcb9e80>
```

Setelah peta interaktif selesai ditampilkan dan divalidasi:

2. Ekspor dataset hasil validasi spasial ke format standar geospasial (GeoJSON dan CSV) agar dapat langsung dibuka di QGIS atau perangkat lunak GIS studio lainnya.

```
1  output_csv = "../dataset-ganjil-2026-2027/ruko_bdl_spasial.csv"
2  output_geojson = "../dataset-ganjil-2026-2027/ruko_bdl_spasial.geojson"
3
4  gdf_ruko.to_csv(output_csv, index=False, encoding="utf-8-sig")
5  gdf_ruko.to_file(output_geojson, driver="GeoJSON")
6
7  print(f"✓ Dataset CSV berhasil disimpan      : {output_csv}")
8  print(f"✓ Dataset GeoJSON berhasil disimpan : {output_geojson}")
```

Baris 1

Jalur berkas keluaran format CSV.

Baris 2

Jalur berkas keluaran format GeoJSON.

Baris 4

Metode `.to_csv()` mengekspor tabel lengkap dengan koordinat 4 desimal berpenyandian UTF-8.

Baris 5

Metode `.to_file()` mengekspor lapisan spasial vektor ber-CRS WGS84 yang siap diimpor ke QGIS atau ArcGIS.

```
✓ Dataset CSV berhasil disimpan      : ../dataset-ganjil-2026-2027/
ruko_bdl_spasial.csv
✓ Dataset GeoJSON berhasil disimpan : ../dataset-ganjil-2026-2027/
ruko_bdl_spasial.geojson
```

8.4 Latihan Mandiri

Untuk memperdalam keterampilan memanen data mandiri sebagai bekal Tugas Besar studio, kerjakan penugasan berikut:

1. **Pemanenan Data Properti Wilayah Studi Kelompok:** Gunakan formula **Prompt Satu Langkah (One-Shot Prompt)** yang telah dipelajari pada Bagian 2 untuk memanen data properti (rumah hunian

atau ruko komersial) pada **wilayah studi kelompok studio Anda masing-masing** di Provinsi Lampung (misalnya: Kota Metro, Kabupaten Pringsewu, atau Kabupaten Lampung Selatan).

2. **Validasi Bounding Box dan Presisi Spasial:** Muat dataset hasil panen agen ke dalam Pandas, tentukan batas *Bounding Box* koordinat wilayah studi Anda, dan verifikasi bahwa seluruh koordinat memiliki presisi **4 angka di belakang koma (4 desimal)**.
3. **Pemanenan Jaringan Jalan OSMnx:** Gunakan `osmnx` untuk mengunduh jaringan jalan kendaraan bermotor pada pusat kawasan perkotaan wilayah studi Anda dalam radius minimal 1.500 meter.
4. **Sintesis Peta Interaktif:** Susun peta interaktif *Folium* yang menumpangsusunkan titik properti hasil panen agen di atas jaringan jalan OpenStreetMap, lalu simpan peta tersebut ke dalam berkas HTML (`peta_studi_mandiri.html`) dan simpan dataset akhirnya ke format GeoJSON.